사용자 매뉴얼

v.1.1.10 (build 20250917)

목차

배틀싸피 소개
배틀싸피 실행
초기 화면
도움말 보기
과목 평가(Stage Mode)
    사용자 인증
    연습하기
    소스 코드 확인
    과목 평가 응시
팀 대결(Team Battle Mode)
    사용자 인증
    대기 화면
    대결 화면
기본코드 활용 방법
    C언어
    C++
    Python
    Java
입출력 데이터의 이해
    입력 데이터
        첫 번째 행
        H개의 행 (맵)
        M개의 행 (아군)
        E개의 행 (적군)
        S개의 행 (암호문)
    출력 데이터(=커맨드)
    메가 포탄 얻기 (보급 시설)
        카이사르 암호
※ 별첨. 사용 가능한 커맨드 모음

배틀싸피 소개

배틀싸피는

알고리즘 해결 능력을

게임을 통해 시각적으로 이해하고 해결하는

게이미피케이션 과정입니다.

배틀싸피라는 시뮬레이션 환경에서는

여러분이 제어할 수 있는 탱크가 한 대 제공되며

변화하는 각 상황에 대응하는 알고리즘 코드 하나로 탱크를 조종하여

여러분에게 주어진 미션을 해결해야 합니다.

여러분이 작성한 알고리즘 코드가

눈과 손으로 직접 제어하는 것을 대신하여

스스로 지능적으로 알맞은 동작을 수행할 수 있도록

나름의 전략과 알고리즘을 잘 구현하는 것이 중요합니다.

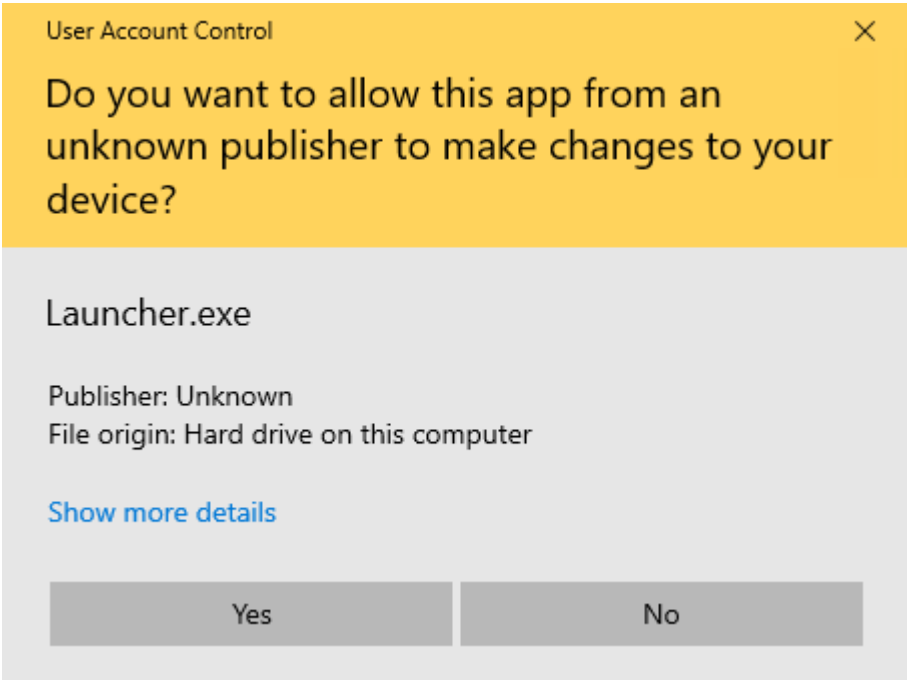
배틀싸피 실행

- 다운로드 받은 배틀싸피 압축파일을 압축 해제합니다.
  (아이콘 모양은 PC의 기본 압축프로그램에 따라 다를 수 있습니다.)

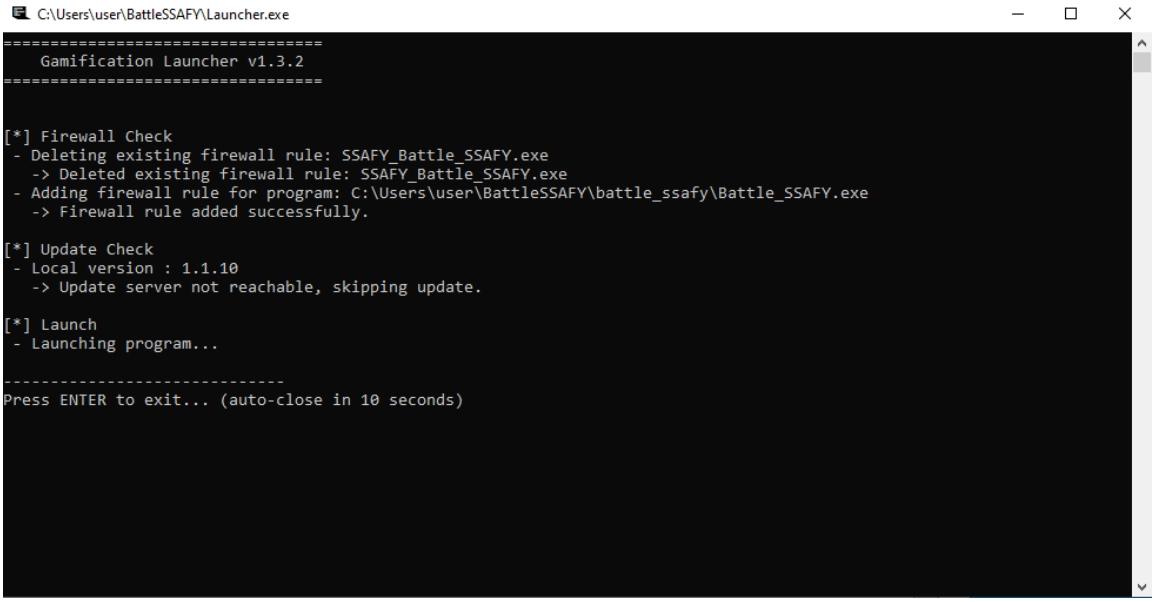
BattleSSAFY-v1.x.x.zip

- 압축 해제 후 **Battle_SSAFY.exe** 아이콘을 더블 클릭하여 실행합니다.

- 관리자 권한을 묻는 화면이 나오면 확인을 클릭합니다.



- Launcher 콘솔에서 방화벽, 업데이트, 중복 실행 체크 후에 자동으로 배틀싸피 메인 화면으로 넘어갑니다. 넘어가지 않을 경우에는 콘솔 화면의 출력 내용을 확인해주세요.
  - 업데이트가 진행 중일 때: 완료될 때까지 대기
  - 중복 실행 및 기타 예기치 못한 오류일 때: 출력된 오류 메시지를 참고하여 PC 상태 확인 후 재실행

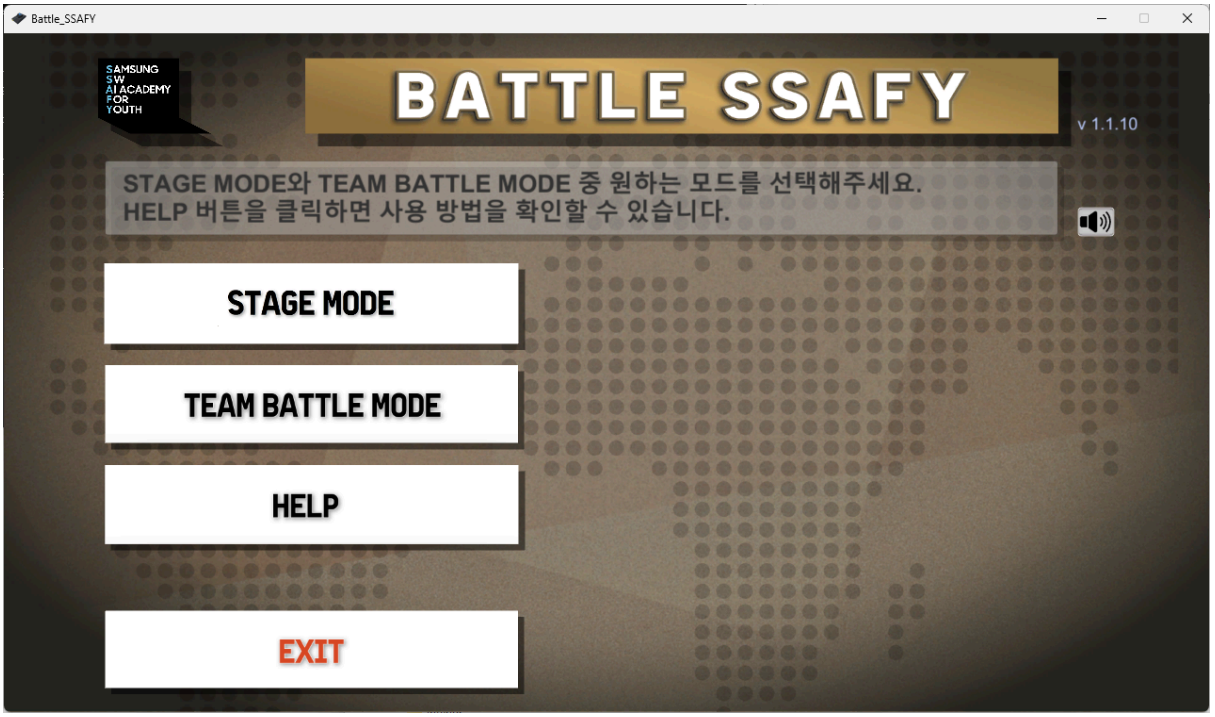


초기 화면

Launcher를 통해 배틀싸피가 실행되고 나면 다음과 같이 **초기화면**이 나타납니다.

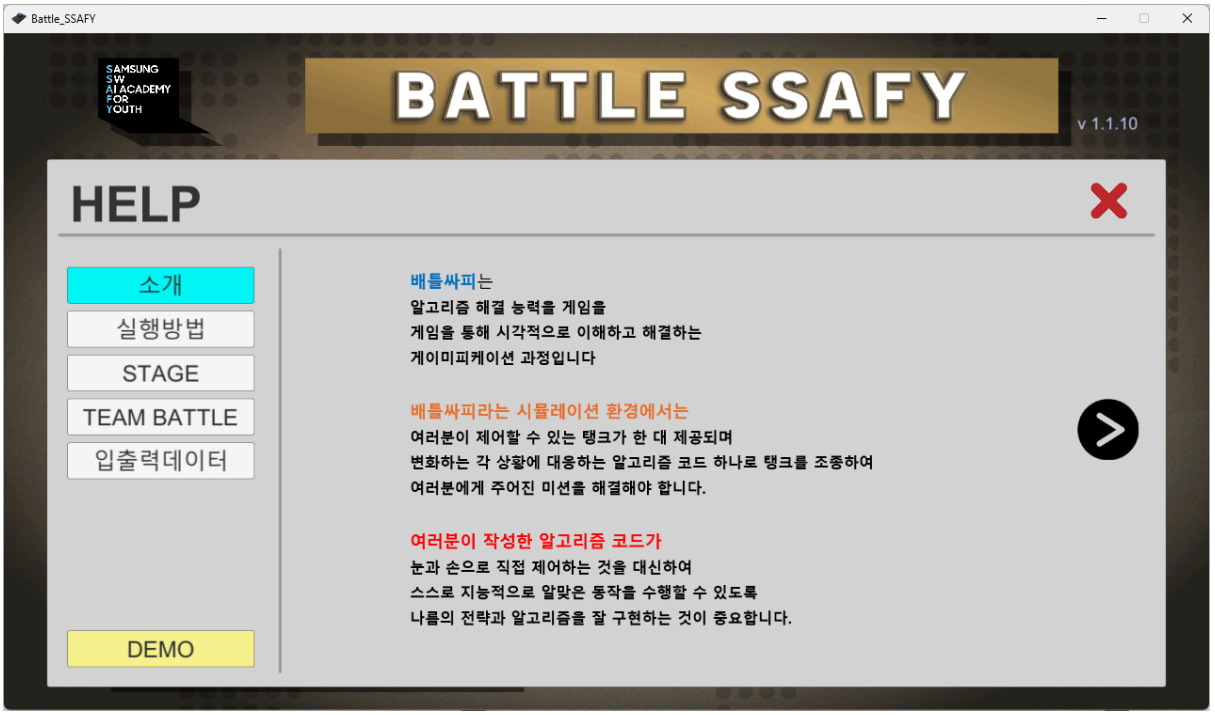
각 버튼을 클릭하여 원하는 메뉴에 진입할 수 있습니다.

- **STAGE MODE: 과목 평가를 위한 메뉴**
- **TEAM BATTLE MODE: 대항전을 위한 메뉴**
- **HELP: 배틀싸피에 대한 도움말 메뉴**
- **EXIT: 프로그램 종료**
- **우측 상단 볼륨 아이콘: 각 MODE 진입 시 출력되는 배경음/효과음을 미리 On/Off하는 버튼**



도움말 보기

HELP 버튼을 클릭하면 배틀싸피를 진행하는 데에 필요한 안내사항을 확인할 수 있습니다.



DEMO 버튼을 클릭하면 배틀싸피의 과목평가 또는 대항전 모드에 진입하기 전에 미리 탱크를 제어하는 데에 필요한 기본 커맨드를 확인해볼 수 있습니다.

배틀싸피와 연결하는 **소스 코드(클라이언트)**는 배틀싸피와 소켓 통신할 수 있는 코드가 구현된 상태로 제공됩니다. 소스 코드 내에 정의된 **output 변수**에 탱크를 제어할 커맨드를 문자열 형태로 넣어주고, 이를 인자로 하여 **submit( ) 함수를 호출**하면 배틀싸피에 해당 커맨드를 전달합니다.

DEMO 기능을 통해 소스코드를 살펴보기 전에 배틀싸피의 탱크가 이동/발사 커맨드(명령어)를 어떻게 이해하는지 살펴볼 수 있습니다.

output 입력상자에 U A를 입력하고 submit 버튼을 누르면 탱크가 전진하는 것을 볼 수 있습니다.

- A: **현재 바라보는 방향**으로 **전진** 가능하나 가급적 방향으로 명시할 것을 권장
- U A: 화면의 **위쪽** 방향으로 회전 후 **전진**
- D A: 화면의 **아래쪽** 방향으로 회전 후 **전진**
- L A: 화면의 **왼쪽** 방향으로 회전 후 **전진**
- R A: 화면의 **오른쪽** 방향으로 회전 후 **전진**

다음으로 **output 입력상자에 U F를 입력하고 submit 버튼**을 누르면 탱크가 회전 후 포탄을 발사하는 것을 볼 수 있습니다.

- F: 현재 바라보는 방향으로 **일반 포탄 발사**
- U/D/L/R F: 화면의 **상/하/좌/우** 방향으로 회전 후 **일반 포탄 발사**

배틀싸피의 커맨드는 **최대 3개의 알파벳**까지 결합할 수 있습니다.(사이 띄어쓰기)

- **1개일 때**
  - **[행동]**만 지시할 때
    - 예: A(전진), F(발사), S(대기)
- **2개일 때**
  - **[방향] + [행동]**을 지시할 때
    - 예: U A(위쪽으로 전진), R F(오른쪽으로 발사)
  - **[행동] + [옵션]**을 지시할 때
    - 예: F M(메가 포탄 발사), G HELLO(HELLO란 문구로 암호 해독 시도)
- **3개일 때**
  - **[방향] + [행동] + [옵션]**을 지시할 때
    - 예: L F M(왼쪽으로 메가 포탄 발사)

쉽게 말하자면 **[행동] 커맨드는 필수로, 앞에는 [방향]이 추가될 수 있고 뒤에는 [옵션]이 추가될 수 있는 형태**라고 생각하면 되겠습니다.

DEMO 화면에 R F M을 입력하여 한 번 더 결과를 확인해 봅시다.

빨간색으로 구분되는 메가 포탄이 발사되는 것을 볼 수 있습니다.

전체 커맨드 열람표는 **별첨**을 참고해 주세요.

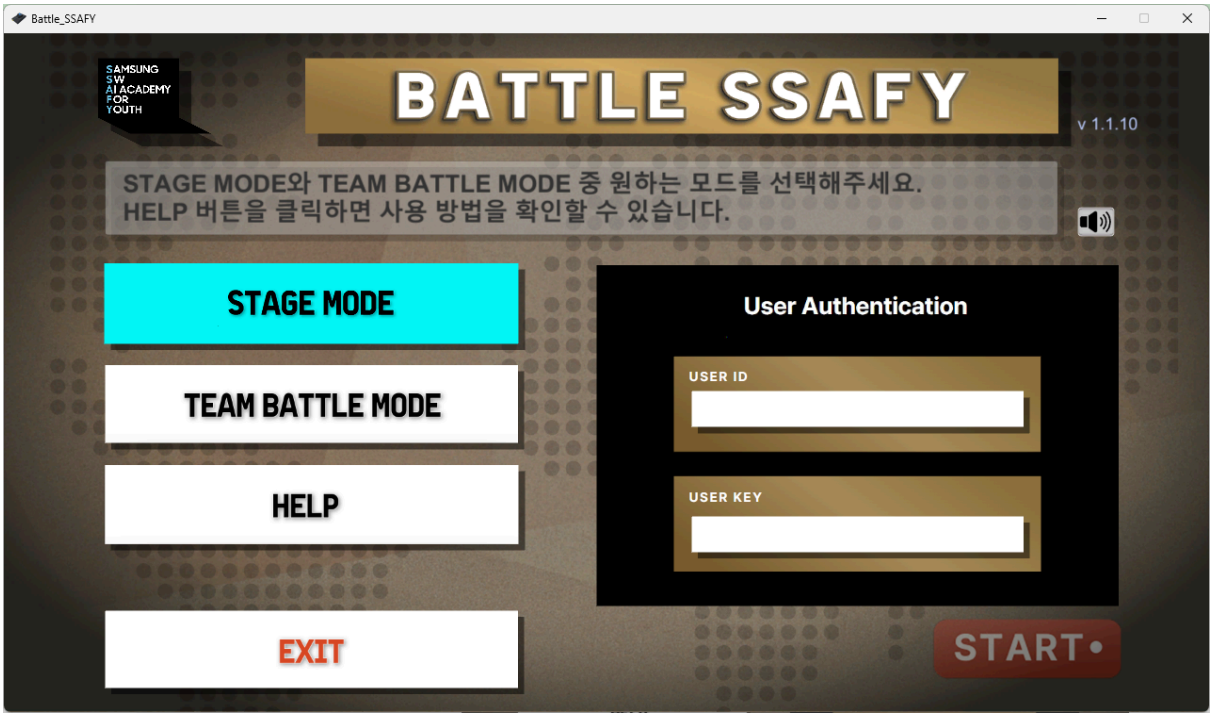
도움말은 과목 평가, 대항전 모드에 진입한 후에도 HELP 버튼을 클릭하여 확인할 수 있지만, **DEMO 기능은 초기 화면의 HELP 메뉴로 진입했을 때에만 사용**할 수 있다는 점에 유의해주세요.

과목 평가(Stage Mode)

사용자 인증

STAGE MODE 버튼을 클릭하면 사용자 인증창이 나타납니다.

SSAFY GIT에서 제공하는 팀ID, 팀KEY로 인증할 수 있습니다.

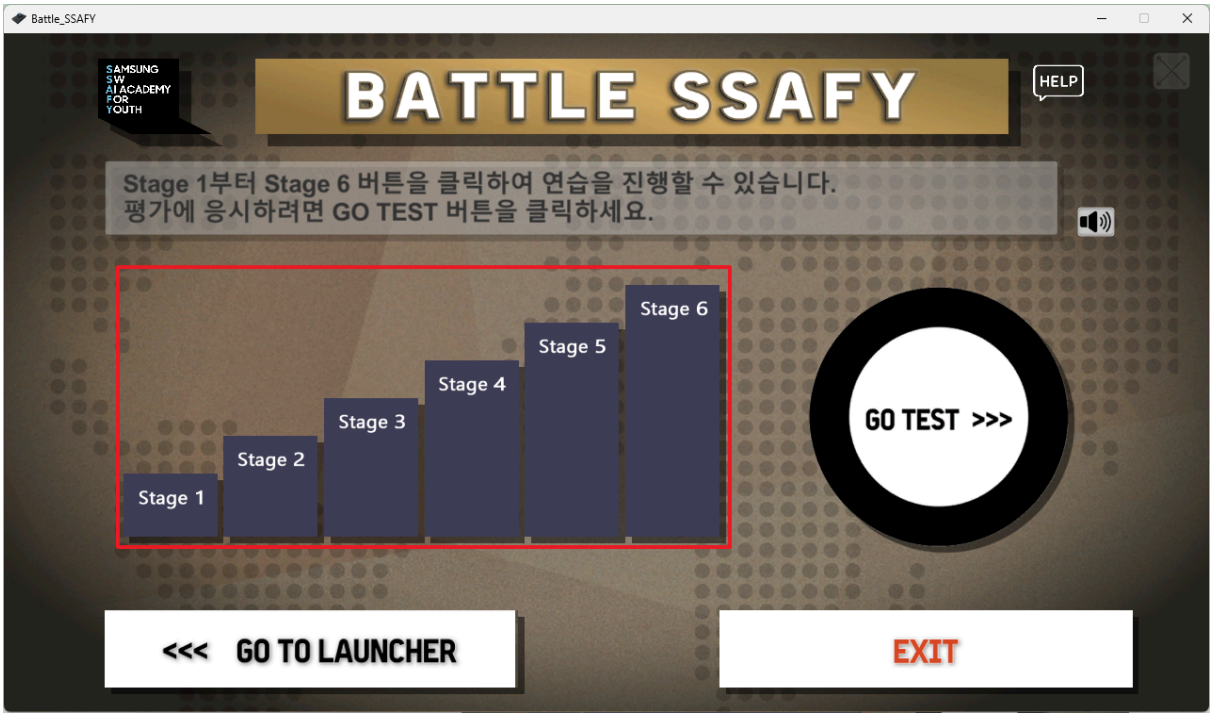


이제 스테이지를 선택할 수 있는 화면을 볼 수 있습니다.

연습하기

스테이지는 총 6단계로, 청색 계단 형태로 된 각 Stage 버튼을 클릭하면 **개별 Stage 단위로 연습**해볼 수 있습니다. 각 Stage 버튼은 허용된 기간에만 활성화되며, 이에 관한 자세한 사항은 사무국 안내를 참고해주시기 바랍니다.

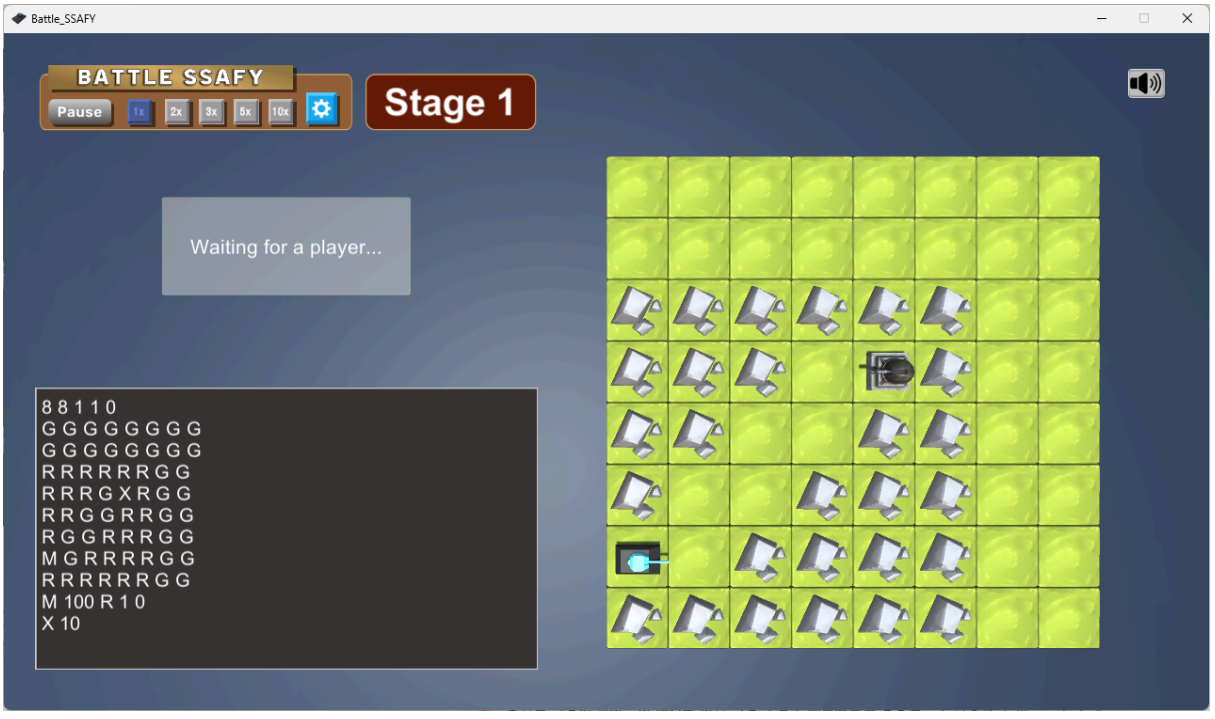
각 Stage 버튼을 눌러 연습하는 것에는 순서 상관 없이 무제한으로 할 수 있으며, 진행 상황이나 결과는 어디에도 기록되지 않습니다.



Stage 1을 클릭하면 화면이 전환됩니다.

Waiting for a player... 메시지는 배틀싸피 프로그램이 소켓을 열고 클라이언트의 TCP 접속을 기다리는 상태라는 의미로, 소스 코드를 실행하면 통신이 시작됩니다.

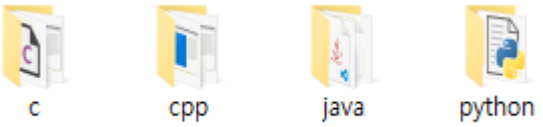
화면 우측에는 실제 탱크가 움직이는 필드 화면이 나타납니다.



⚠️ 소켓 통신을 위해 Windows 방화벽 설정이 선행되어야 하며, 이는 Launcher.exe를 통해 자동 설정됩니다. 통신에 문제가 있는 경우 프로그램을 닫고 Launcher.exe를 다시 실행해주세요.

소스 코드 확인

제공된 소스 코드 압축파일을 해제하면 C, C++, Python, Java 각각의 코드 파일을 볼 수 있습니다.



각 코드 파일은 **알고리즘 구현부** 및 **배틀싸피와의 통신부**가 모두 포함된 단일 파일로 구성되어 있습니다.

각 언어에 맞는 IDE 또는 VS Code 등으로 파일을 열어 수정하고 실행할 수 있습니다.

SSAFY 과정을 고려하여 각 코드 파일은 다음 IDE에서 실행에 문제가 없음이 검증 완료된 상태입니다.

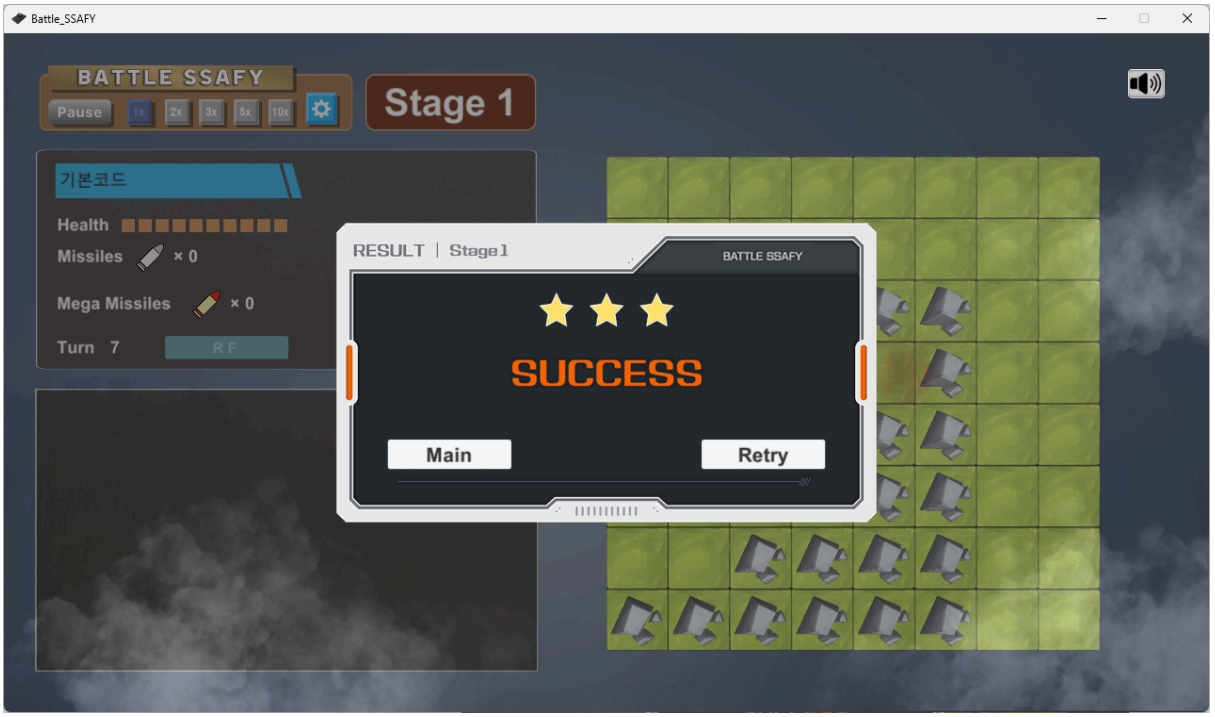
- C, C++: VS Express 2017 for Windows Desktop (기본 내장 C 컴파일러)
- Python: PyCharm Community Edition 2024.2.3 (Python 3.9+)
- Java: Eclipse IDE 2024-06 (JDK 17+)

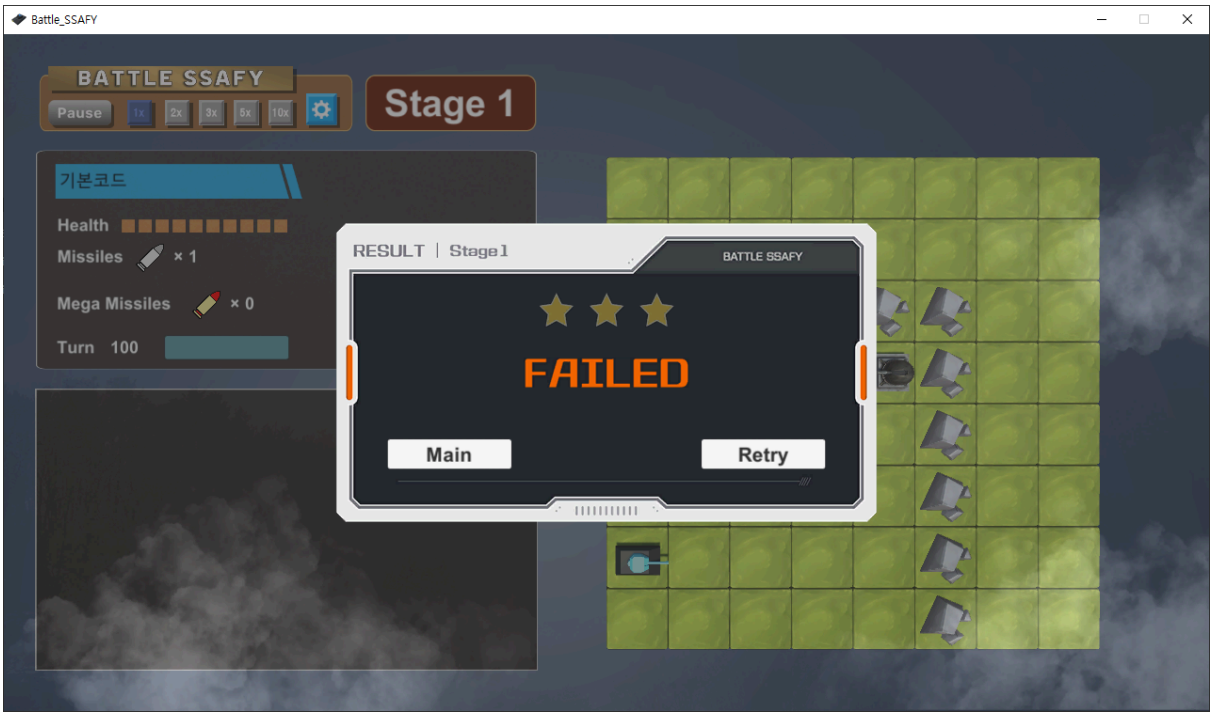
본인이 원하는 코드 파일을 익숙한 개발 환경에서 열어 실행하면 배틀싸피 화면이 움직이는 것을 볼 수 있습니다.

제공된 소스 코드는 실행된 Client가 배틀싸피로부터 맵 정보를 받아 콘솔에 출력하고, 대표적인 경로 탐색 중 하나인 BFS의 기본적인 구현 형태로써 경로 탐색 후 포탑을 공격하도록 작성되어 있습니다.

해당 스테이지를 **기준 턴 수 이내로 클리어**하거나 **일정 턴 수를 초과**하면 스테이지 종료창이 나타납니다.

진행 도중 **시간 초과(60초)** 또는 **소스 코드의 런타임 오류** 등으로 인해 연결이 끊긴 경우에도 더이상 진행할 수 없으므로 실패로 종료됩니다.





Main 버튼을 눌러 이전의 스테이지 선택 화면으로 돌아가거나 **Retry 버튼**을 클릭하여 현재 스테이지를 다시 연습할 수 있습니다.

과목 평가 응시

GO TEST 버튼을 클릭하면 **과목 평가에 응시**합니다.

시작 방법은 스테이지 연습과 동일하게 여러분이 작성한 소스 코드를 실행하면 됩니다.

연습과 시험 응시의 다른 점은

- **연습**은 각 스테이지의 맵 구조를 파악하고 요구를 충족하는지 확인할 수 있도록 매번 코드를 실행했다면,
- **시험 응시**에서는 **처음 실행한 코드 하나로** 모든 스테이지가 자동 진행된다는 점입니다.

시험 응시에서는 다음 스테이지로 넘어가면 새로운 클라이언트 연결을 기다리는 것이 아니라, 기존 연결된 클라이언트에게 화면 상의 맵 데이터를 계속해서 보내줍니다.

⚠️ 즉, 여러분이 시험용으로 작성하는 코드는 특정 스테이지만을 충족하는 것이 아니라, 여러 가지 맵에서 주어진 환경과 변화를 인지하여 대응할 수 있는 코드여야 합니다.
어떤 환경에서도 미션을 해결할 수 있는 여러분만의 탱크봇(Bot)을 만든다고 생각하면 됩니다.

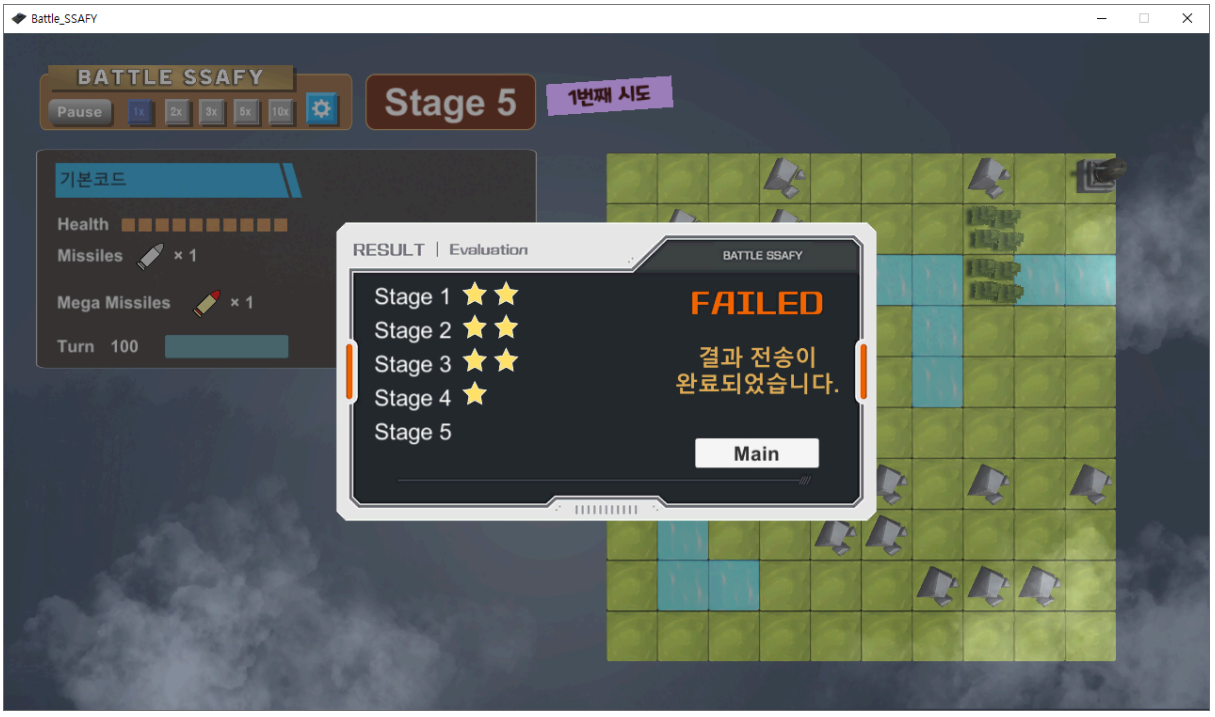
과목 평가는 스테이지1부터 스테이지6까지 진행하며, **한 스테이지당 2회씩** 총 12회의 도전을 하게 됩니다. 단, **중간에 실패하는 스테이지가 있다면** 해당 단계까지 획득한 점수만 기록하고 **과목 평가 응시가 종료**됩니다.

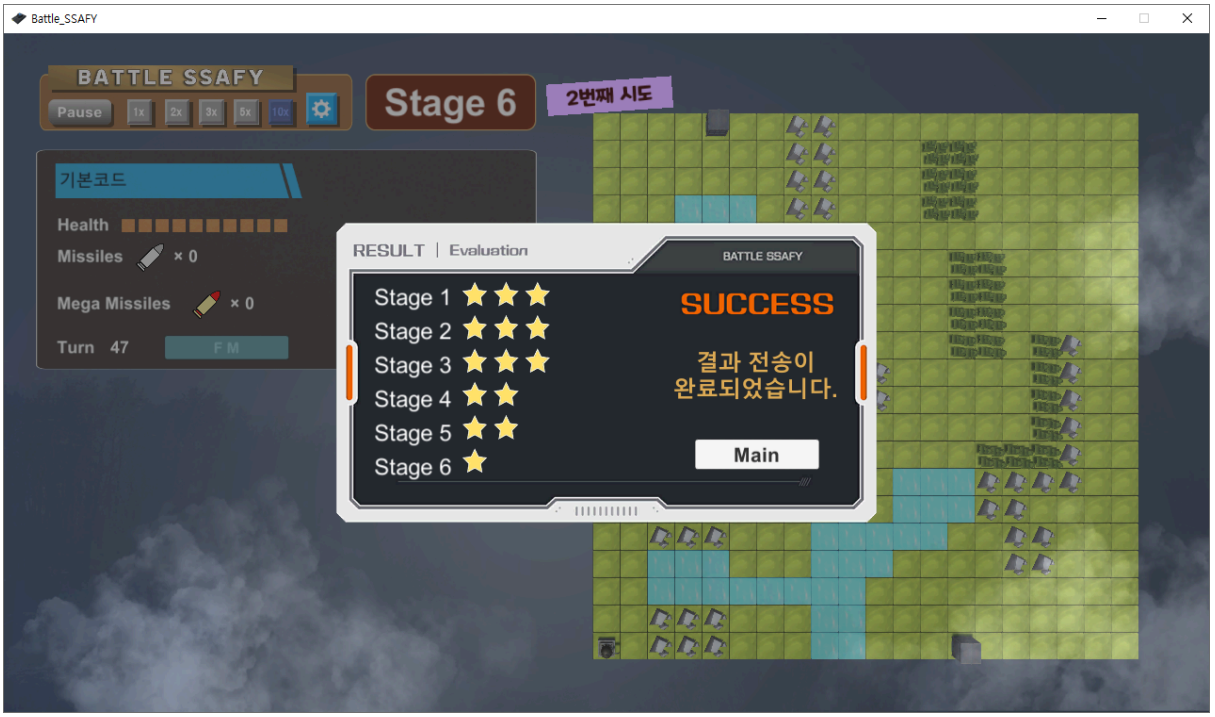
각 스테이지마다 **2번의 시도 후에 해당 스테이지 별 개수가 결정**되므로, 1번째 시도 후에 보이는 별 개수는 **일시적인 것**입니다.

2번째 시도 후에 1번째 시도와 비교하여 더 낮은 별 개수를 해당 스테이지의 성적으로 인정합니다. 이는 특정 맵에만 유리하게 코딩(하드코딩)하였거나 다양한 맵에 대한 검증이 되었는지 여부를 점수에 반영하기 위함입니다.

1번째와 2번째 시도 모두 동일한 개수의 별을 획득했다면 그대로 인정됩니다.

중도에 실패하거나 **스테이지6의 2번째 시도까지 성공**한 경우, **SSAFY GIT으로 결과 데이터가 전송**됩니다.





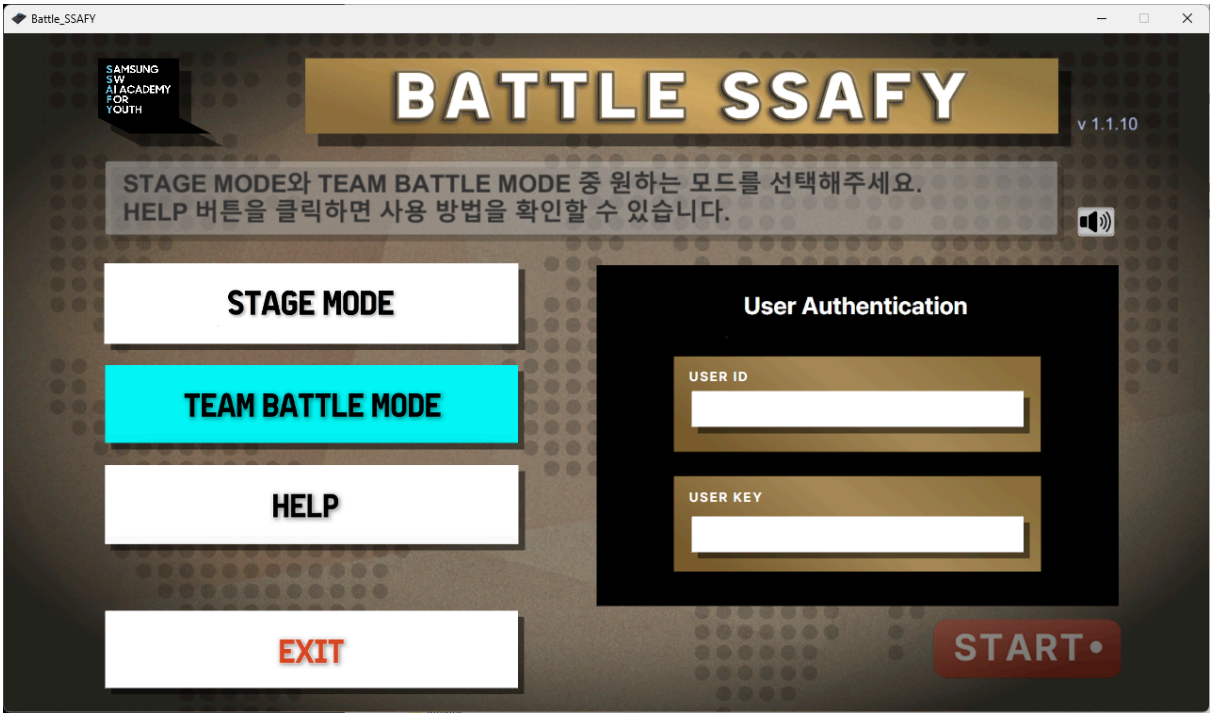
과목 평가 응시는 운영 방침에 따르며, 현재는 시도 횟수에 제한이 없도록 되어 있는 상태입니다.(더 나은 점수를 얻기 위해 코드를 개선해가며 반복 응시해도 됩니다.)

자세한 사항의 별도의 운영 안내를 참고하시기 바랍니다.

팀 대결(Team Battle Mode)

사용자 인증

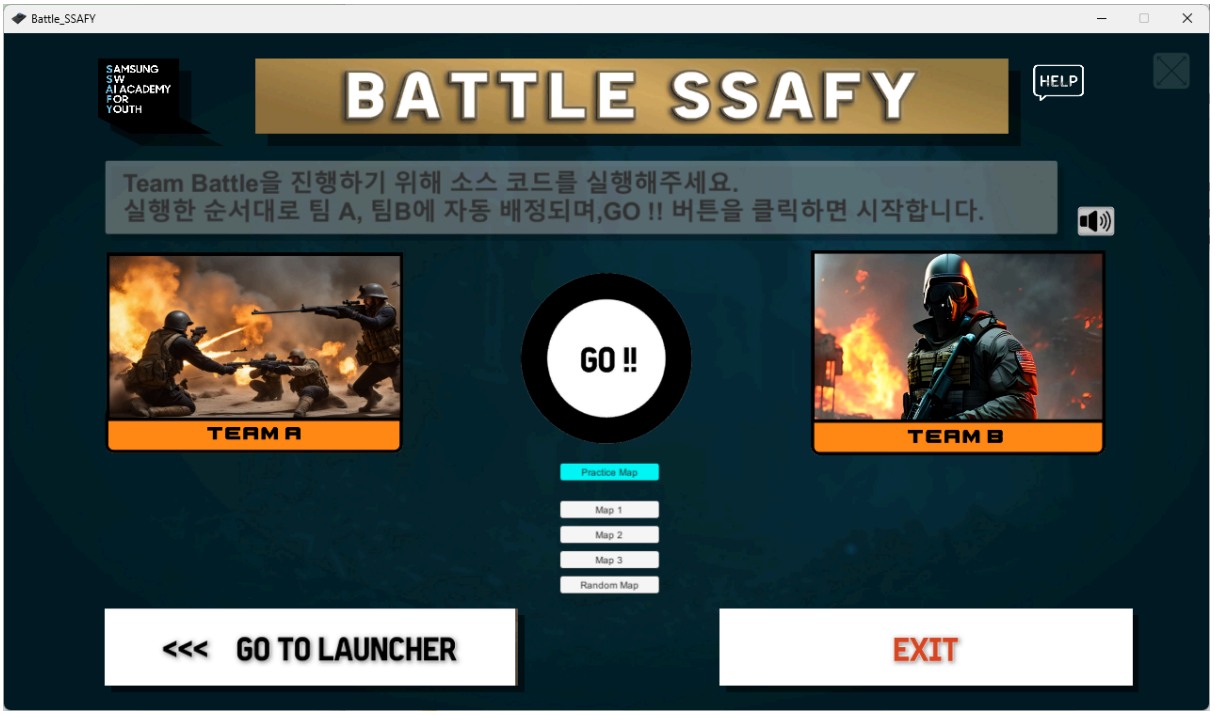
STAGE MODE와 마찬가지로 **TEAM BATTLE MODE 버튼**을 클릭하면 사용자 인증창이 나타납니다. SSAFY GIT에서 제공하는 팀ID, 팀KEY로 인증할 수 있습니다.



대기 화면

사용자 인증 후에 **팀 대결 대기 화면**을 볼 수 있습니다.

두 팀으로 나뉘어 플레이하게 되며, 한 팀에는 최소 1명에서 **최대 3명**까지 참가할 수 있습니다.

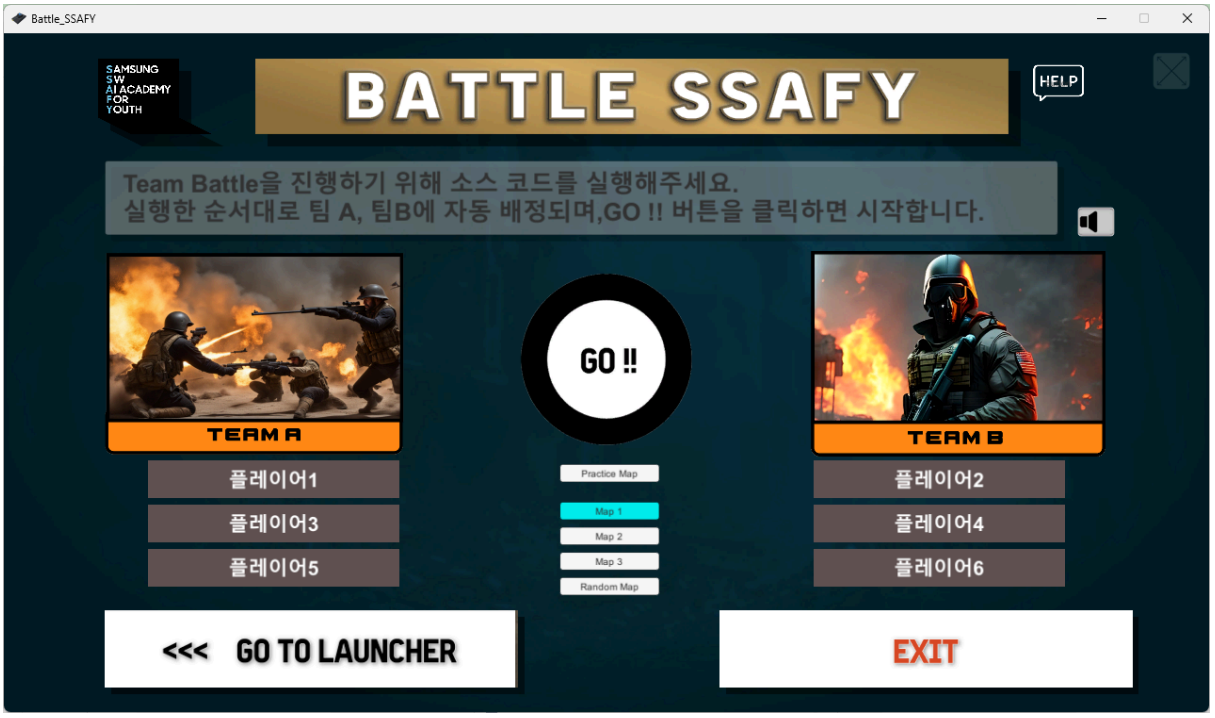


위와 같은 대기 상태에서 이전과 같은 방식으로 **소스 코드를 실행**하면 사용자 닉네임이 화면에 나타나면서 연결이 되었음을 알 수 있습니다.

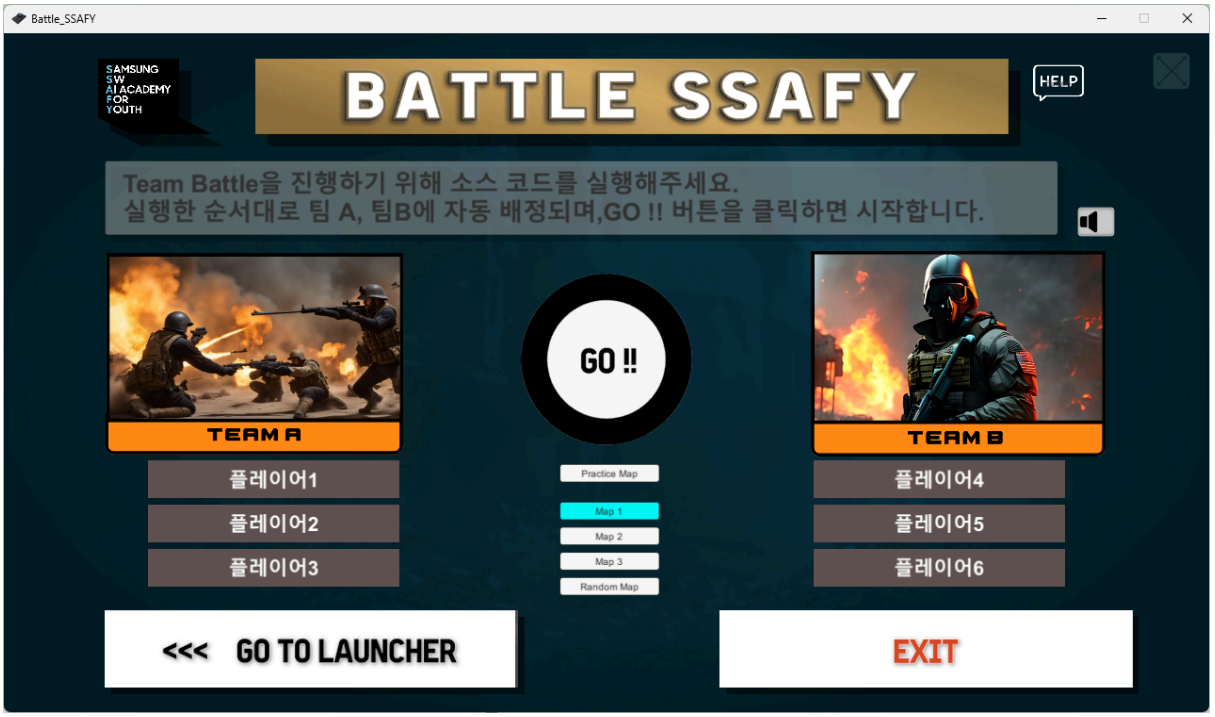
같은 방식으로 연이어 다른 코드들을 실행하면 순서대로 **TEAM A**, **TEAM B**에 들어가게 됩니다.

Client 코드를 실행할 때마다 소켓으로 연결되는 방식이므로, 동일한 소스 코드를 (또는 컴파일한 파일을) 연습을 위해 중복 실행해도 무방합니다. 다만 이런 경우에는 사용자 닉네임이 같아 구분이 어려울 수 있다는 점을 감안해야 합니다.

테스트 목적으로 같은 코드를 사용하더라도 닉네임을 다르게 하여 연결할 것을 권장합니다.



이미 실행한 플레이어의 **팀, 순서를 변경하고 싶다면** **닉네임 상자를 드래그하여 다른 닉네임 상자에 드롭**하면 서로 바뀌게 됩니다.



모든 준비가 끝났다면, **map을 선택**하고 **GO 버튼을 클릭**하여 다음으로 넘어 갑니다.

(양 팀이 동수일 때만 대결을 시작할 수 있습니다.)

제공되는 맵은 총 네 가지로, 다음과 같습니다.

- Practice Map: 출발 거점에서 가까운 위치에 다양한 지형이 있어 빠른 테스트 용도에 적합
- Map 1: 모래가 주요 경로를 덮고 있는 맵
- Map 2: 수풀이 우거진 맵
- Map 3: 배후 공간이 넓고 숨기 좋은 맵
- Random Map: Map 1 ~ Map 3 중 하나의 맵이 랜덤으로 선택되어 진행

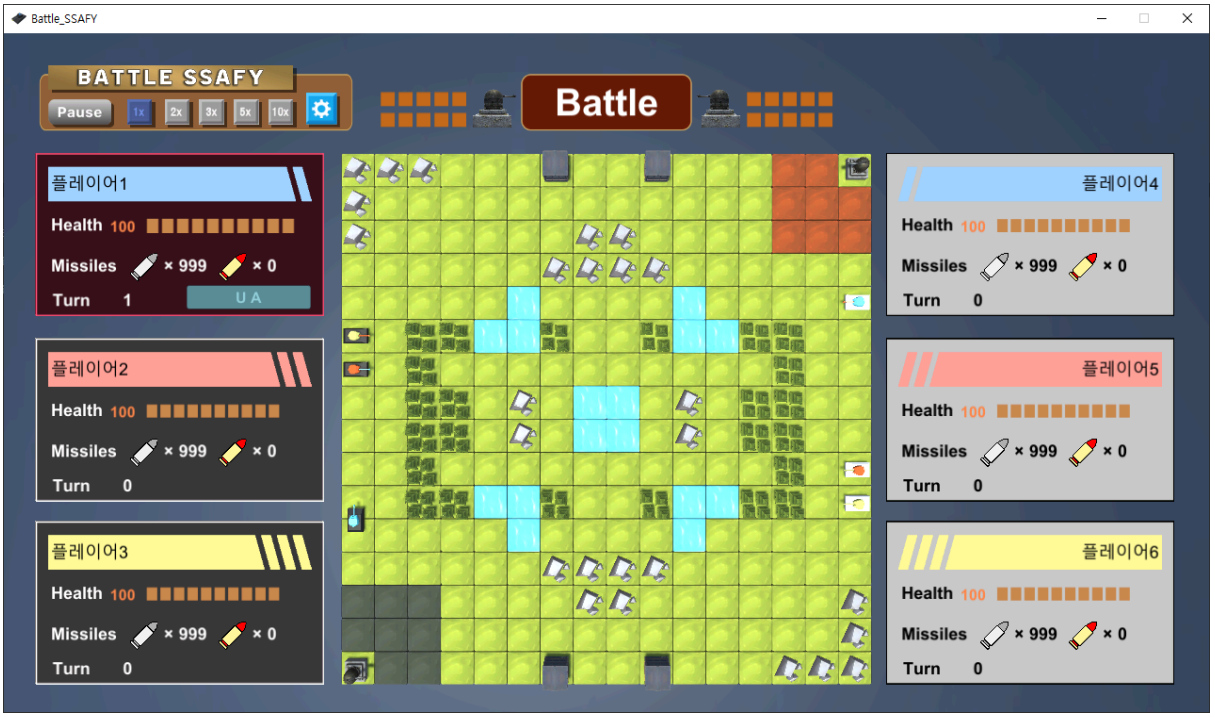
대결 화면

대결 화면에서는 각 탱크의 체력, 보유 포탄, 턴 수와 포탑의 체력 등 필요한 정보를 확인할 수 있습니다.

맵에서 움직이는 탱크는 **Team A의 탱크는 흑색**, **Team B의 탱크는 백색**으로 구분 가능합니다. 또한 각 팀 내에서의 순서에 따라 탱크의 상단이 **하늘색, 핑크색, 노란색 순**으로 구분됨

니다.(탱크의 색상과 플레이어 정보창의 색이 동일)

현재 차례를 진행중인 플레이어의 정보창은 **적색**으로 강조됩니다.



대결은 **다음 조건에 해당할 경우 즉시 종료**되며, **승패는 열거된 우선 순위에 따라** 판정됩니다.

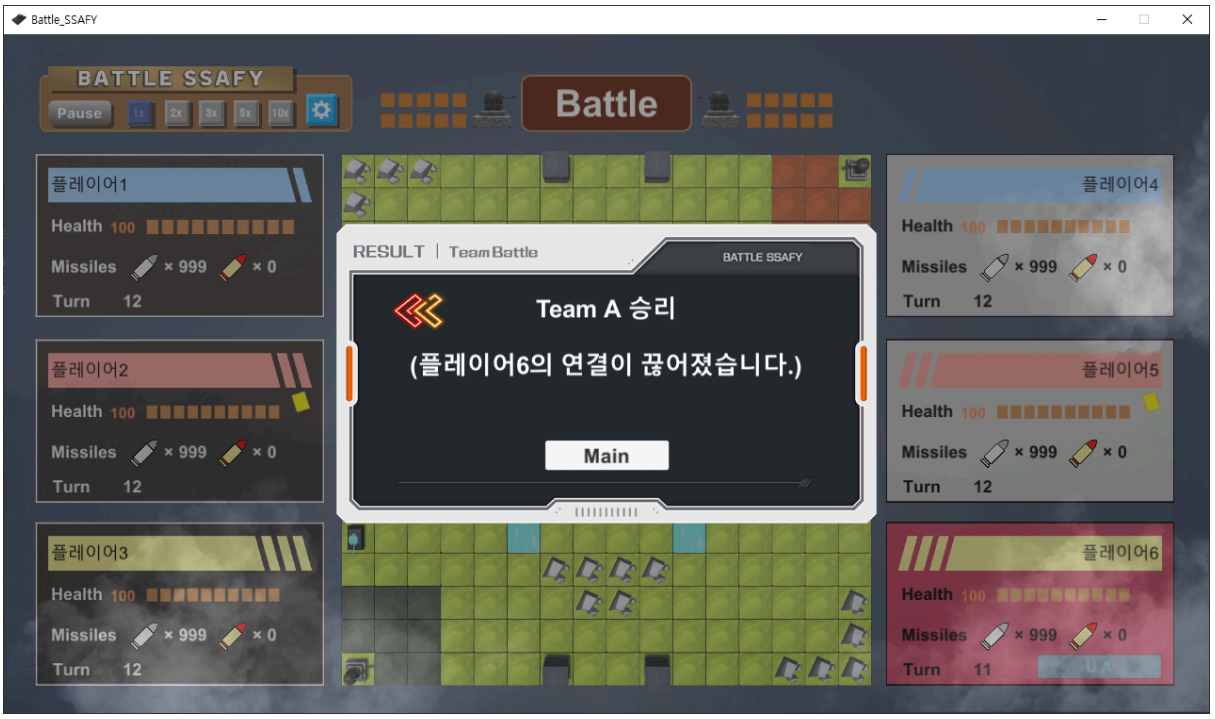
1. **차례가 된 탱크가 시간 초과, 런타임 오류** 등의 사유로 진행할 수 없는 경우 **(해당 팀의 패배)**
2. **상대의 포탑을 파괴**한 경우 **(포탑을 파괴한 팀의 승리)**
3. 한 팀의 탱크가 상대의 공격 또는 페널티에 의해 **2대 파괴되었거나 단 하나도 남아있지 않은 경우 (해당 팀의 패배)**
4. 모든 탱크가 **100턴에 도달**한 경우
    a. 팀의 탱크와 포탑의 **피해가 적은 팀이 승리**
    b. 피해량이 동일한 경우 **탱크와 적 포탑간 거리 합이 작은 팀이 승리**
    c. 피해량과 거리가 동일한 경우 **후공으로 플레이한 팀**의 승리

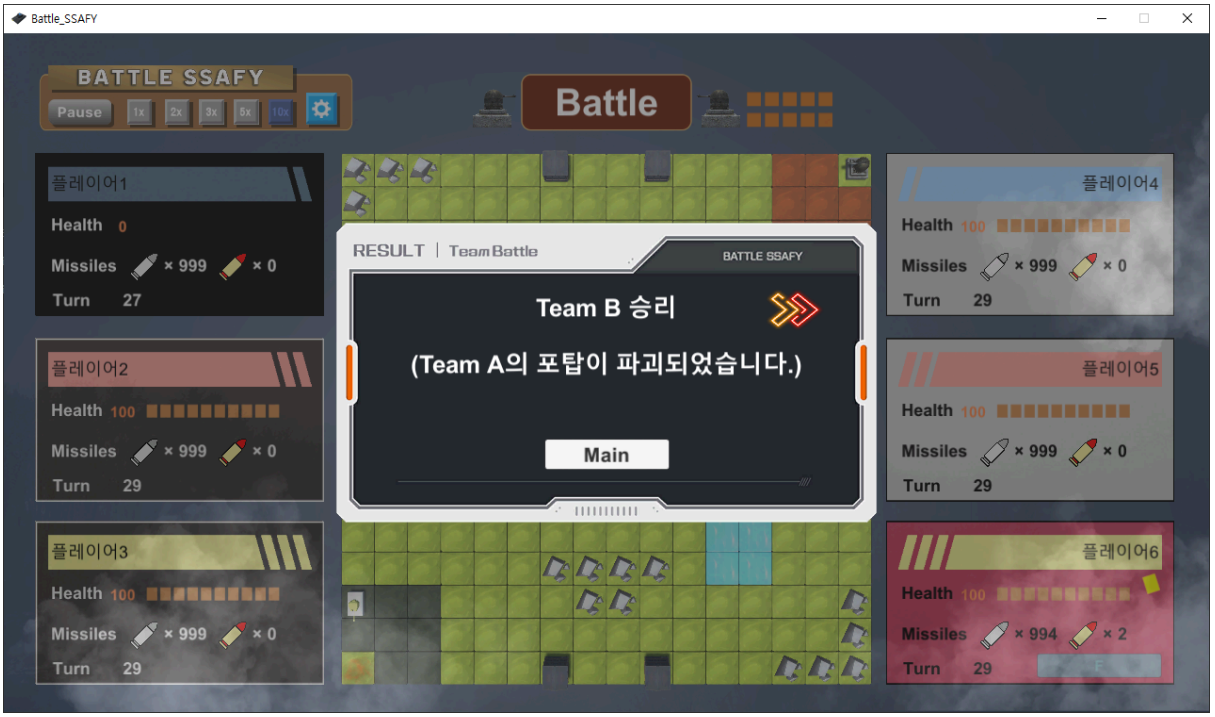
보다 상세한 규정은 별도로 안내되는 운영 지침을 참고하십시오.

⚠️ 특히 **1번 조건의 경우,**
열심히 구현한 소스 코드의 알고리즘이 시간 복잡도 측면에서 문제가 있거나 런타임 오류가 발생하는 상황 등이 해당할 수 있습니다. 이런 경우 대결이 더이상 진행될 수 없기 때문에 **팀 전체의 패배**로 이어지는 불상사를 겪게 됩니다.

따라서 팀 대결에서는 무엇보다도 코드의 최적화와 예외 처리에 많은 노력을 기울여야 하겠습니다.

대결이 종료되면 다음과 같이 **종료창**을 통해 결과를 알 수 있습니다.





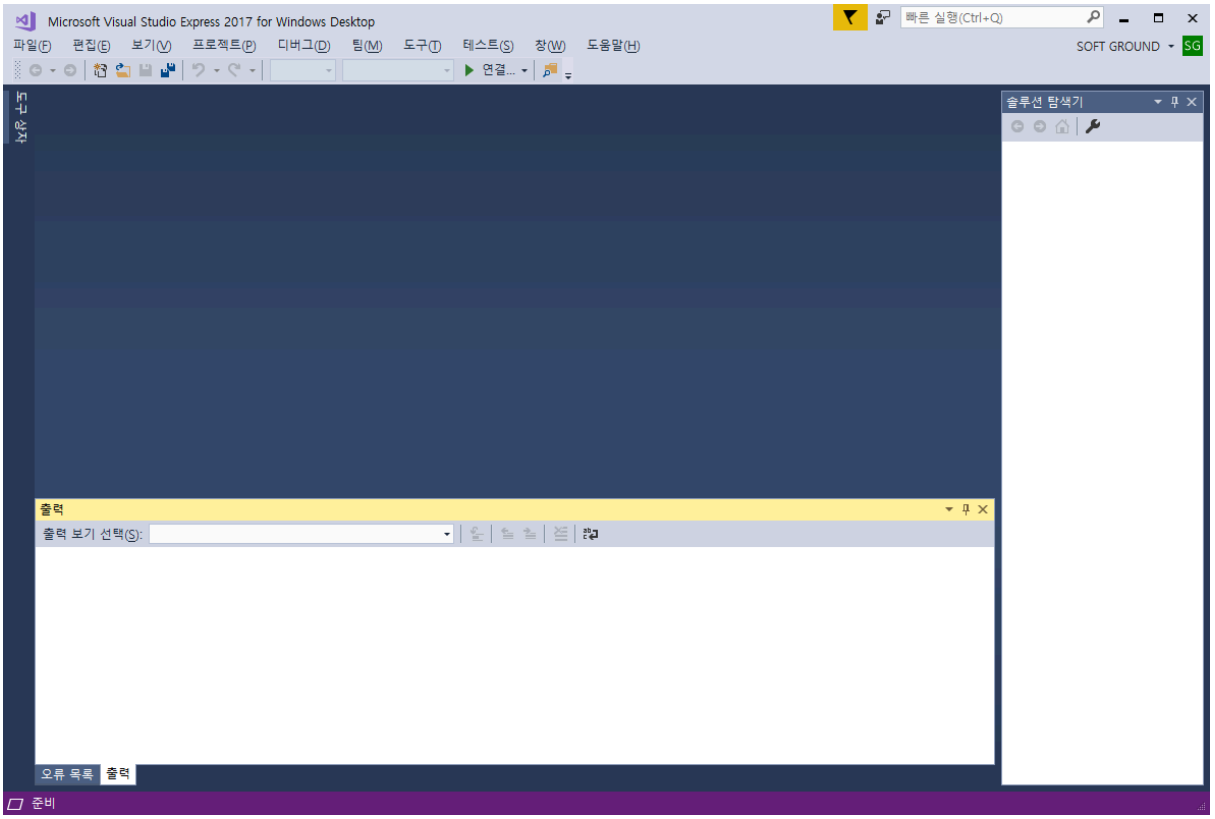
기본코드 활용 방법

여기에서는 각 언어별로 대표적인 경우 하나씩에 대해 제공된 기본 코드를 사용하는 방법을 설명하고 있습니다.

사용할 수 있는 개발 도구와 환경은 다양하므로 반드시 여기에 설명한 것을 따를 필요는 없으므로, 코드 작성 및 실행에 문제가 없다면 환경을 다르게 구성하여도 무방합니다.

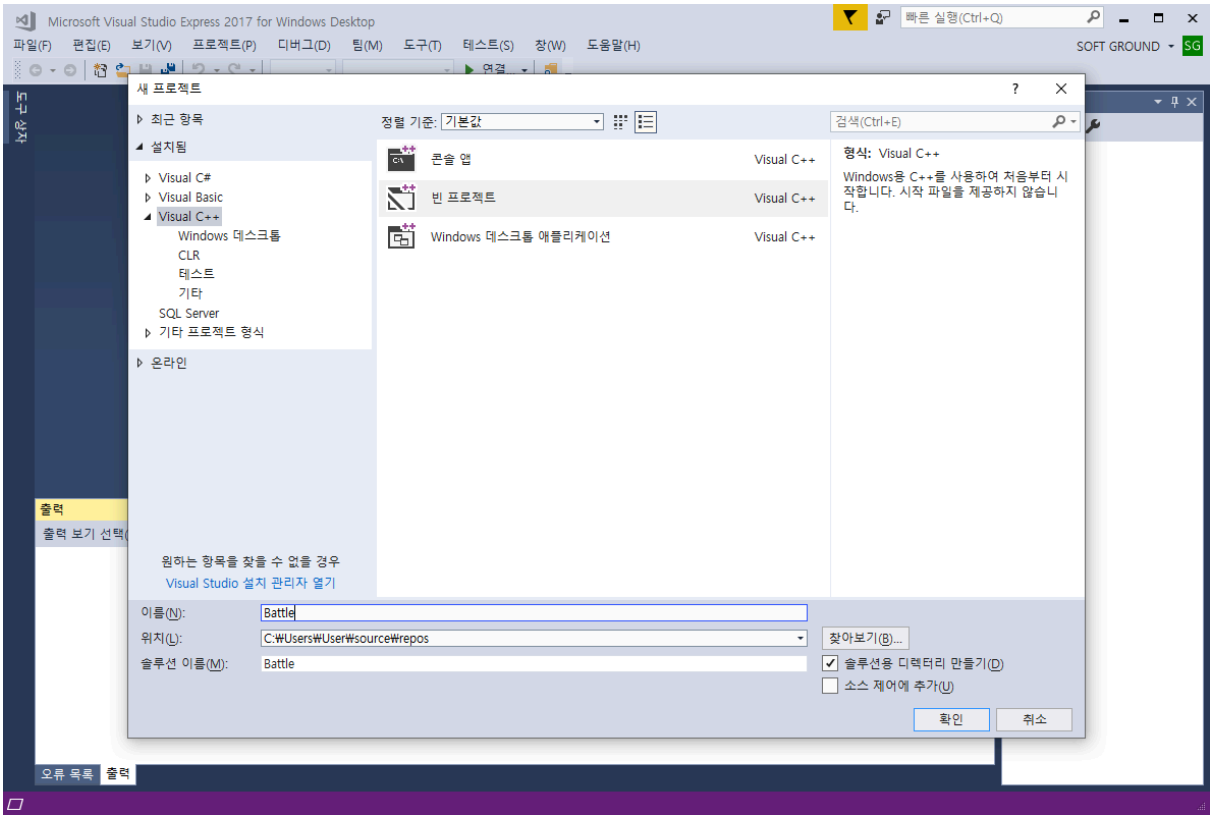
C언어

VS Express 2017 for Windows Desktop을 실행합니다.



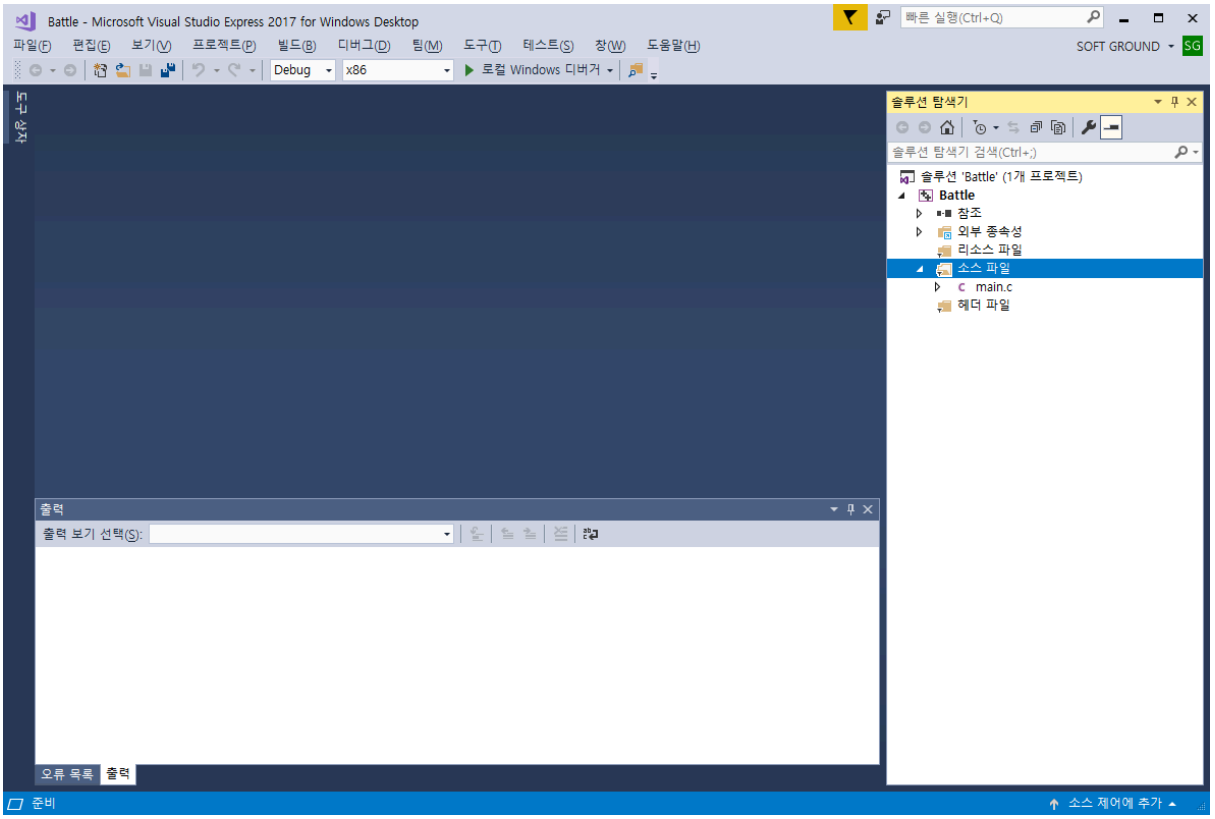
파일 > 새 프로젝트를 선택합니다.

새 프로젝트 창에서 Visual C++를 선택하고 빈 프로젝트를 선택합니다. 프로젝트의 이름은 임의로 입력해도 됩니다.



확인 버튼을 누르면 솔루션 탐색기에 새로운 프로젝트가 나타나는 것을 확인할 수 있습니다.

제공된 소스 코드의 main.c 파일을 드래그하여 솔루션 탐색기의 소스 파일에 가져다 놓습니다.



main.c를 더블 클릭하여 소스 코드를 살펴 봅니다.

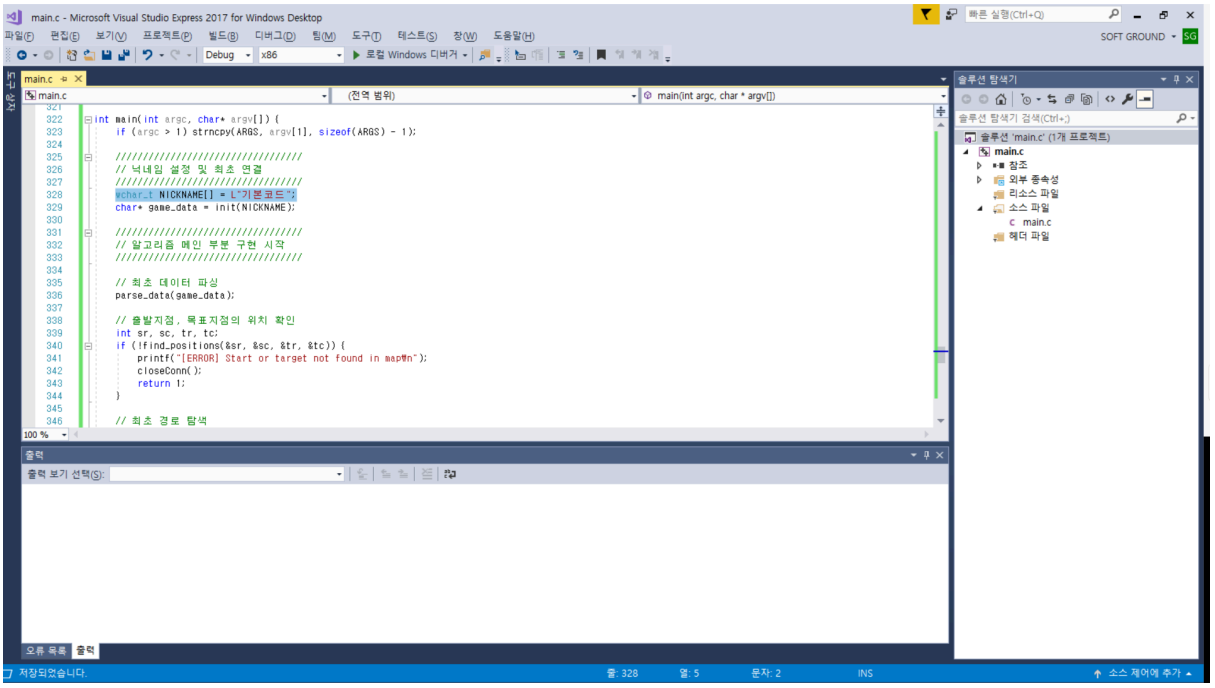
코드가 복잡하게 느껴질 수도 있지만, 대부분 배틀싸피와의 연동을 위해 작성된 코드이므로 직접 건드릴 일이 없습니다.

328번째 줄의 nickname을 `기본코드` 대신 `지역반_이름`으로 수정합니다.

nickname은 배틀싸피에 Client가 연결되었을 때 본인임을 확인할 수 있는 수단입니다.

(nickname은 12자를 초과할 수 없으며, 초과 시 13자부터는 무시됩니다.)

- 예: `wchar_t nickname[] = L"기본코드";` → `wchar_t nickname[] = L"서울1_김싸피";`

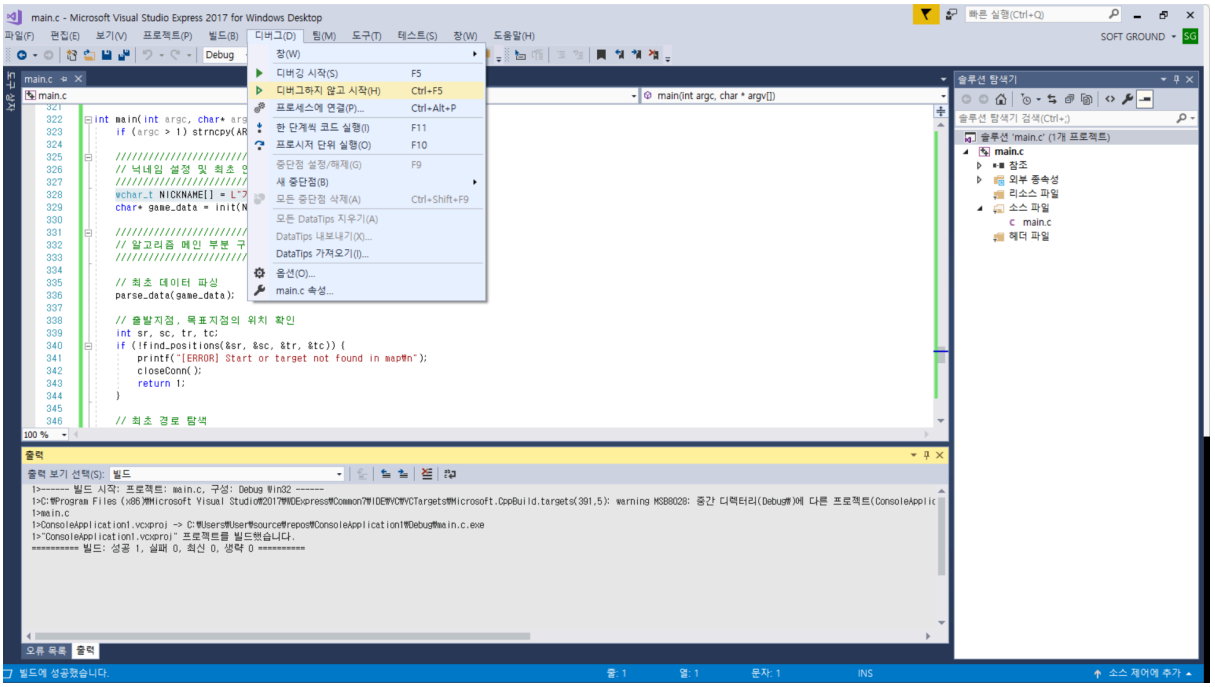


코드를 수정하였다면, 저장(Ctrl+S)합니다.

배틀싸피를 실행하고 스테이지1 연습 모드에 진입합니다.

Waiting for a player 메시지가 보이는 상태에서 main.c를 실행합니다.

VS Express의 상단 메뉴에서 디버그 > 디버그하지 않고 시작(Ctrl+F5)을 선택하면 됩니다.

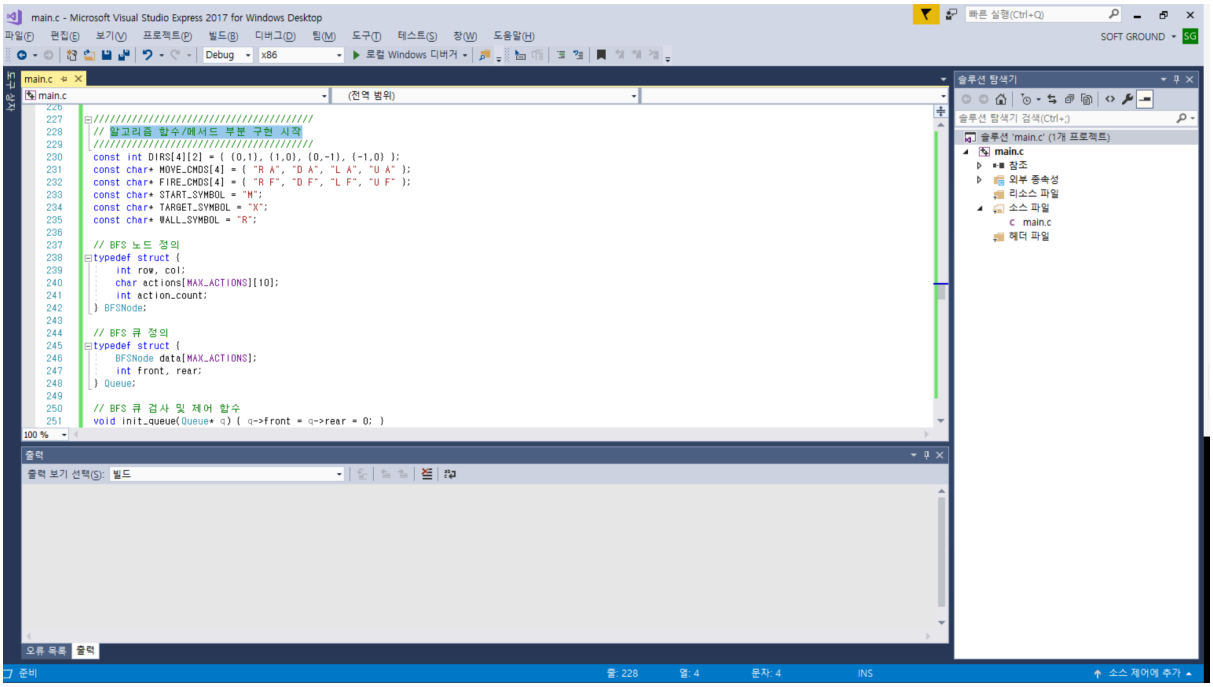


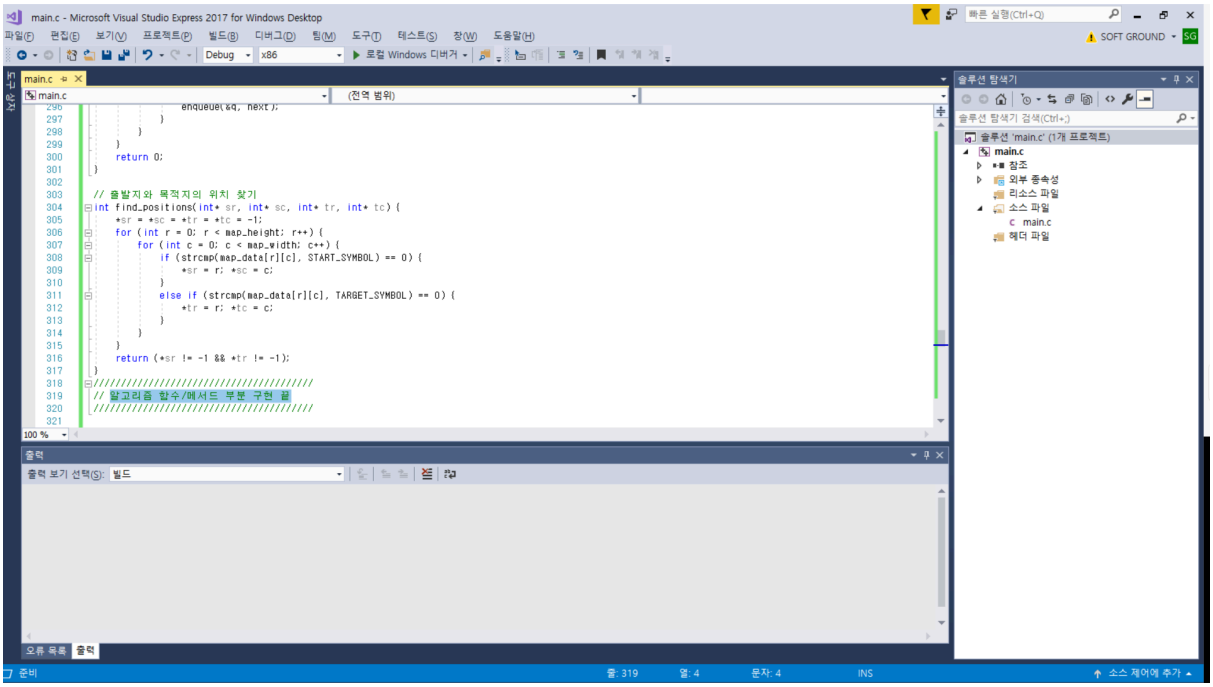
정상적으로 빌드되었다면, 실행 콘솔창이 나타나고 소스 코드에 의한 출력 데이터를 확인할 수 있습니다.

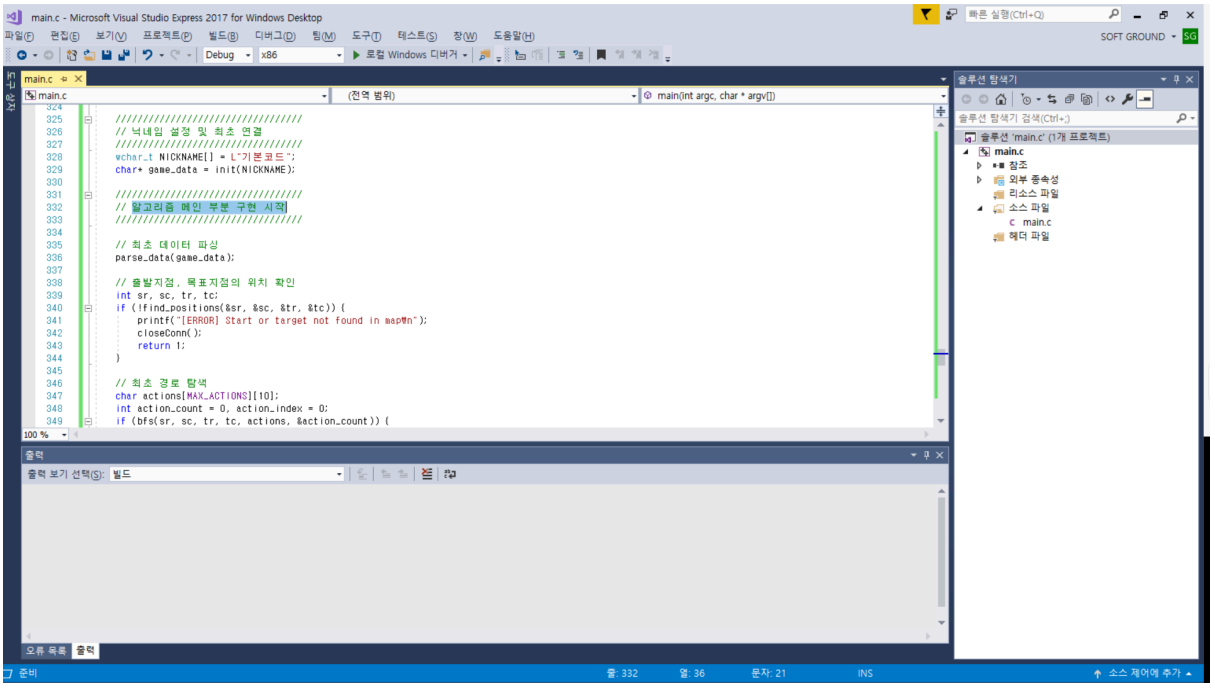
배틀싸피 화면도 방금 수정한 Nickname의 Client가 연결되었음을 표시해주고, 배틀싸피 ↔ Client가 데이터를 주고 받으며 탱크가 움직이기 시작합니다.

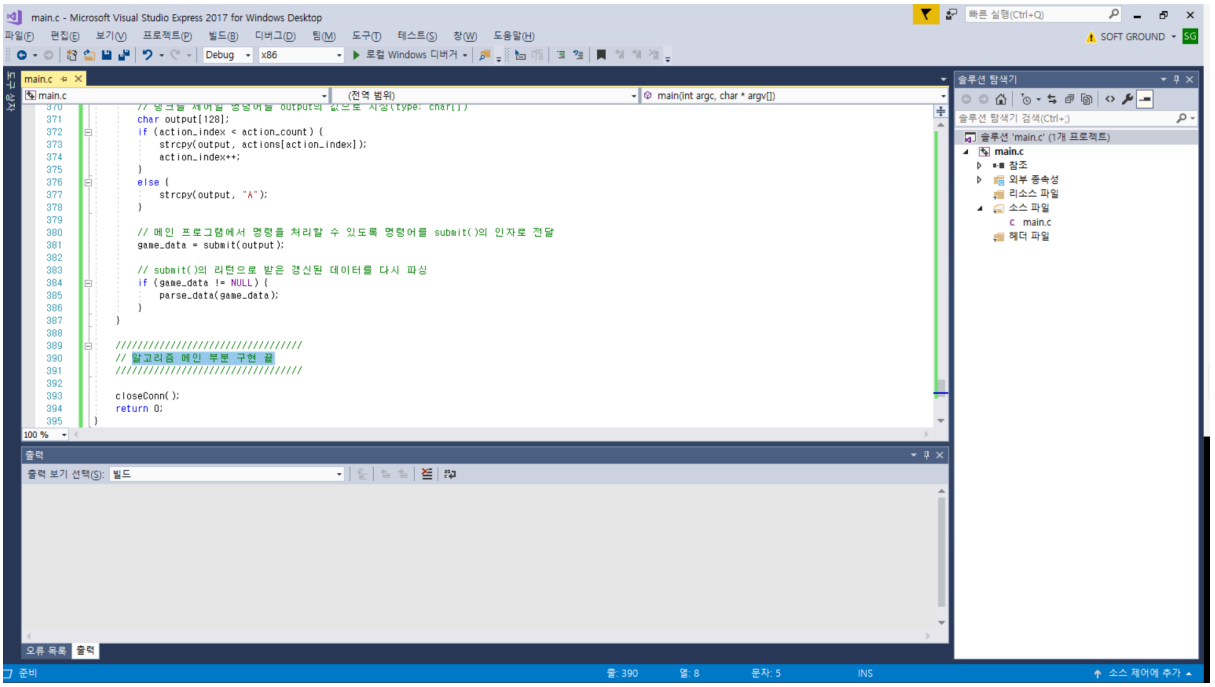
알고리즘을 구현할 수 있는 부분은

- main 함수 바깥 부분
  - “228번 줄: 알고리즘 함수/메서드 부분 구현 시작” 부터
  - “319번 줄: 알고리즘 함수/메서드 부분 구현 끝” 까지
- main 함수 내부
  - “332번 줄: 알고리즘 메인 부분 구현 시작” 부터
  - “390번 줄: 알고리즘 메인 부분 구현 끝” 까지


main.c - Microsoft Visual Studio Express 2017 for Windows Desktop
main.c
(전역 범위)
//
// 알고리즘 함수/메서드 부분 구현 시작
//
const int DIRS[4][2] = { {0,1}, {1,0}, {0,-1}, {-1,0} };
const char* MOVE_CMDS[4] = { "R A", "D A", "L A", "U A" };
const char* FIRE_CMDS[4] = { "R F", "D F", "L F", "U F" };
const char* START_SYMBOL = "M";
const char* TARGET_SYMBOL = "X";
const char* WALL_SYMBOL = "R";
// BFS 노드 정의
typedef struct {
int row, col;
char actions[MAX_ACTIONS][10];
int action_count;
} BFSNode;
// BFS 큐 정의
typedef struct {
BFSNode data[MAX_ACTIONS];
int front, rear;
} Queue;
// BFS 큐 검사 및 제어 함수
void init_queue(Queue* q) { q->front = q->rear = 0; }
솔루션 탐색기
솔루션 'main.c' (1개 프로젝트)
main.c
참조
외부 종속성
리소스 파일
소스 파일
main.c
헤더 파일
출력
출력 보기 선택(S): 빌드
오류 목록
출력
준비
줄: 228
열: 4
문자: 4
INS
소스 제어에 추가


main.c - Microsoft Visual Studio Express 2017 for Windows Desktop
main.c
(전역 범위)
enqueue(&q, next);
}
}
}
return 0;
}
// 출발지와 목적지의 위치 찾기
int find_positions(int* sr, int* sc, int* tr, int* tc) {
*sr = *sc = *tr = *tc = -1;
for (int r = 0; r < map_height; r++) {
for (int c = 0; c < map_width; c++) {
if (strcmp(map_data[r][c], START_SYMBOL) == 0) {
*sr = r; *sc = c;
}
else if (strcmp(map_data[r][c], TARGET_SYMBOL) == 0) {
*tr = r; *tc = c;
}
}
}
return (*sr != -1 && *tr != -1);
}
//
// 알고리즘 함수/메서드 부분 구현 끝
//
솔루션 탐색기
솔루션 'main.c' (1개 프로젝트)
main.c
참조
외부 종속성
리소스 파일
소스 파일
main.c
헤더 파일
출력
출력 보기 선택(S): 빌드
오류 목록
출력
준비
줄: 319
열: 4
문자: 4
INS
소스 제어에 추가





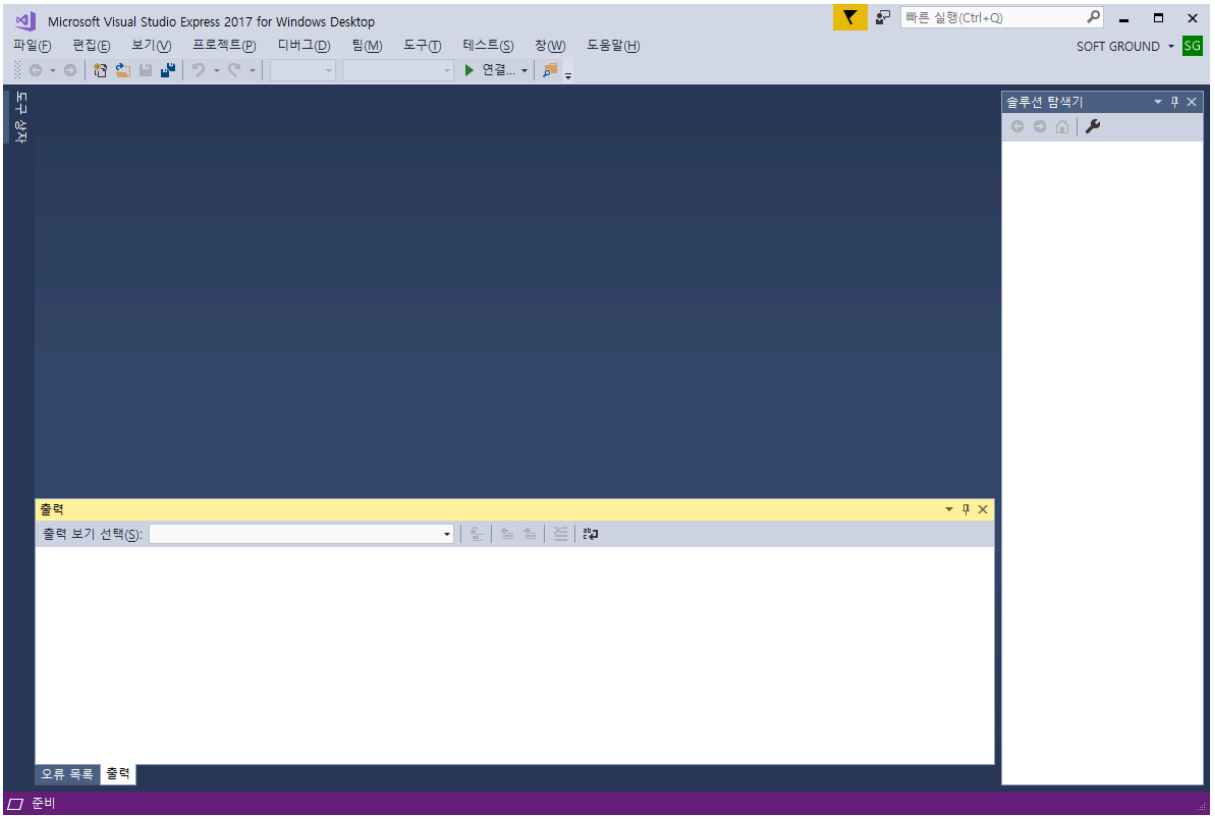
코드의 양을 반드시 표기한 줄 번호 안에만 맞게끔 작성해야 함을 의미하는 것이 아닙니다. 이는 제공된 소스 코드 기준의 줄 번호로, 코드의 양이 100줄, 1000줄 이상이 되어도 괜찮습니다. 여러분이 작성한 코드에 의해 해당 주석들의 줄 번호가 변경되는 것이 자연스러운 현상입니다.

함수, 전역 변수/상수 등의 자유로운 구현을 위해 구현 부분을 두 군데 표시해놓은 것으로, 메인 부분에만 코드를 작성해도 무방합니다.

⚠ 프로그램 통신부와 while 반복문의 역할이 중요하므로 가급적 위에 명시한 범위 내에서 코드를 작성할 것을 권장합니다.

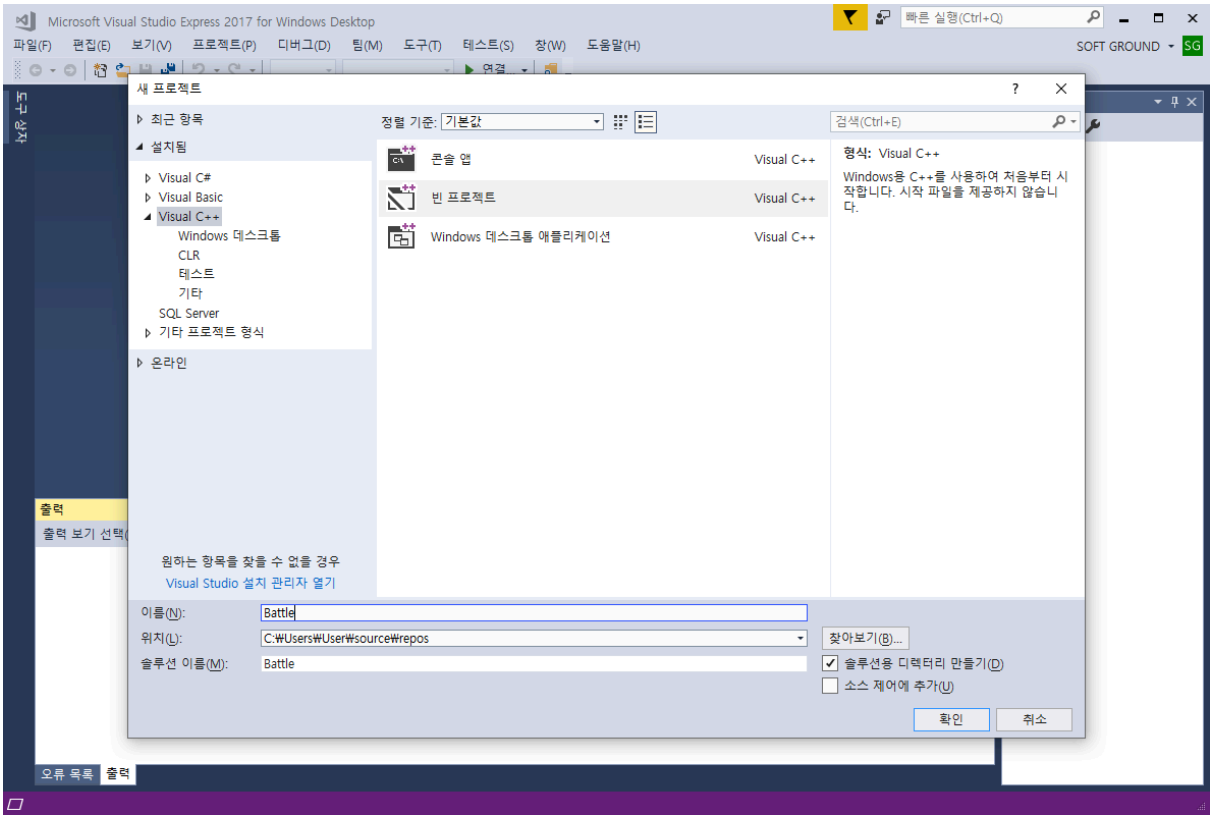
C++

VS Express 2017 for Windows Desktop을 실행합니다.



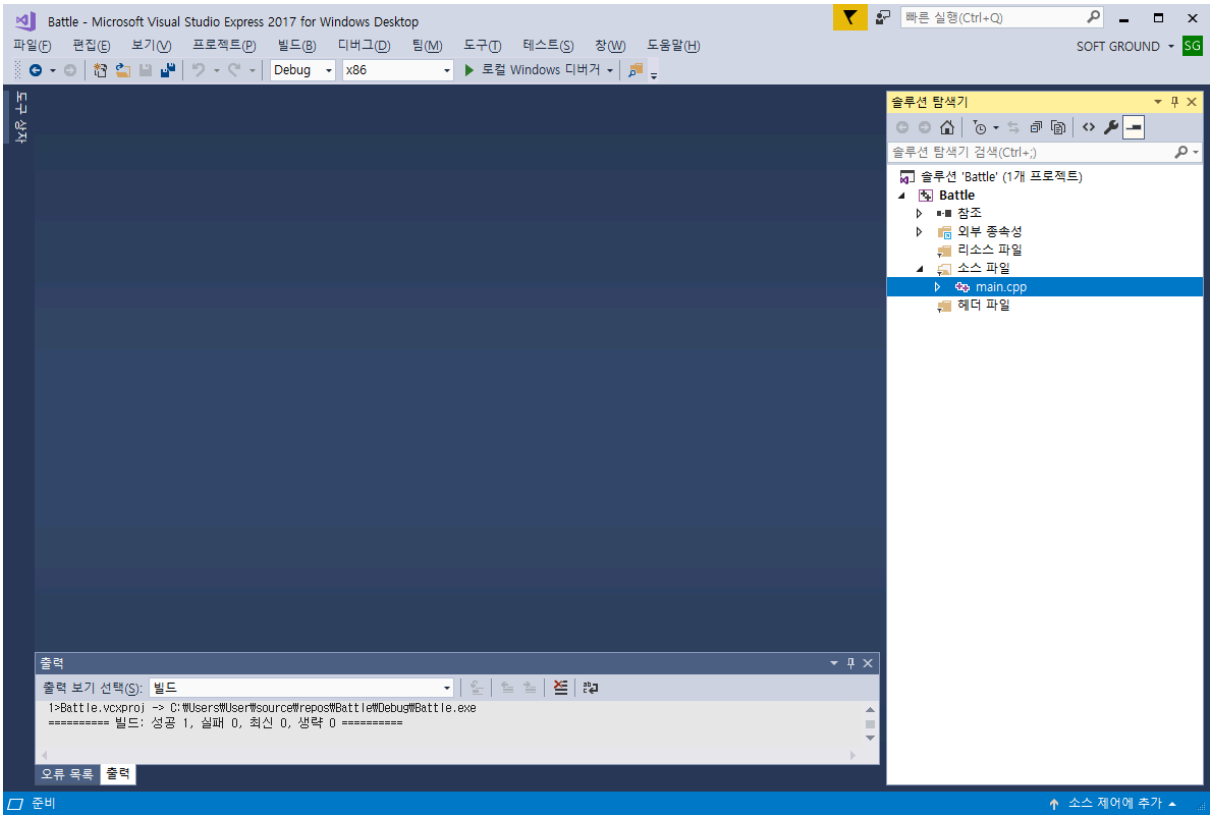
파일 > 새 프로젝트를 선택합니다.

새 프로젝트 창에서 Visual C++를 선택하고 빈 프로젝트를 선택합니다. 프로젝트의 이름은 임의로 입력해도 됩니다.



확인 버튼을 누르면 솔루션 탐색기에 새로운 프로젝트가 나타나는 것을 확인할 수 있습니다.

제공된 소스 코드의 main.cpp 파일을 드래그하여 솔루션 탐색기의 소스 파일에 가져다 놓습니다.



main.cpp를 더블 클릭하여 소스 코드를 살펴 봅니다.

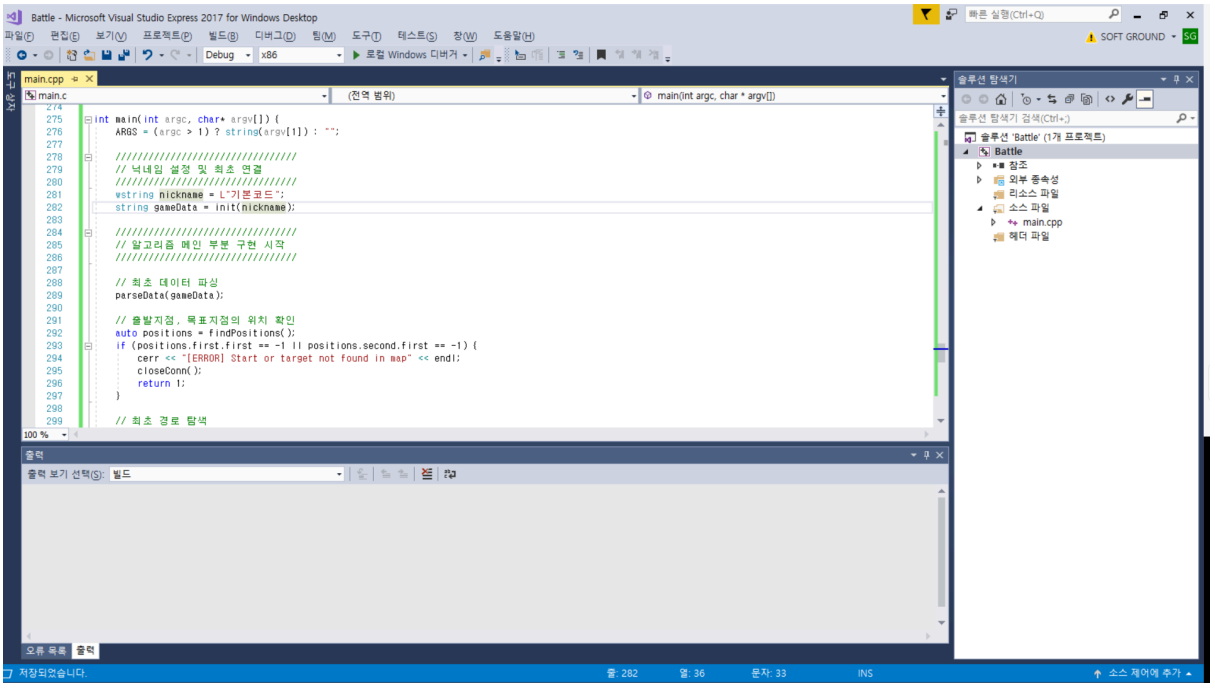
코드가 복잡하게 느껴질 수도 있지만, 대부분 배틀싸피와의 연동을 위해 작성된 코드이므로 직접 건드릴 일이 없습니다.

281번째 줄의 NICKNAME을 `기본코드` 대신 `지역반_이름`으로 수정합니다.

nickname은 배틀싸피에 Client가 연결되었을 때 본인임을 확인할 수 있는 수단입니다.

(nickname은 12자를 초과할 수 없으며, 초과 시 13자부터는 무시됩니다.)

- 예: `wstring nickname = L"기본코드";` → `wstring nickname = L"서울1_김싸피";`

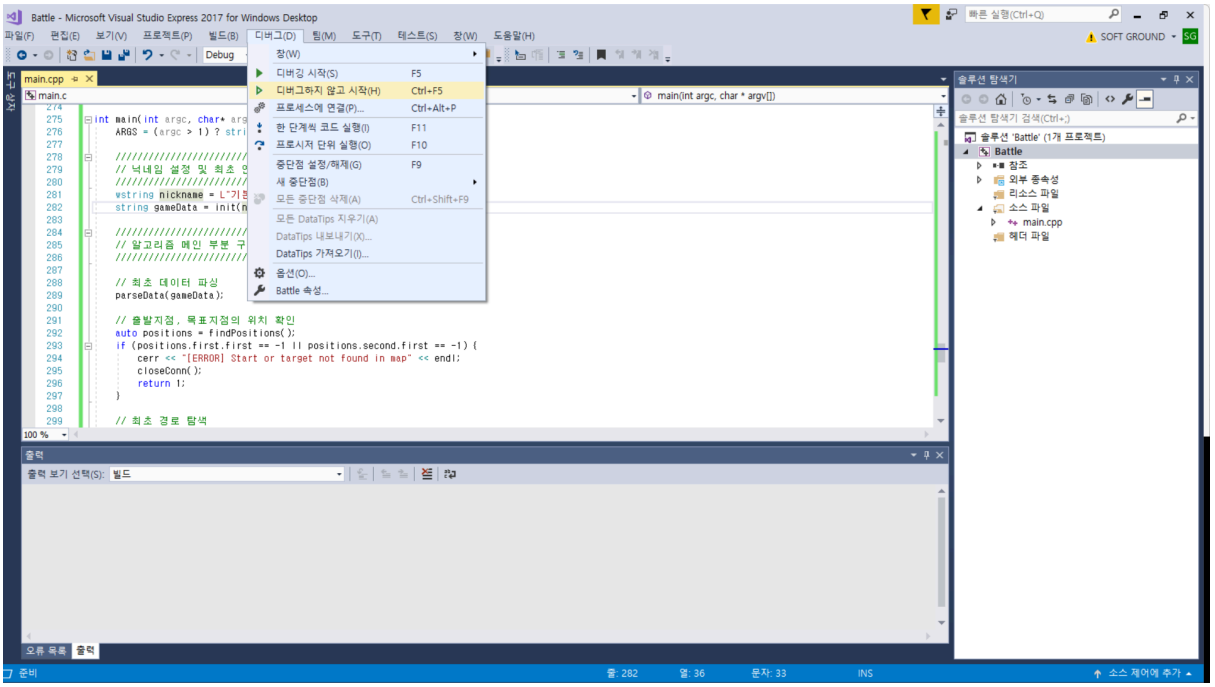


코드를 수정하였다면, 저장(Ctrl+S)합니다.

배틀싸피를 실행하고 스테이지1 연습 모드에 진입합니다.

Waiting for a player 메시지가 보이는 상태에서 main.c를 실행합니다.

VS Express의 상단 메뉴에서 디버그 > 디버그하지 않고 시작(Ctrl+F5)을 선택하면 됩니다.

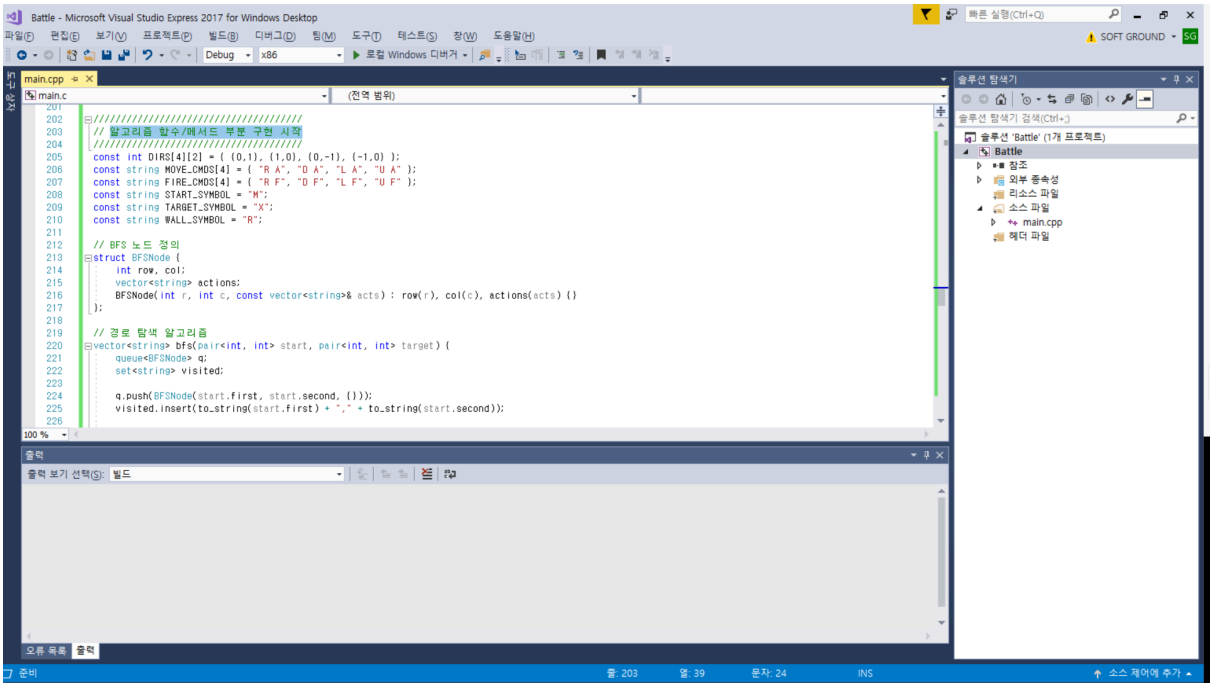


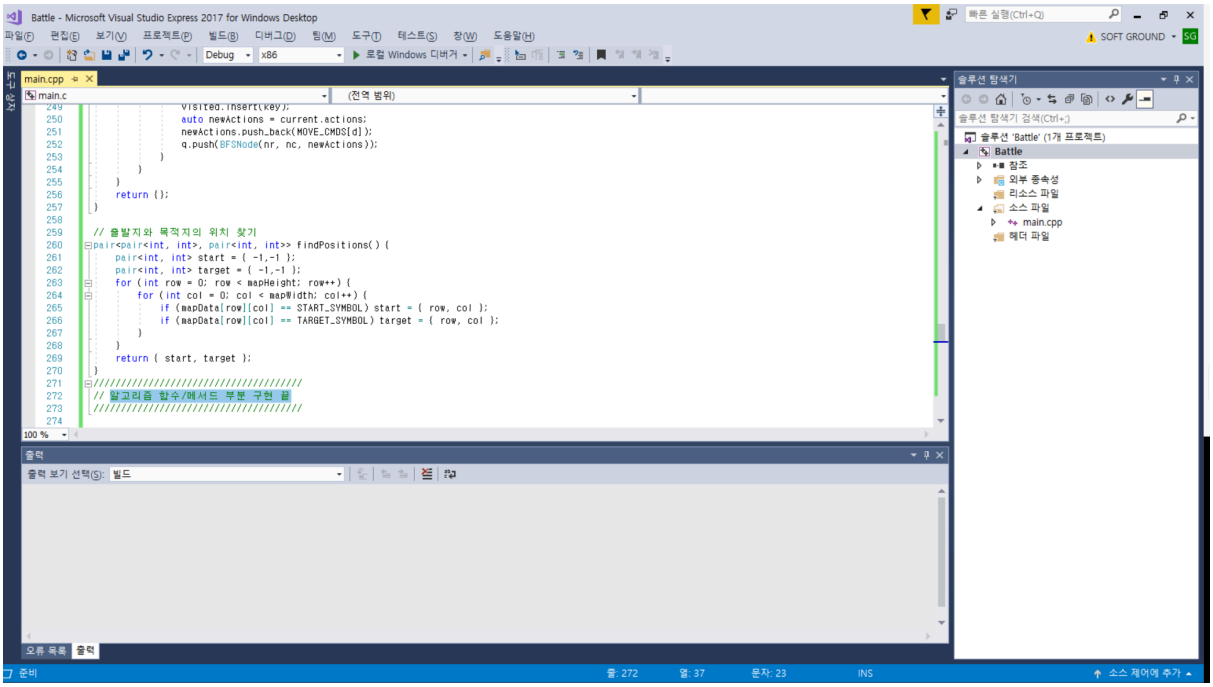
정상적으로 빌드되었다면, 실행 콘솔창이 나타납니다.

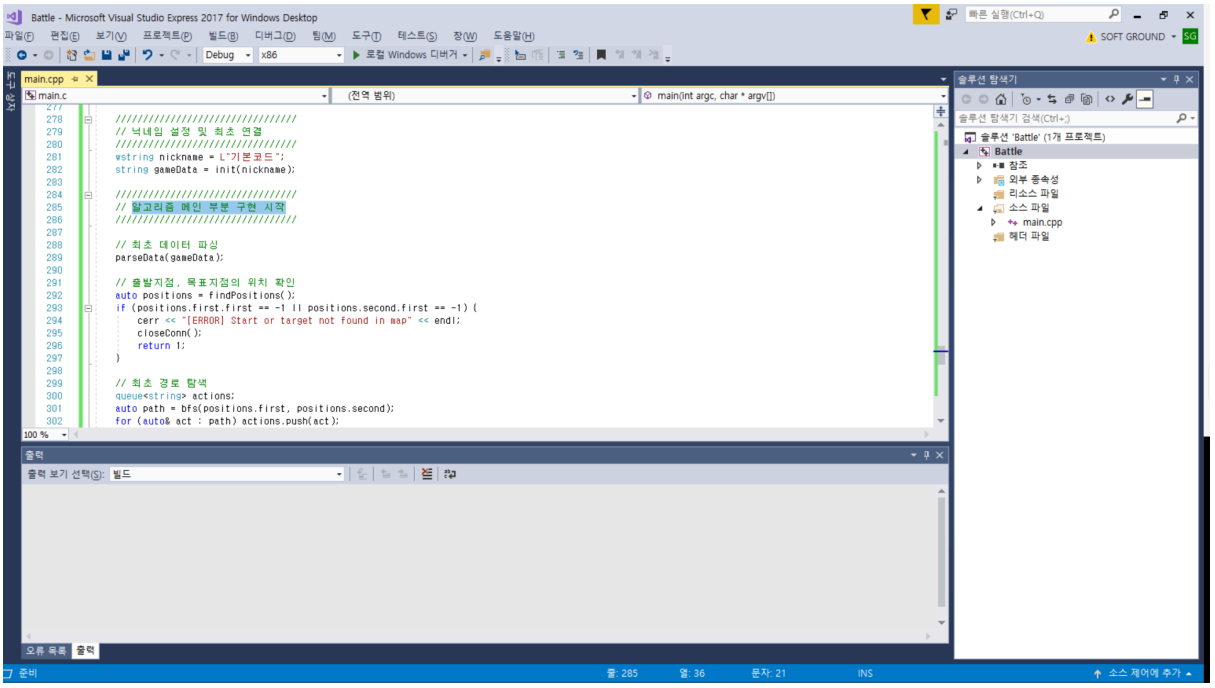
배틀싸피 화면도 방금 수정한 Nickname의 Client가 연결되었음을 표시해주고, 배틀싸피 ↔ Client가 데이터를 주고 받으며 탱크가 움직이기 시작합니다.

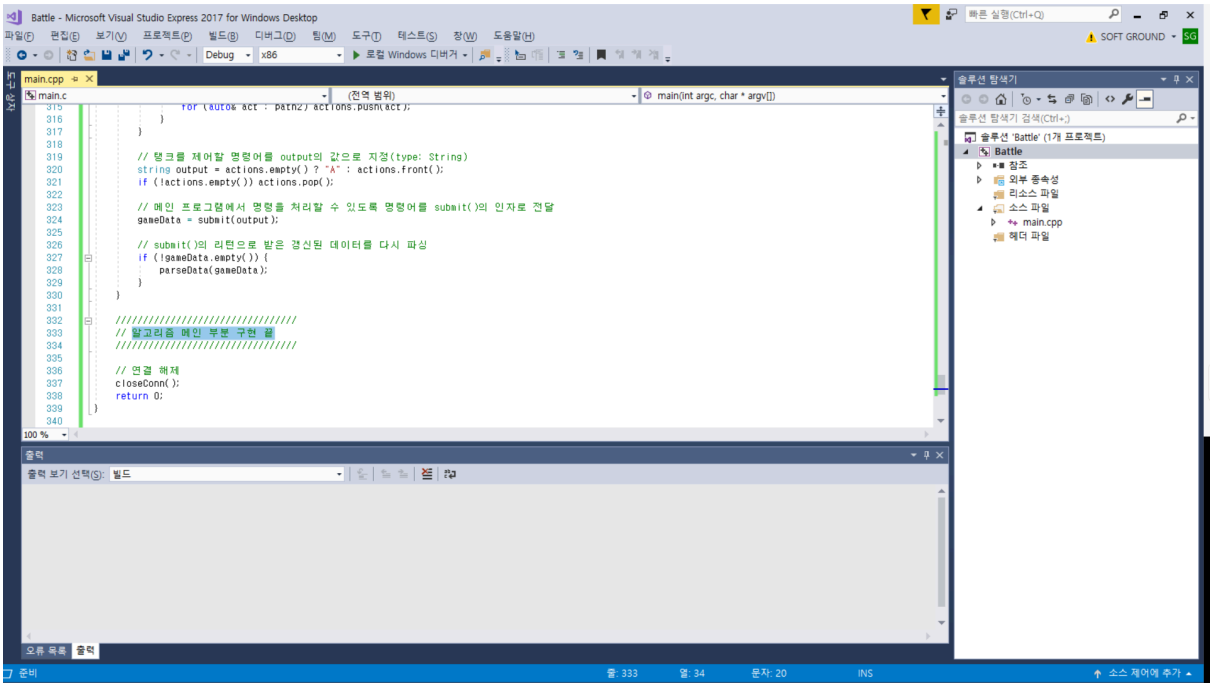
알고리즘을 구현할 수 있는 부분은

- main 함수 바깥 부분
  - “203번 줄: 알고리즘 함수/메서드 부분 구현 시작” 부터
  - “272번 줄: 알고리즘 함수/메서드 부분 구현 끝” 까지
- main 함수 내부
  - “285번 줄: 알고리즘 메인 부분 구현 시작” 부터
  - “333번 줄: 알고리즘 메인 부분 구현 끝” 까지


Battle - Microsoft Visual Studio Express 2017 for Windows Desktop
main.cpp
main.c
(전역 범위)
//
// 알고리즘 함수/메서드 부분 구현 시작
//
const int DIRS[4][2] = { {0,1}, {1,0}, {0,-1}, {-1,0} };
const string MOVE_CMDS[4] = { "R A", "D A", "L A", "U A" };
const string FIRE_CMDS[4] = { "R F", "D F", "L F", "U F" };
const string START_SYMBOL = "M";
const string TARGET_SYMBOL = "X";
const string WALL_SYMBOL = "R";
// BFS 노드 정의
struct BFSNode {
int row, col;
vector<string> actions;
BFSNode(int r, int c, const vector<string>& acts) : row(r), col(c), actions(acts) {}
};
// 경로 탐색 알고리즘
vector<string> bfs(pair<int, int> start, pair<int, int> target) {
queue<BFSNode> q;
set<string> visited;
q.push(BFSNode(start.first, start.second, {}));
visited.insert(to_string(start.first) + "," + to_string(start.second));
출력
출력 보기 선택(S): 빌드
오류 목록
출력
준비
줄: 203
열: 39
문자: 24
INS
소스 제어에 추가
솔루션 탐색기
솔루션 'Battle' (1개 프로젝트)
Battle
참조
외부 종속성
리소스 파일
소스 파일
main.cpp
헤더 파일


Battle - Microsoft Visual Studio Express 2017 for Windows Desktop
main.cpp
main.c
(전역 범위)
auto newActions = current.actions;
newActions.push_back(MOVE_CMDS[d]);
q.push(BFSNode(nr, nc, newActions));
}
}
}
return {};
}
// 출발지와 목적지의 위치 찾기
pair<pair<int, int>, pair<int, int>> findPositions() {
pair<int, int> start = { -1,-1 };
pair<int, int> target = { -1,-1 };
for (int row = 0; row < mapHeight; row++) {
for (int col = 0; col < mapWidth; col++) {
if (mapData[row][col] == START_SYMBOL) start = { row, col };
if (mapData[row][col] == TARGET_SYMBOL) target = { row, col };
}
}
return { start, target };
}
//
// 알고리즘 함수/메서드 부분 구현 끝
//
출력
출력 보기 선택(S): 빌드
오류 목록
출력
준비
줄: 272
열: 37
문자: 23
INS
소스 제어에 추가
솔루션 탐색기
솔루션 'Battle' (1개 프로젝트)
Battle
참조
외부 종속성
리소스 파일
소스 파일
main.cpp
헤더 파일





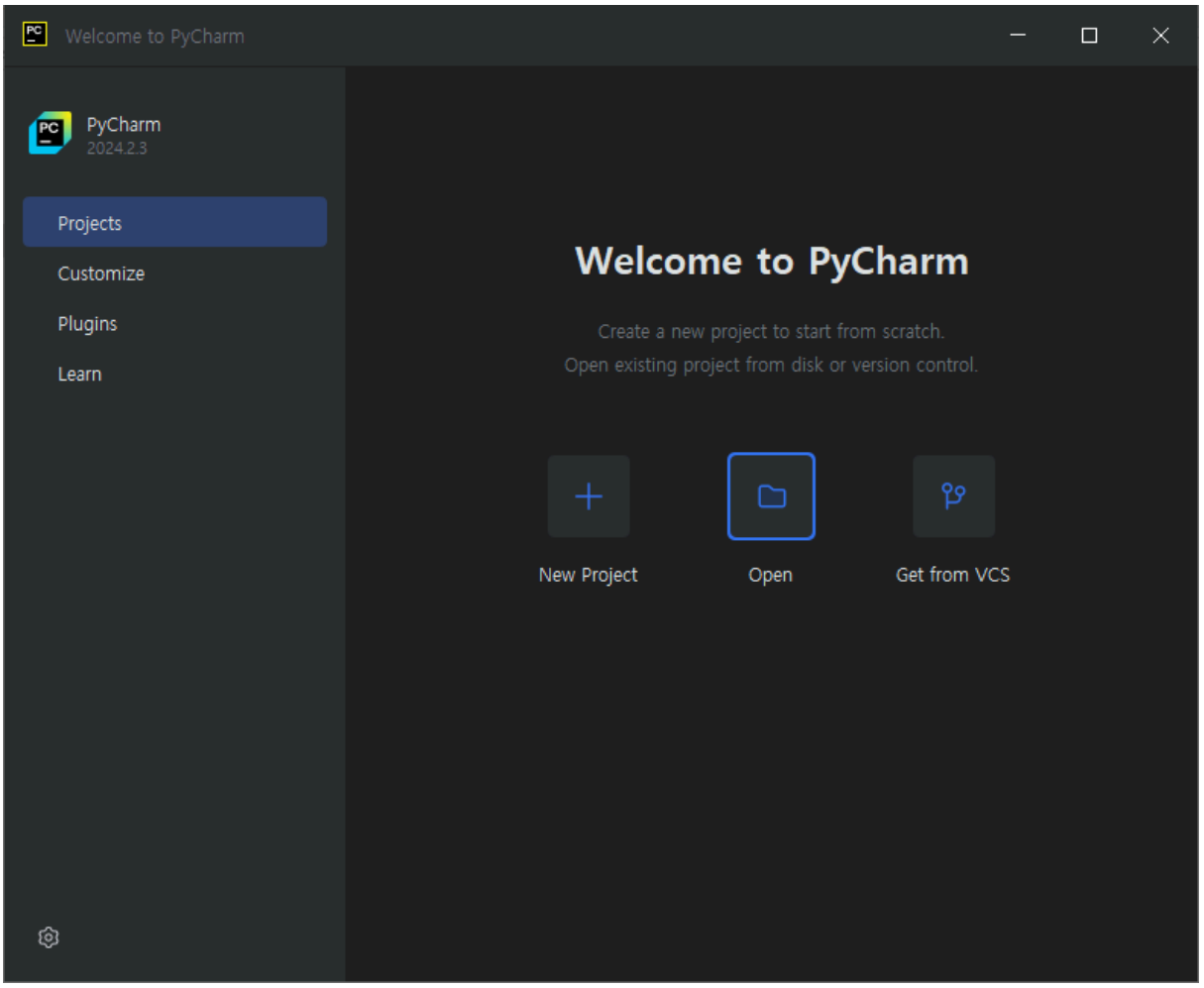
코드의 양을 반드시 표기한 줄 번호 안에만 맞게끔 작성해야 함을 의미하는 것이 아닙니다. 이는 제공된 소스 코드 기준의 줄 번호로, 코드의 양이 100줄, 1000줄 이상이 되어도 괜찮습니다. 여러분이 작성한 코드에 의해 해당 주석들의 줄 번호가 변경되는 것이 자연스러운 현상입니다.

함수, 전역 변수/상수 등의 자유로운 구현을 위해 구현 부분을 두 군데 표시해놓은 것으로, 메인 부분에만 코드를 작성해도 무방합니다.

⚠️ 프로그램 통신부와 while 반복문의 역할이 중요하므로 가급적 위에 명시한 범위 내에서 코드를 작성할 것을 권장합니다.

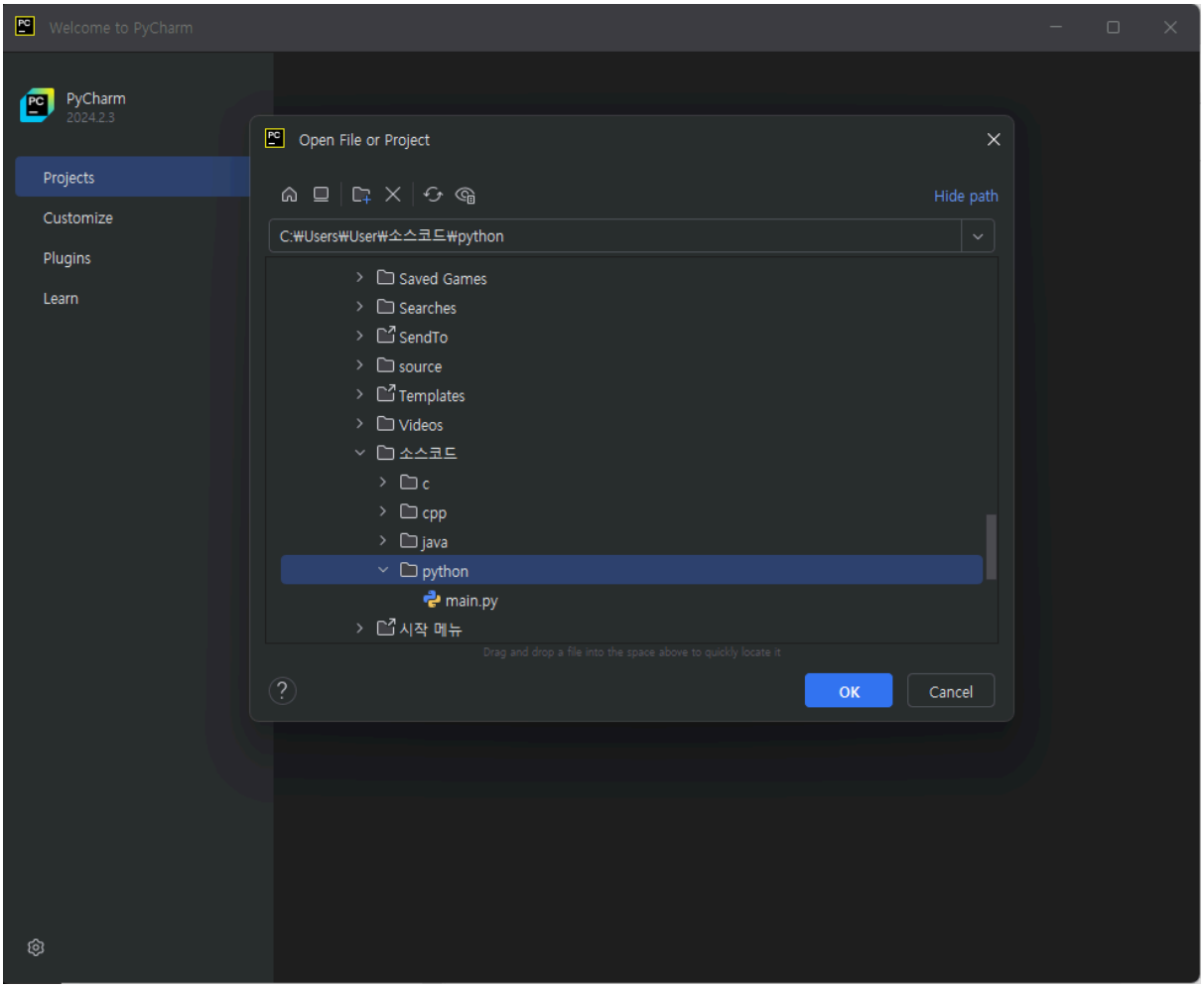
Python

PyCharm을 실행합니다.



Open을 선택합니다.(이미 열려있는 프로젝트가 있다면 화면이 다르게 보일 수 있습니다. 상단 메뉴에서 Open Project를 선택하면 됩니다.)

제공된 소스 코드 중 Python 폴더 경로를 찾아 선택 또는 입력하고 OK 버튼을 클릭하면 다음으로 진행됩니다.



프로젝트 화면이 나타나면 main.py 파일을 선택하여 소스 코드를 살펴 봅니다.

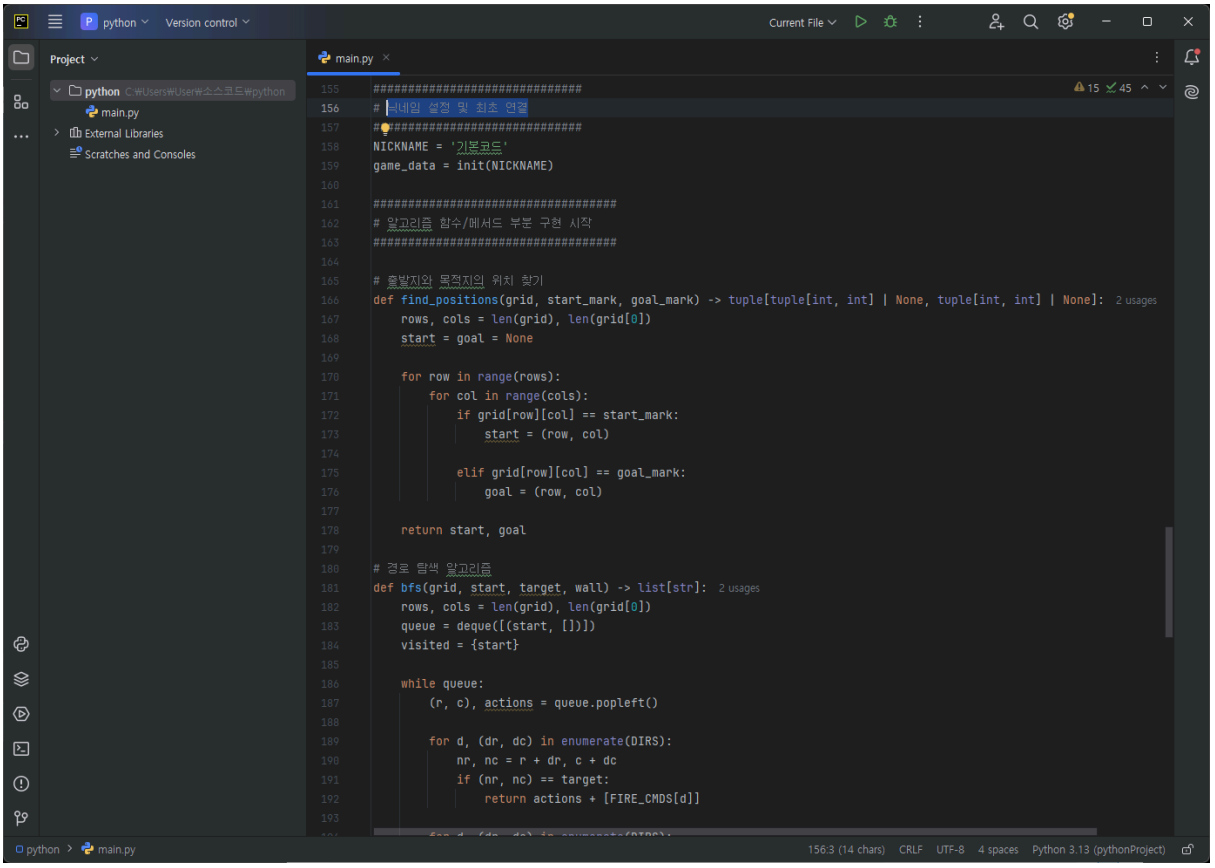
코드가 복잡하게 느껴질 수도 있지만, 대부분 배틀싸피와의 연동을 위해 작성된 코드이므로 직접 건드릴 일이 없습니다.

156번째 줄의 NICKNAME을 `기본코드` 대신 `지역반_이름`으로 수정합니다.

nickname은 배틀싸피에 Client가 연결되었을 때 본인임을 확인할 수 있는 수단입니다.

(nickname은 12자를 초과할 수 없으며, 초과 시 13자부터는 무시됩니다.)

- 예: `NICKNAME = '기본코드'` → `NICKNAME = '서울1_김싸피'`

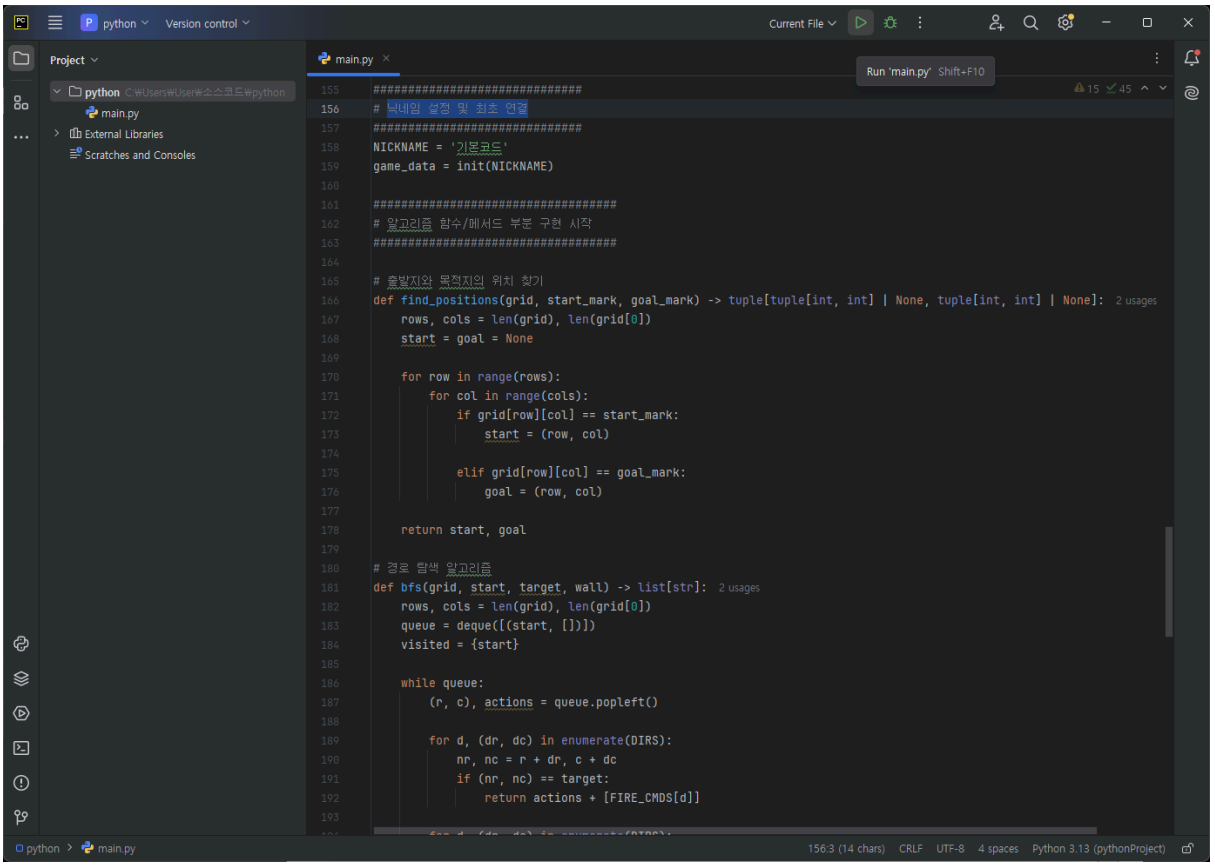


코드를 수정했다면 저장(Ctrl+S)합니다.

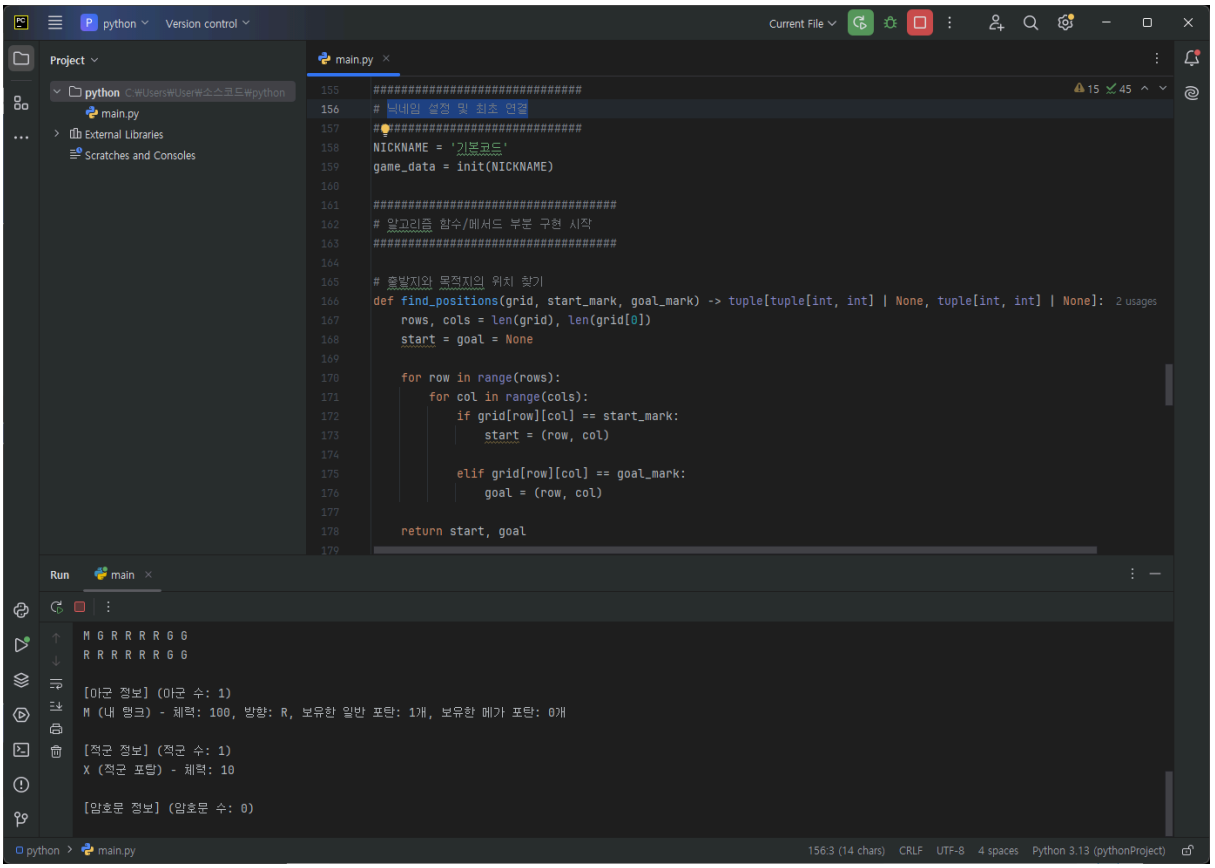
배틀싸피를 실행하고 스테이지1 연습 모드에 진입합니다.

Waiting for a player 메시지가 보이는 상태에서 main.py를 실행합니다.

상단의 실행 버튼을 클릭하면 됩니다.



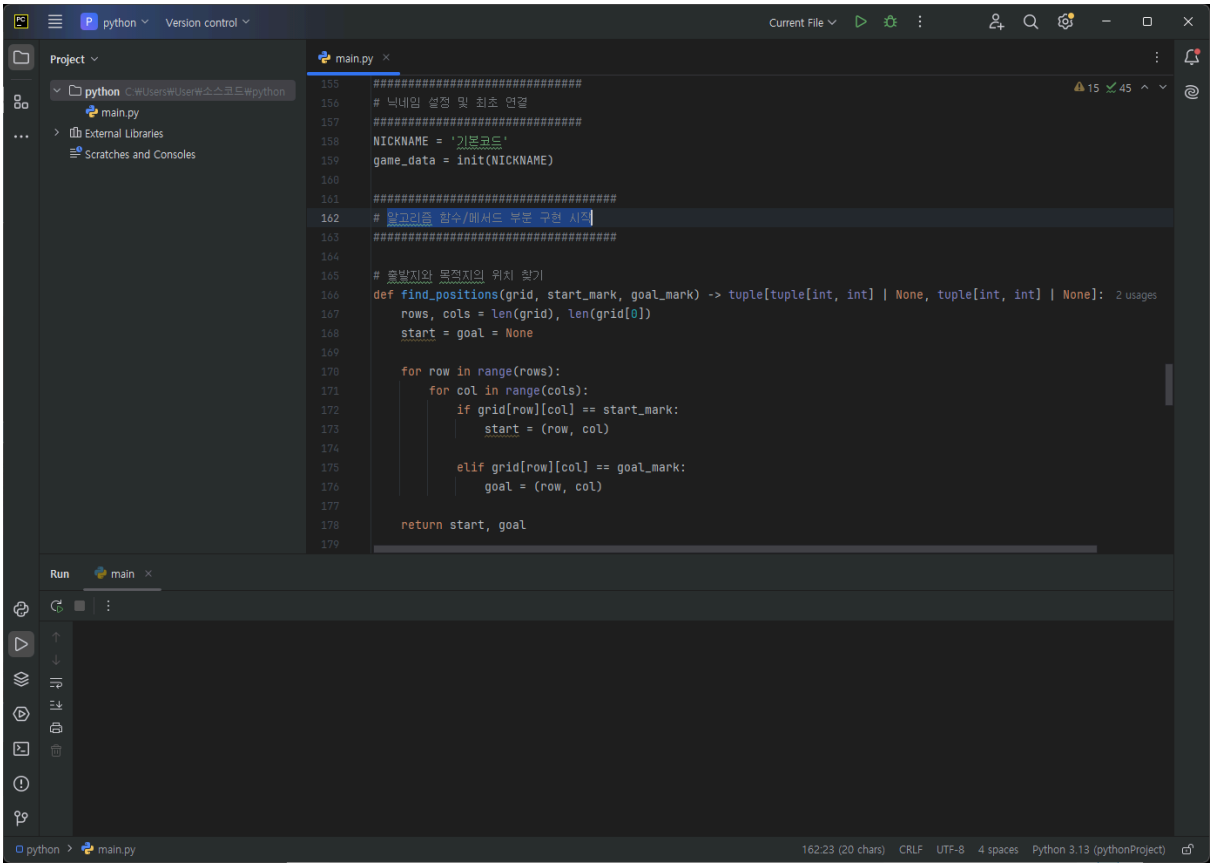
실행 결과를 보여주는 출력창이 나타나고 배틀싸피와 데이터 교환이 이루어지고 있음을 확인할 수 있습니다.



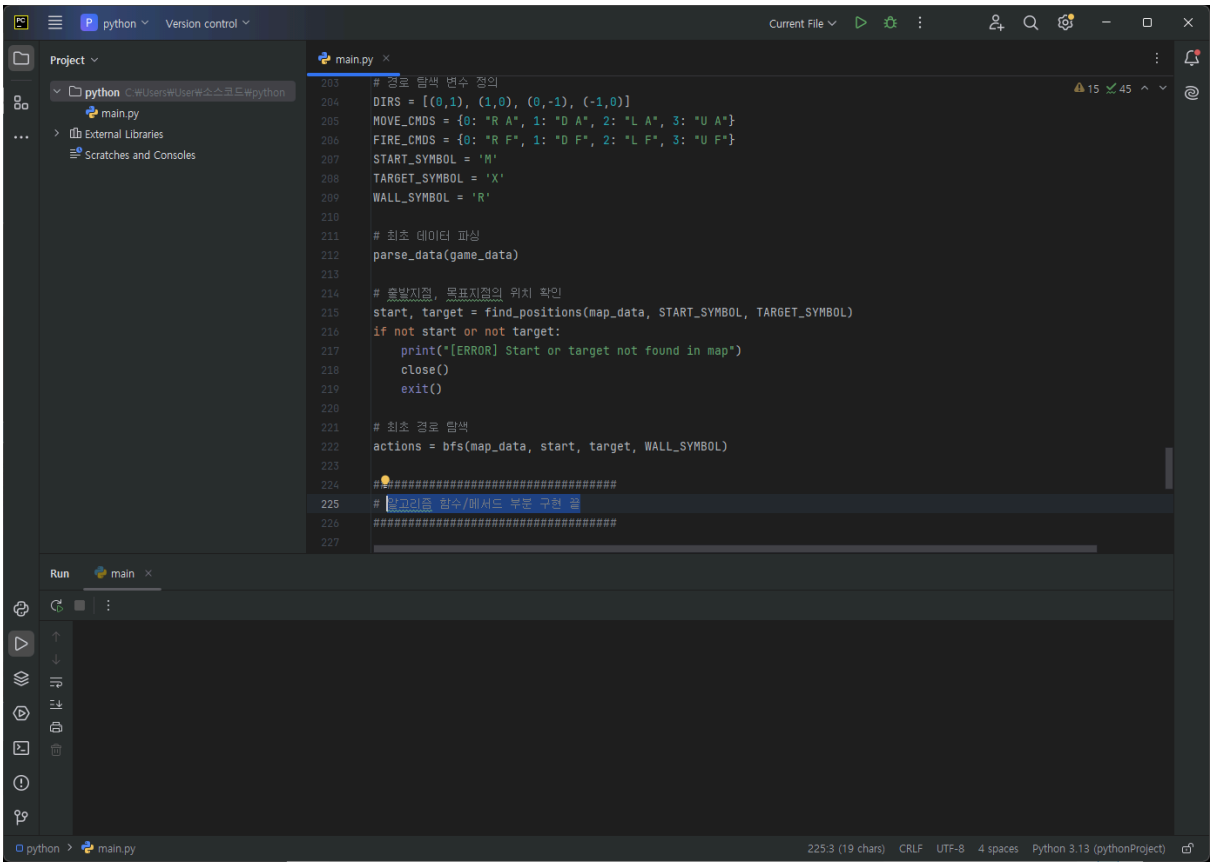
배틀싸피 화면도 방금 수정한 Nickname의 Client가 연결되었음을 표시해주고, 배틀싸피 ↔ Client가 데이터를 주고 받으며 탱크가 움직이기 시작합니다.

알고리즘을 구현할 수 있는 부분은

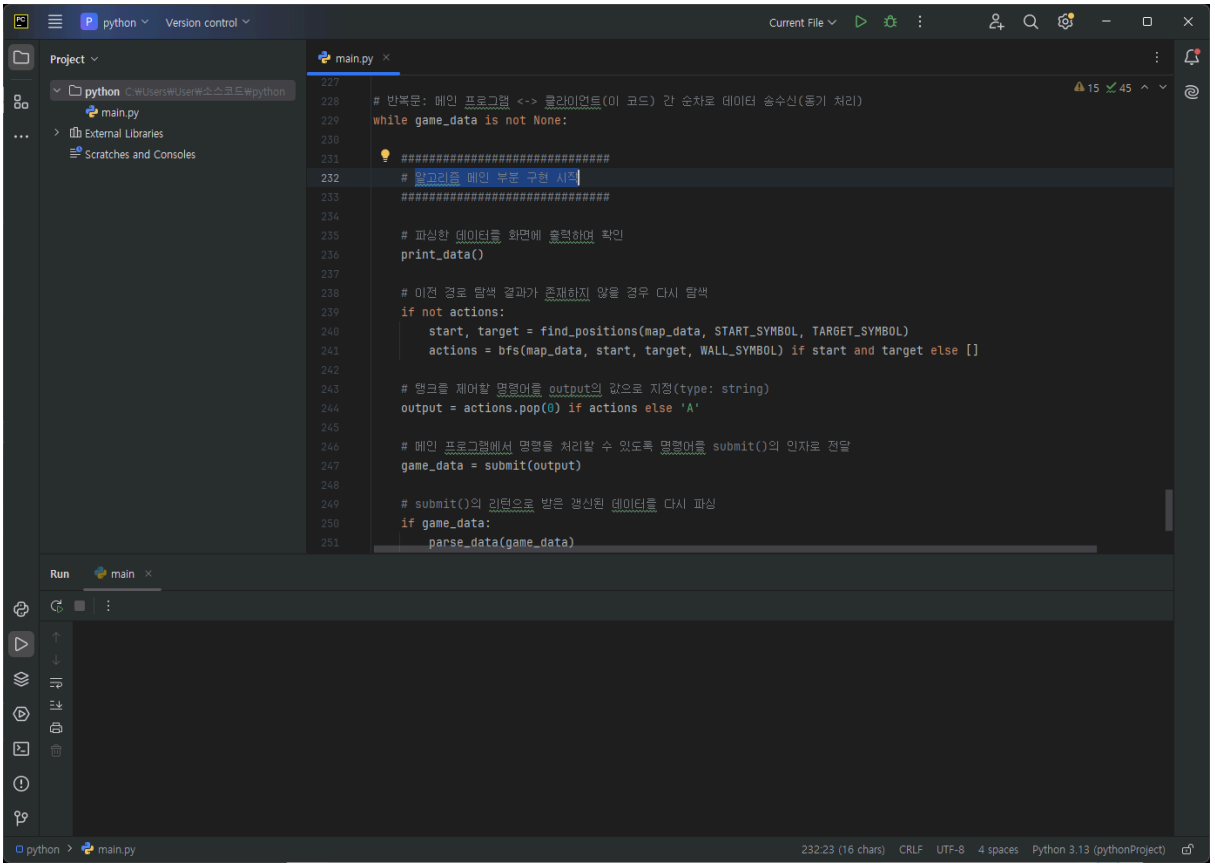
- while 반복문 바깥 부분
  - “162번 줄: 알고리즘 함수/메서드 부분 구현 시작” 부터
  - “225번 줄: 알고리즘 함수/메서드 부분 구현 끝” 까지
- while 반복문 내부
  - “232번 줄: 알고리즘 메인 부분 구현 시작” 부터
  - “254번 줄: 알고리즘 메인 부분 구현 끝” 까지



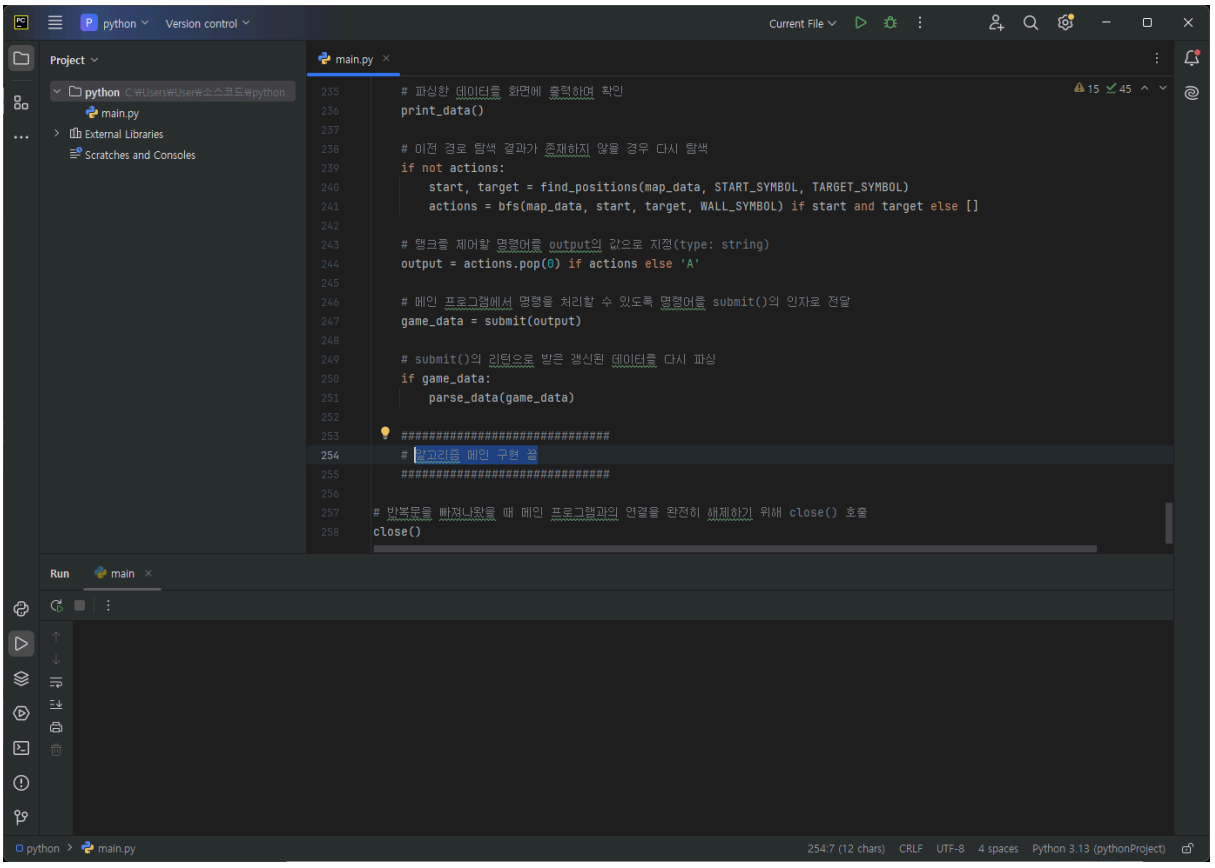
```
##############################
# 닉네임 설정 및 최초 연결
##############################
NICKNAME = '기본코드'
game_data = init(NICKNAME)

##################################
# 알고리즘 함수/메서드 부분 구현 시작
##################################

# 출발지와 목적지의 위치 찾기
def find_positions(grid, start_mark, goal_mark) -> tuple[tuple[int, int] | None, tuple[int, int] | None]:  2 usages
    rows, cols = len(grid), len(grid[0])
    start = goal = None

    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == start_mark:
                start = (row, col)

            elif grid[row][col] == goal_mark:
                goal = (row, col)

    return start, goal
```



```
# 경로 탐색 변수 정의
DIRS = [(0,1), (1,0), (0,-1), (-1,0)]
MOVE_CMDS = {0: "R A", 1: "D A", 2: "L A", 3: "U A"}
FIRE_CMDS = {0: "R F", 1: "D F", 2: "L F", 3: "U F"}
START_SYMBOL = 'M'
TARGET_SYMBOL = 'X'
WALL_SYMBOL = 'R'

# 최초 데이터 파싱
parse_data(game_data)

# 출발지점, 목표지점의 위치 확인
start, target = find_positions(map_data, START_SYMBOL, TARGET_SYMBOL)
if not start or not target:
    print("[ERROR] Start or target not found in map")
    close()
    exit()

# 최초 경로 탐색
actions = bfs(map_data, start, target, WALL_SYMBOL)

##################################
# 알고리즘 함수/메서드 부분 구현 끝
##################################
```



```
# 반복문: 메인 프로그램 <-> 클라이언트(이 코드) 간 순차로 데이터 송수신(동기 처리)
while game_data is not None:

    ##############################
    # 알고리즘 메인 부분 구현 시작
    ##############################

    # 파싱한 데이터를 화면에 출력하여 확인
    print_data()

    # 이전 경로 탐색 결과가 존재하지 않을 경우 다시 탐색
    if not actions:
        start, target = find_positions(map_data, START_SYMBOL, TARGET_SYMBOL)
        actions = bfs(map_data, start, target, WALL_SYMBOL) if start and target else []

    # 랭크를 제어할 명령어를 output의 값으로 지정(type: string)
    output = actions.pop(0) if actions else 'A'

    # 메인 프로그램에서 명령을 처리할 수 있도록 명령어를 submit()의 인자로 전달
    game_data = submit(output)

    # submit()의 리턴으로 받은 갱신된 데이터를 다시 파싱
    if game_data:
        parse_data(game_data)
```



```
    # 파싱한 데이터를 화면에 출력하여 확인
    print_data()

    # 이전 경로 탐색 결과가 존재하지 않을 경우 다시 탐색
    if not actions:
        start, target = find_positions(map_data, START_SYMBOL, TARGET_SYMBOL)
        actions = bfs(map_data, start, target, WALL_SYMBOL) if start and target else []

    # 랭크를 제어할 명령어를 output의 값으로 지정(type: string)
    output = actions.pop(0) if actions else 'A'

    # 메인 프로그램에서 명령을 처리할 수 있도록 명령어를 submit()의 인자로 전달
    game_data = submit(output)

    # submit()의 리턴으로 받은 갱신된 데이터를 다시 파싱
    if game_data:
        parse_data(game_data)

    ##############################
    # 알고리즘 메인 구현 끝
    ##############################

# 반복문을 빠져나왔을 때 메인 프로그램과의 연결을 완전히 해제하기 위해 close() 호출
close()
```

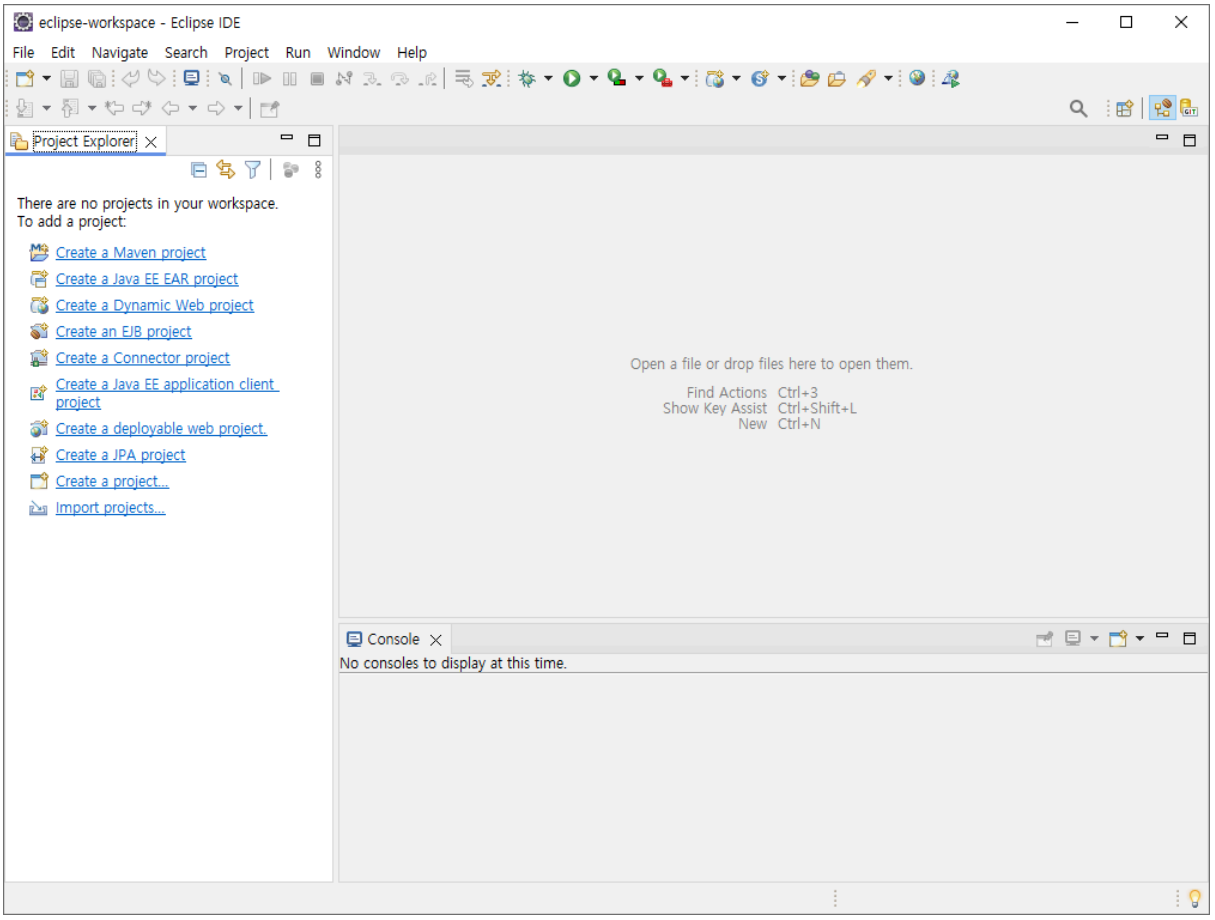
코드의 양을 반드시 표기한 줄 번호 안에만 맞게끔 작성해야 함을 의미하는 것이 아닙니다. 이는 제공된 소스 코드 기준의 줄 번호로, 코드의 양이 100줄, 1000줄 이상이 되어도 괜찮습니다. 여러분이 작성한 코드에 의해 해당 주석들의 줄 번호가 변경되는 것이 자연스러운 현상입니다.

함수, 전역 변수/상수 등의 자유로운 구현을 위해 구현 부분을 두 군데 표시해놓은 것으로, 메인 부분에만 코드를 작성해도 무방합니다.

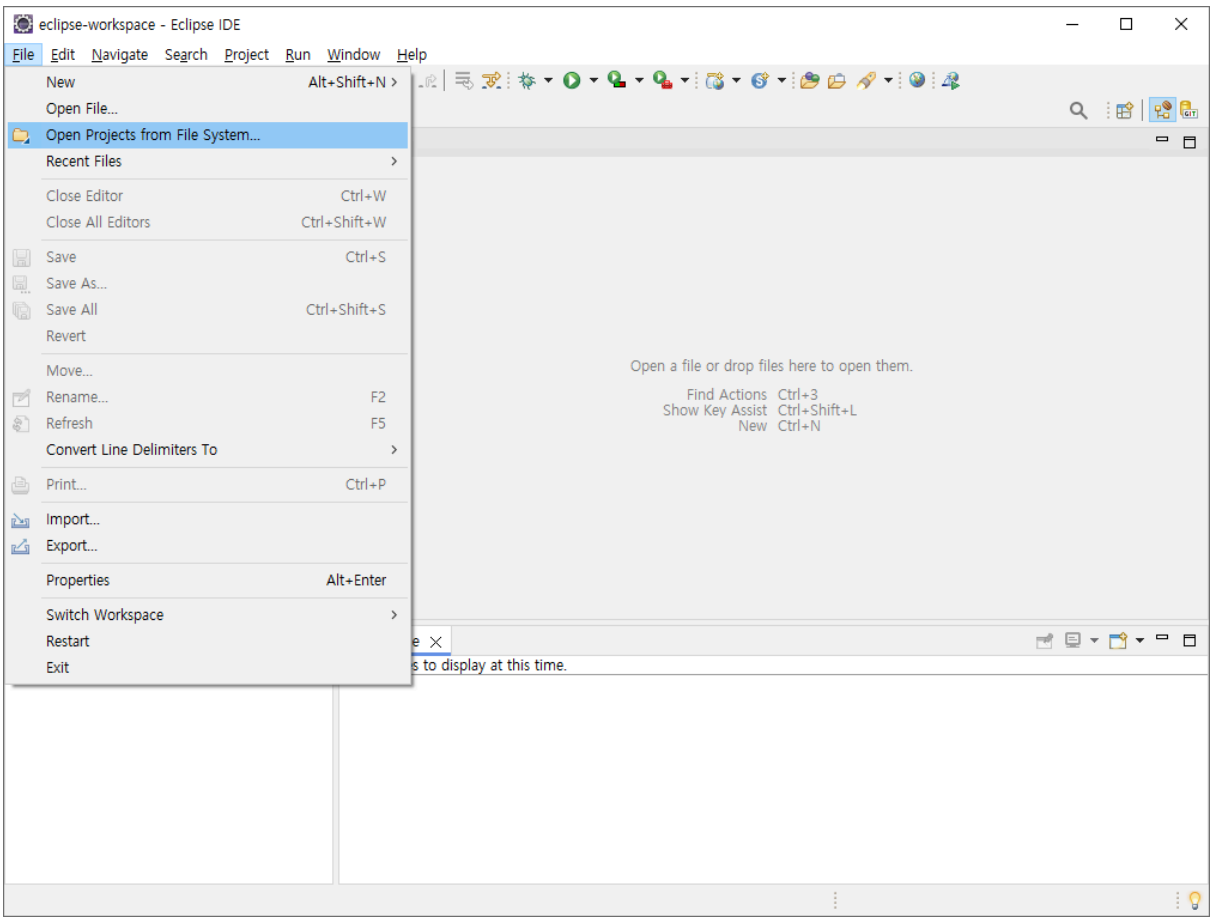
> ⚠️ 프로그램 통신부와 while 반복문의 역할이 중요하므로 가급적 위에 명시한 범위 내에서 코드를 작성할 것을 권장합니다.

Java

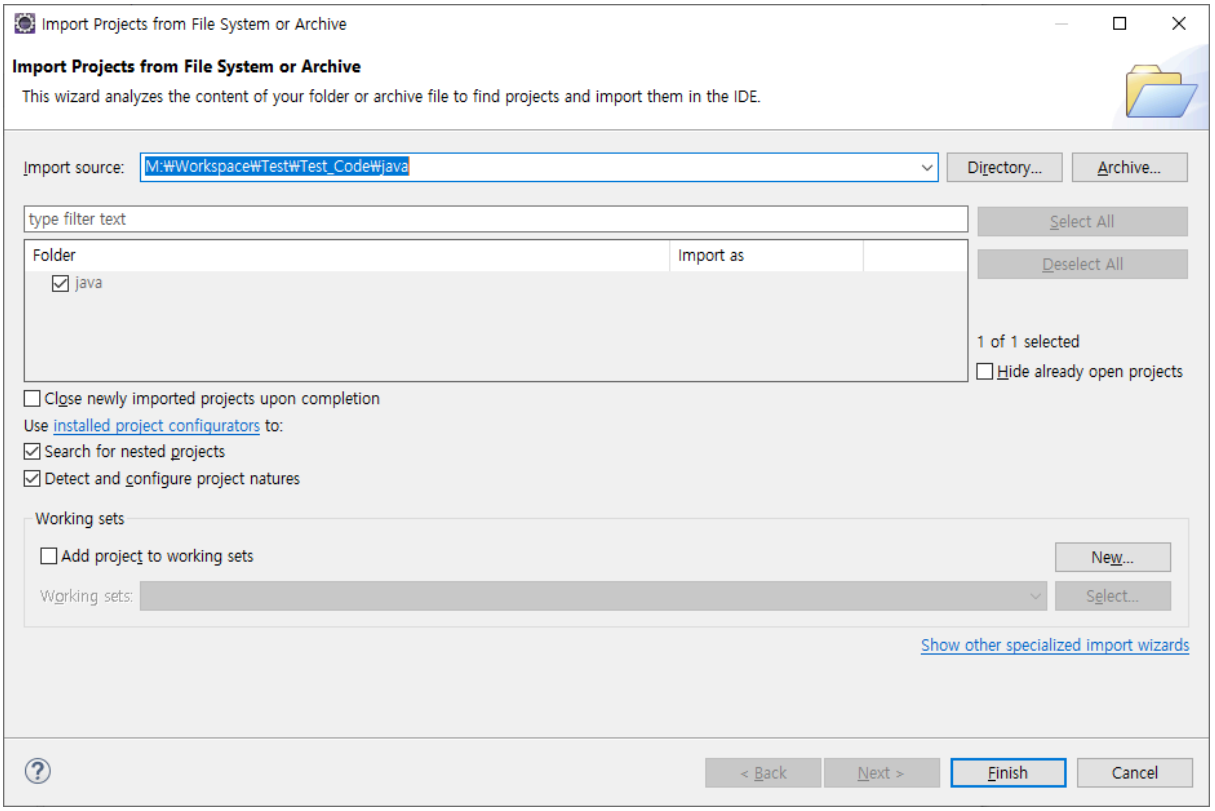
Eclipse를 실행합니다.



File - Open Projects from FIe System... 을 클릭합니다.



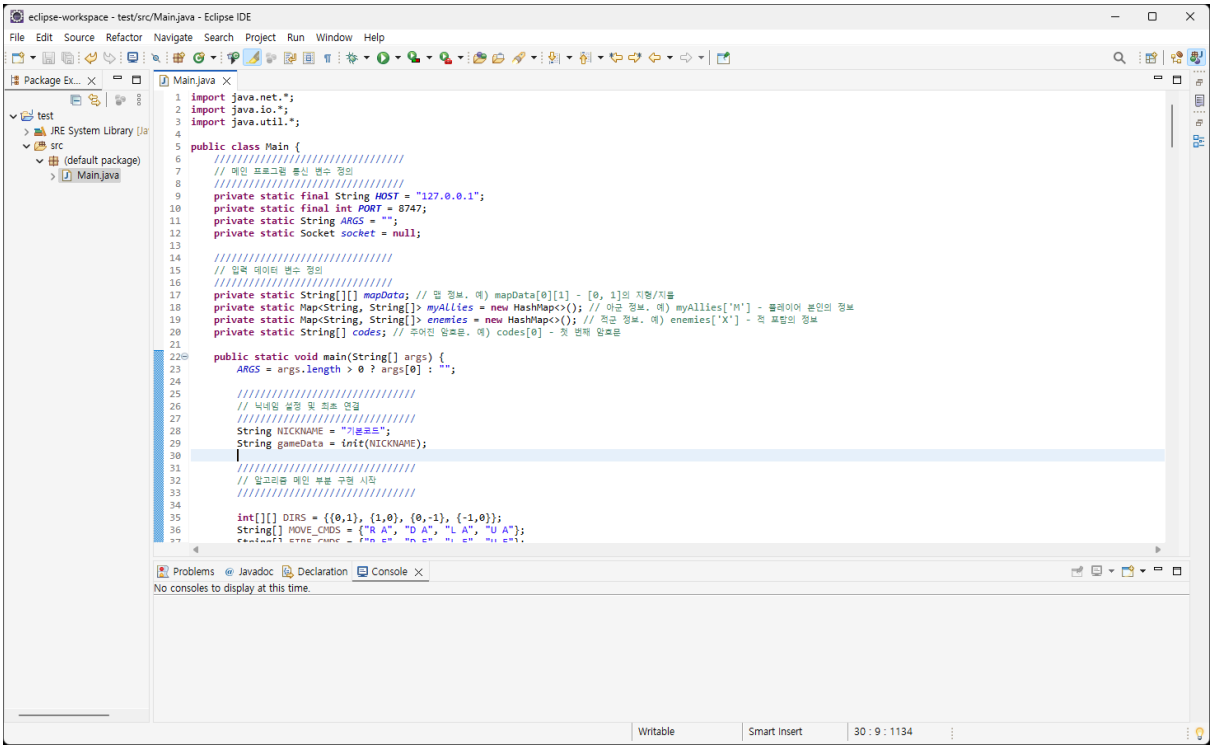
제공된 기본 코드 중 Java 폴더의 경로를 찾아 입력하고 Finish 버튼을 클릭합니다.



Main.java 파일이 (default package)로 인식되어 화면에 나타나는 것을 확인할 수 있습니다.

⚠ Java 소스 코드를 작성하고 제출할 때에는 다른 환경에서 컴파일 오류가 발생하지 않도록 package명을 비워놓아야 합니다. (default package) 상태를 그대로 유지해 주세요.

파일명을 수정하거나 새로 작성할 경우 class명이 파일명과 동일해야 함은 종종 실수할 수 있는 부분이니 유의해 주세요.



Main.java 파일을 더블 클릭하여 소스 코드를 살펴 봅니다.

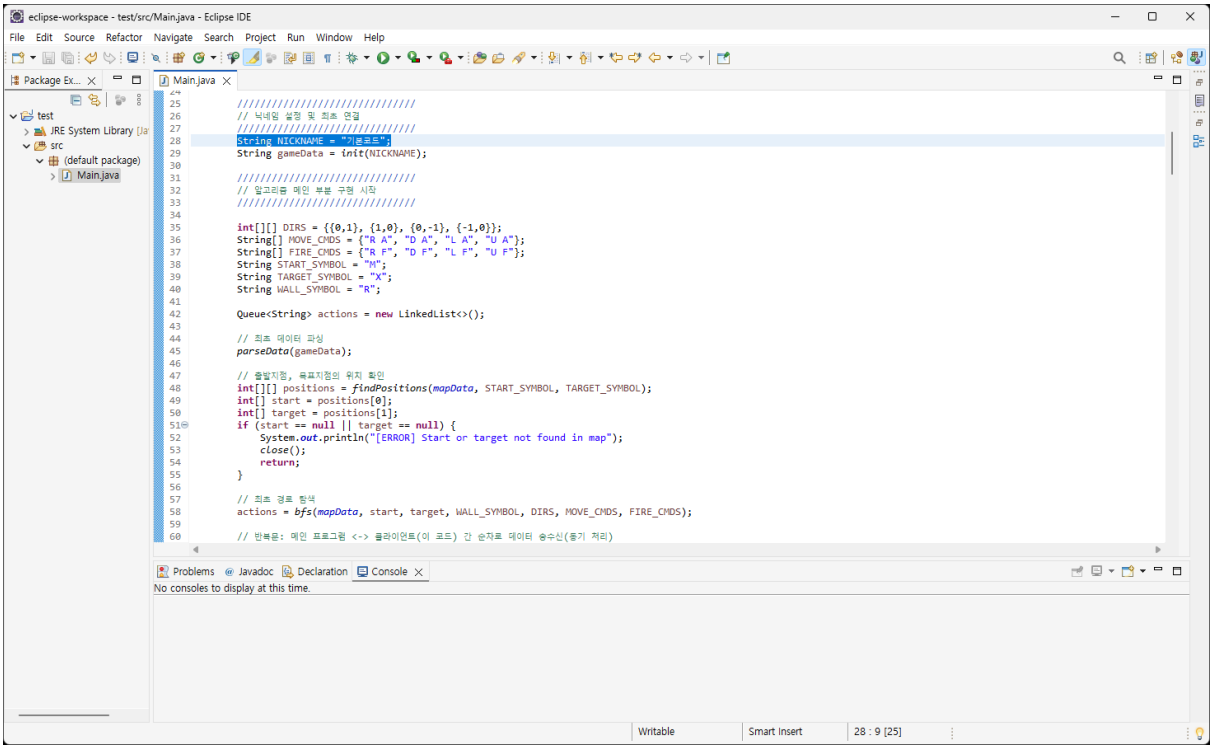
코드가 복잡하게 느껴질 수도 있지만, 대부분 배틀싸피와의 연동을 위해 작성된 코드이므로 직접 건드릴 일이 없습니다.

28번째 줄의 NICKNAME을 `기본코드` 대신 `지역반_이름`으로 수정합니다.

nickname은 배틀싸피에 Client가 연결되었을 때 본인임을 확인할 수 있는 수단입니다.

(nickname은 12자를 초과할 수 없으며, 초과 시 13자부터는 무시됩니다.)

- 예: `private static final String NICKNAME = "기본코드";` → `private static final String NICKNAME = "서울1_김싸피";`

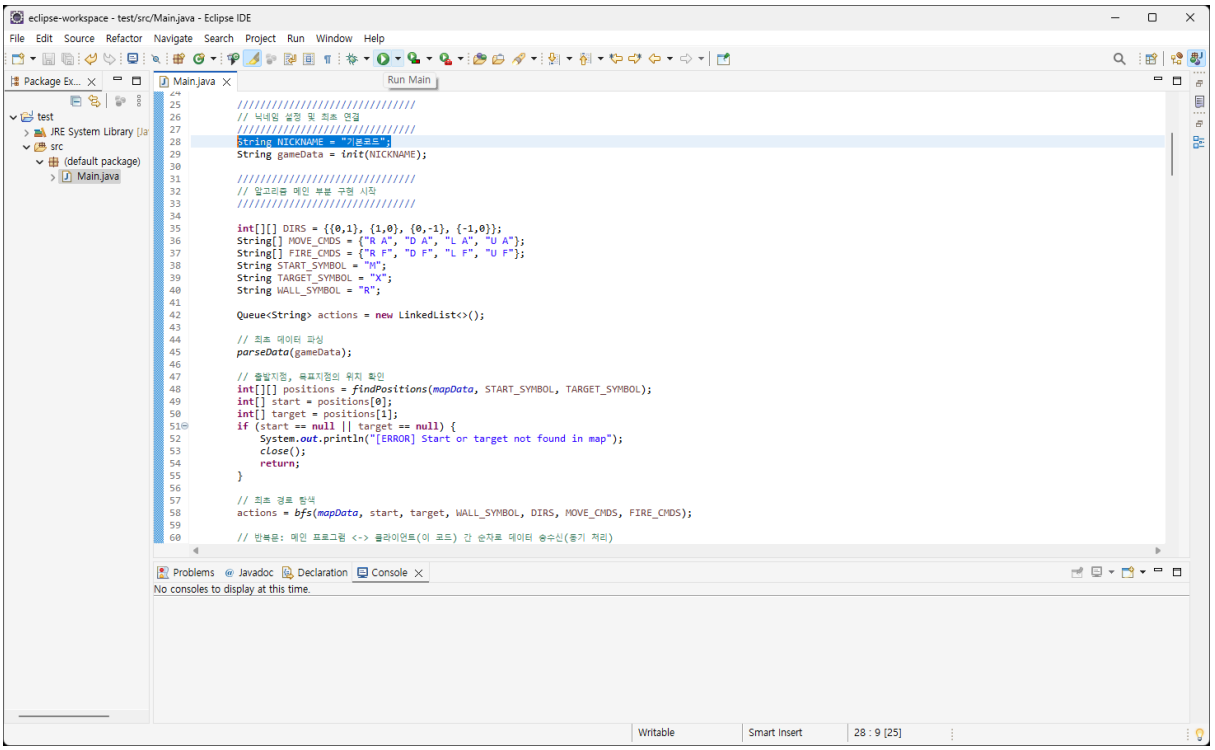


코드를 수정했다면 저장(Ctrl+S)합니다.

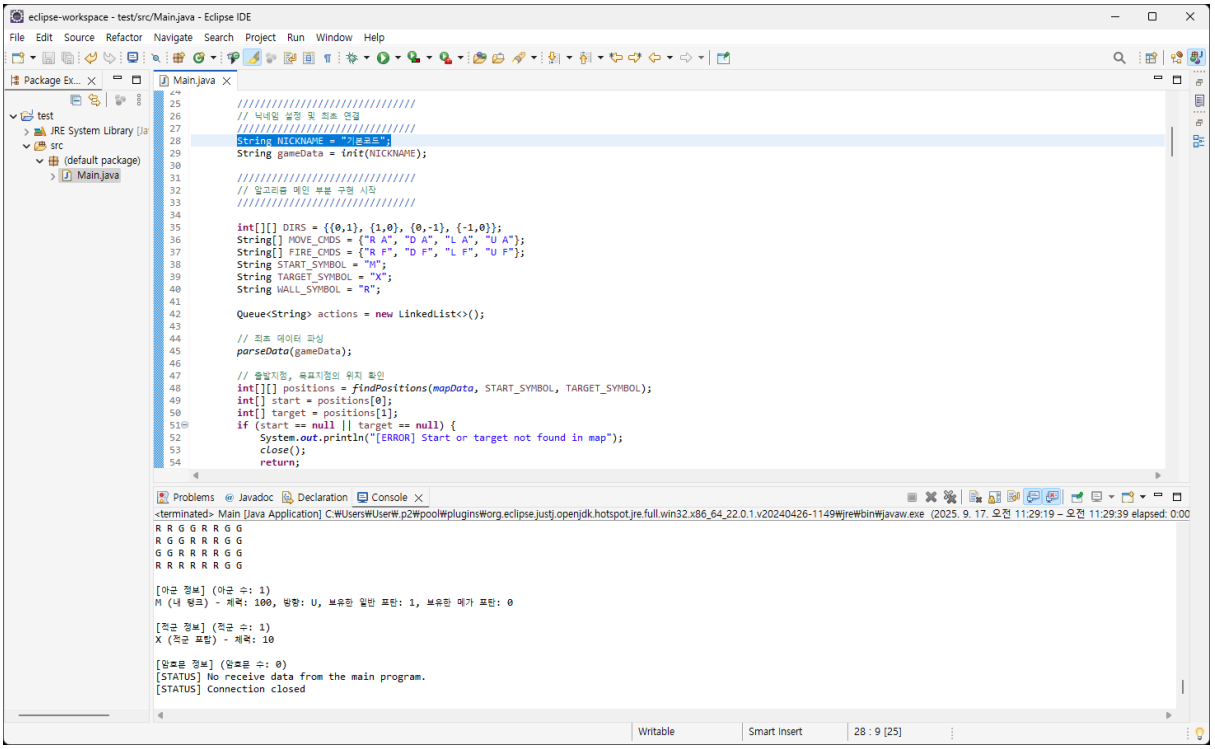
배틀싸피를 실행하고 스테이지1 연습 모드에 진입합니다.

Waiting for a player 메시지가 보이는 상태에서 Main.java를 실행합니다.

상단의 실행 아이콘을 클릭하면 됩니다.



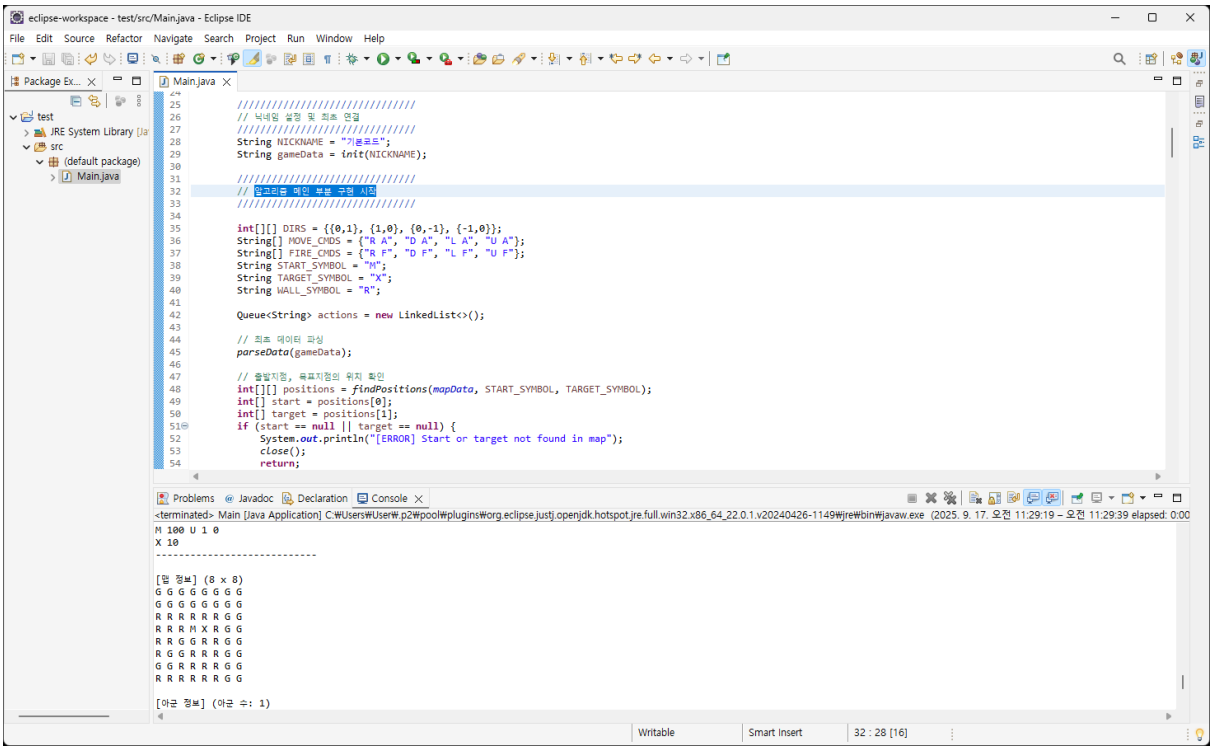
실행 결과를 보여주는 콘솔창이 나타나고 배틀싸피와 데이터 교환이 이루어지고 있음을 확인할 수 있습니다.

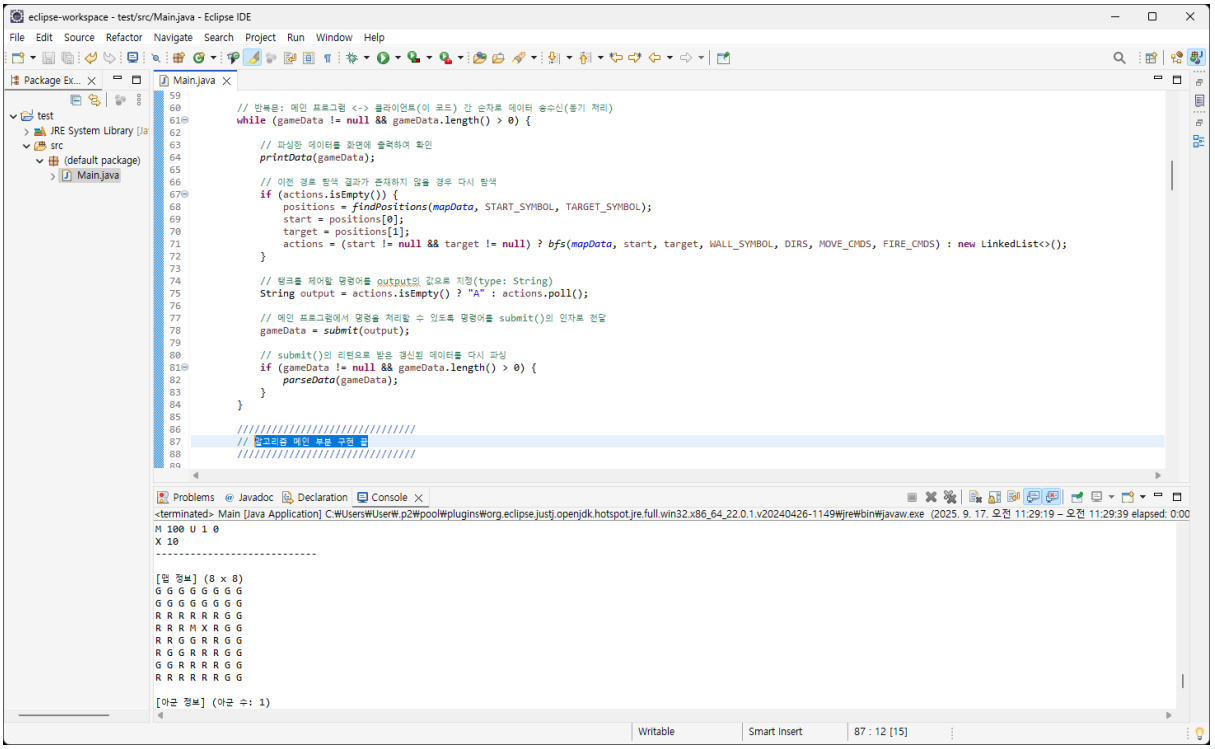


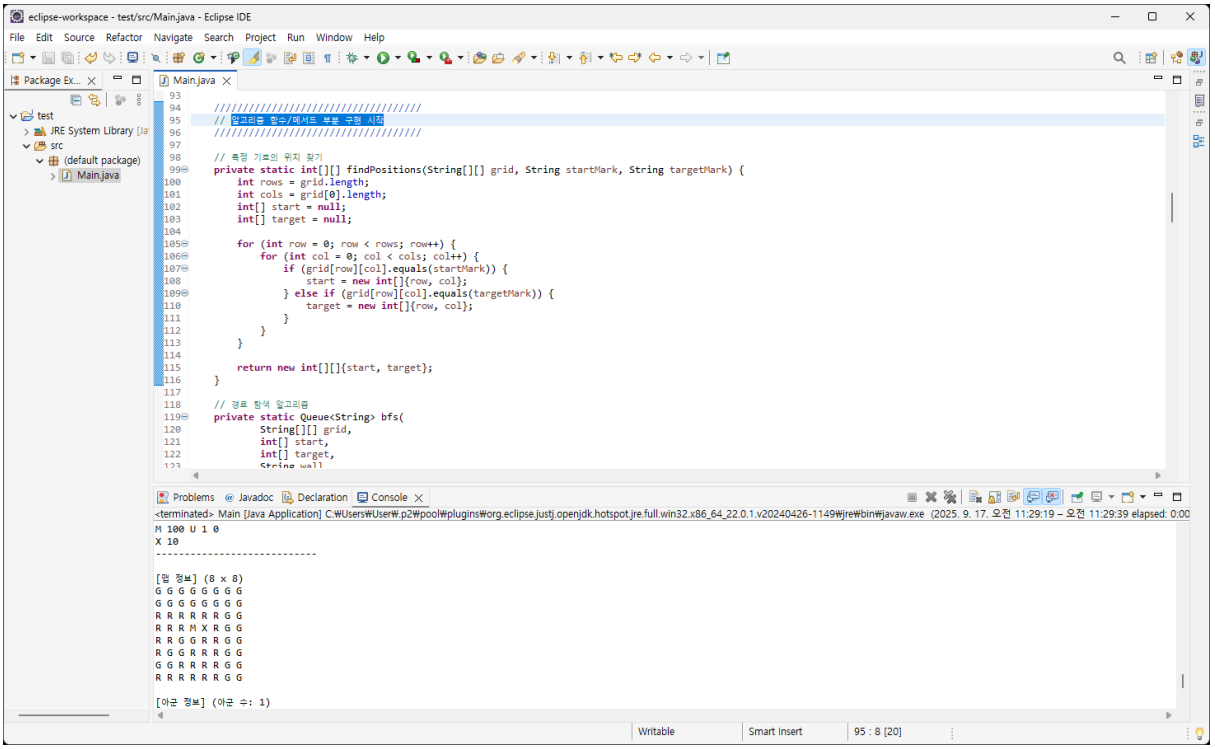
배틀싸피 화면도 방금 수정한 Nickname의 Client가 연결되었음을 표시해주고, 배틀싸피 ↔ Client가 데이터를 주고 받으며 탱크가 움직이기 시작합니다.

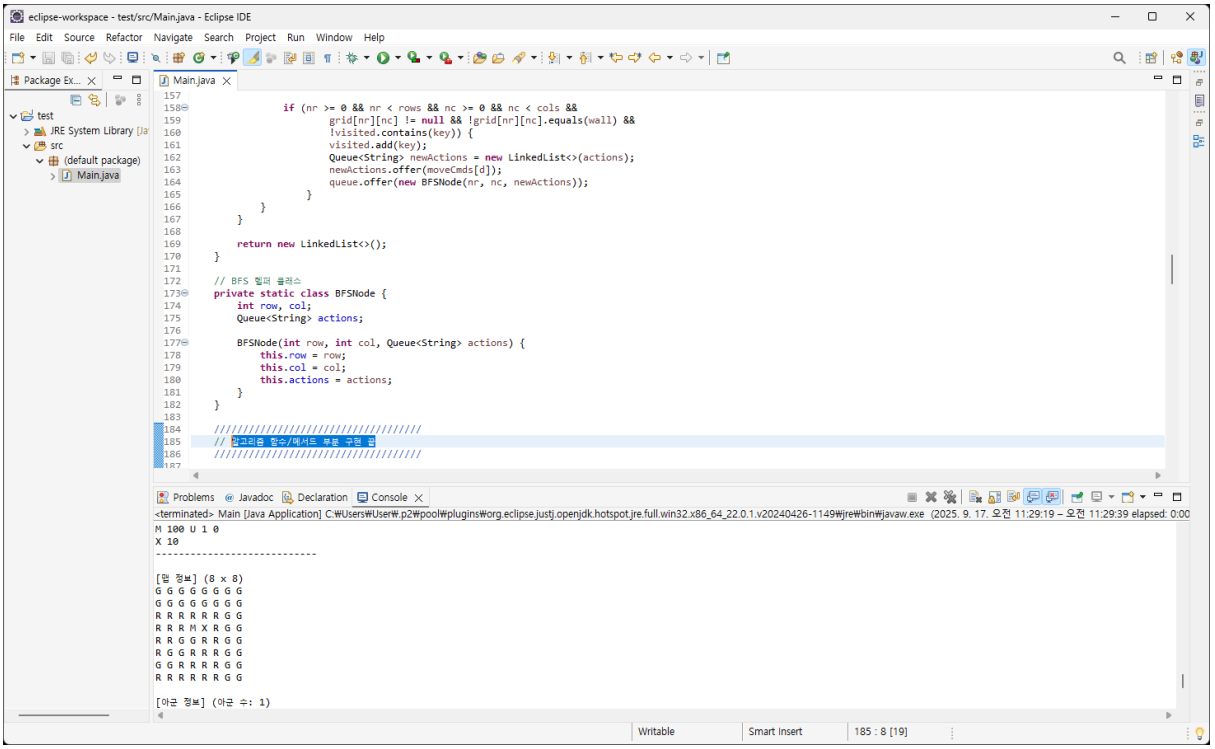
알고리즘을 구현할 수 있는 부분은

- Main 메서드 내부
  - “32번 줄: 알고리즘 메인 부분 구현 시작” 부터
  - “87번 줄: 알고리즘 메인 부분 구현 끝” 까지
- Main 메서드 바깥 부분
  - “95번 줄: 알고리즘 함수/메서드 부분 구현 시작” 부터
  - “254번 줄: 알고리즘 함수/메서드 구현 끝” 까지




eclipse-workspace - test/src/Main.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Ex...
Main.java
test
JRE System Library [Ja
src
(default package)
Main.java
// 반복문: 메인 프로그램 <-> 클라이언트(이 코드) 간 순차로 데이터 송수신(동기 처리)
while (gameData != null && gameData.length() > 0) {
// 파싱한 데이터를 화면에 출력하여 확인
printData(gameData);
// 이전 경로 탐색 결과가 존재하지 않을 경우 다시 탐색
if (actions.isEmpty()) {
positions = findPositions(mapData, START_SYMBOL, TARGET_SYMBOL);
start = positions[0];
target = positions[1];
actions = (start != null && target != null) ? bfs(mapData, start, target, WALL_SYMBOL, DIRS, MOVE_CMDS, FIRE_CMDS) : new LinkedList<>();
}
// 탱크를 제어할 명령어를 output의 값으로 지정(type: String)
String output = actions.isEmpty() ? "A" : actions.poll();
// 메인 프로그램에서 명령을 처리할 수 있도록 명령어를 submit()의 인자로 전달
gameData = submit(output);
// submit()의 리턴으로 받은 갱신된 데이터를 다시 파싱
if (gameData != null && gameData.length() > 0) {
parseData(gameData);
}
}
//////////////////////////////
// 알고리즘 메인 부분 구현 끝
//////////////////////////////
Problems Javadoc Declaration Console
<terminated> Main [Java Application] C:₩Users₩User₩.p2₩pool₩plugins₩org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_22.0.1.v20240426-1149₩jre₩bin₩javaw.exe (2025. 9. 17. 오전 11:29:19 – 오전 11:29:39 elapsed: 0:00
M 100 U 1 0
X 10
[맵 정보] (8 x 8)
G G G G G G G G
G G G G G G G G
R R R R R R G G
R R R M X R G G
R R G G R R G G
R G G R R R G G
G G R R R R G G
R R R R R R G G
[아군 정보] (아군 수: 1)
Writable
Smart Insert
87 : 12 [15]


eclipse-workspace - test/src/Main.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Ex...
Main.java
test
JRE System Library [Ja
src
(default package)
Main.java
//////////////////////////////////////
// 알고리즘 함수/메서드 부분 구현 시작
//////////////////////////////////////
// 특정 기호의 위치 찾기
private static int[][] findPositions(String[][] grid, String startMark, String targetMark) {
int rows = grid.length;
int cols = grid[0].length;
int[] start = null;
int[] target = null;
for (int row = 0; row < rows; row++) {
for (int col = 0; col < cols; col++) {
if (grid[row][col].equals(startMark)) {
start = new int[]{row, col};
} else if (grid[row][col].equals(targetMark)) {
target = new int[]{row, col};
}
}
}
return new int[][]{start, target};
}
// 경로 탐색 알고리즘
private static Queue<String> bfs(
String[][] grid,
int[] start,
int[] target,
Problems Javadoc Declaration Console
<terminated> Main [Java Application] C:₩Users₩User₩.p2₩pool₩plugins₩org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_22.0.1.v20240426-1149₩jre₩bin₩javaw.exe (2025. 9. 17. 오전 11:29:19 – 오전 11:29:39 elapsed: 0:00
M 100 U 1 0
X 10
[맵 정보] (8 x 8)
G G G G G G G G
G G G G G G G G
R R R R R R G G
R R R M X R G G
R R G G R R G G
R G G R R R G G
G G R R R R G G
R R R R R R G G
[아군 정보] (아군 수: 1)
Writable
Smart Insert
95 : 8 [20]



코드의 양을 반드시 표기한 줄 번호 안에만 맞게끔 작성해야 함을 의미하는 것이 아닙니다. 이는 제공된 소스 코드 기준의 줄 번호로, 코드의 양이 100줄, 1000줄 이상이 되어도 괜찮습니다. 여러분이 작성한 코드에 의해 해당 주석들의 줄 번호가 변경되는 것이 자연스러운 현상입니다.

함수, 전역 변수/상수 등의 자유로운 구현을 위해 구현 부분을 두 군데 표시해놓은 것으로, 메인 부분에만 코드를 작성해도 무방합니다.

⚠ 프로그램 통신부와 while 반복문의 역할이 중요하므로 가급적 위에 명시한 범위 내에서 코드를 작성할 것을 권장합니다.

입출력 데이터의 이해

입력 데이터

배틀싸피와 Client 소스 코드의 통신이 연결되면 **배틀싸피 프로그램은 현재의 지형, 지물 정보를 약속된 형태**로 Client에 보내줍니다. 여러분이 소스 코드를 실행하여 보게되는 콘솔창

에서는 이를 파싱(parsing)한 정보를 기본적으로 보여주도록 미리 코드가 작성되어 있습니다.

배틀싸피가 보내주는 데이터는 다음과 같은 형태이며, 각 언어별 **기본코드에서 gameData 또는 game_data**를 출력하면 직접 확인할 수 있습니다.

```
10 10 3 3 1
G R R G T G G R G X
G R R G T R G G G G
G R R G R R R G R R
F M G G G G R G G G
G R T T R G R G E1 G
G G T T R G G G G G
R G G G G R E2 R G R
G G R W W W G G G G
G G R W W W R R R G
H G G G M2 G G G G G
M 100 R 1 1
M2 10
H 10
E1 10
E2 10
X 10
AKWNSDN
```

데이터의 의미는 다음과 같습니다.

첫 번째 행

[맵의 세로크기] [맵의 가로크기] [아군 수] [적군 수] [현재 주어진 암호문 수]

이는 게임에 대한 기본 정보뿐만 아니라 다음 행부터 이어지는 나머지 데이터를 파싱하는 데에도 도움이 됩니다.

(설명 편의상 각각을 알파벳 H, W, M, E, S로 표현. 코드의 구현 내용과는 무관함)

- 이어지는 행부터 **R(맵의 세로(height))개의 행에 걸쳐** 맵 정보

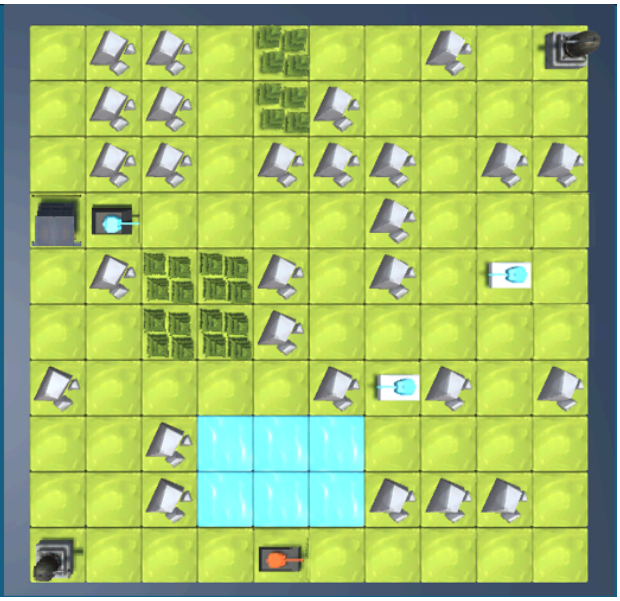
- 그 다음 행부터 **M(아군 수)개의 행에 걸쳐** 아군의 정보
- 그 다음 행부터 **E(적군 수)개의 행에 걸쳐** 적군의 정보
- 그 다음 행부터 **S(암호문 수)개의 행에 걸쳐** 현재 해독 가능한 암호문 수

H개의 행 (맵)

H*W 즉, 맵의 세로크기*맵의 가로크기를 알고 문자열 배열을 읽어올 수 있습니다.

입력데이터의 두 번째 행부터 10*10이 보여주는 데이터는 맵과 비교하면 다음과 같습니다.

G	R	R	G	T	G	G	R	G	X
G	R	R	G	T	R	G	G	G	G
G	R	R	G	R	R	R	G	R	R
F	M	G	G	G	G	R	G	G	G
G	R	T	T	R	G	R	G	G	G
G	G	T	T	R	G	G	G	G	G
R	G	G	G	G	R	G	R	G	R
G	G	R	W	W	W	G	G	G	G
G	G	R	W	W	W	R	R	R	G
H	G	G	G	M2	G	G	G	G	G



알파벳으로 표현되는 지형/기물의 종류는 다음과 같습니다.

- 지형
  - G: 일반 지형 - 탱크가 이동할 수 있음
  - S: 모래 - 탱크가 이동할 수 있음(통행 시 체력 -10)
  - W: 물 - 탱크가 이동할 수 없음
  - 지형은 공통적으로 포탄이 지나갈 수 있습니다.
- 지물(중립)
  - T: 나무 - 탱크가 지나갈 수 없음. 포탄으로 파괴 가능
  - R: 바위 - 탱크가 지나갈 수 없음. 포탄으로 파괴 불가
  - F: 보급시설 - 특수기능. 탱크가 지나갈 수 없고 파괴 불가
- 유닛(아군)
  - M: 플레이어 본인의 탱크
  - M1~M3: 아군의 탱크
  - H: 아군의 포탑
- 유닛(적군)
  - E1~E3: 적군의 탱크
  - X: 적의 포탑

상기 지형/지물은 팀 배틀 모드에서는 모두 등장할 수 있으나, 스테이지 모드에서는 일부만 등장합니다.

M개의 행 (아군)

본인을 포함한 아군의 탱크와 포탑 정보를 얻습니다.

- 본인의 탱크에 대한 정보는 상대적으로 자세히 얻습니다.
  - M [체력] [바라보는 방향] [보유한 일반 포탄 수] [보유한 메가 포탄 수]
- 아군 탱크에 대한 정보는 체력 정보만 얻습니다.
  - M1 [체력]
  - M2 [체력]
  - M3 [체력]
- 아군 포탑에 대한 정보는 체력 정보만 얻습니다.
  - H [체력]

E개의 행 (적군)

적군의 탱크화 포탑 정보를 얻습니다.

- 적군 탱크에 대한 정보는 체력 정보만 얻습니다.
  - E1 [체력]
  - E2 [체력]
  - E3 [체력]
- 적군 포탑에 대한 정보는 체력 정보만 얻습니다.
  - X [체력]

S개의 행 (암호문)

현재 본인의 위치가 상하좌우로 1개 이상의 보급시설과 인접한 경우 해독을 시도할 수 있는 암호문이 보급 시설 개수만큼 주어집니다.

현재 버전에서는 보급 시설이 2칸 이내로 인접한 경우가 없으므로, 플레이어의 탱크가 어느 위치에 있든 0 또는 1의 값을 받게 됩니다.

0인 경우는 공란, 1인 경우는 위처럼 “AKWNSDN”와 같은 암호문이 입력 데이터에 포함되어 있습니다.

입력 데이터는 여러분의 편의와 코드 구현 속도 향상을 위해 각 언어별 기본 코드에 적합한 변수명을 지정하여 파싱하도록 되어 있습니다.

따라서 기본 코드를 처음 실행하면 콘솔창에서 각 데이터의 의미와 값이 출력됨을 보게될 것이며, 배틀싸피 통신의 큰 구조를 해치지 않는 범위 내에서 여러분이 원하는 방식으로 다시 파싱해도 됩니다.

다만, 오류의 여지를 최소화 하기 위해 주어진 while 반복문 내에서만 코드를 작성할 것을 권장합니다.(외부에 함수를 선언하고 반복문 안에서 호출하는 경우는 코드의 큰 틀을 해치는 것이 아니므로 문제 없음)

출력 데이터(=커맨드)

수신한 입력 데이터를 바탕으로 현재 게임의 상태를 파악하고 탱크가 적절한 행동을 할 수 있도록 여러분이 지시를 내려주어야 합니다.

단, 지시는 미리 작성한 코드의 알고리즘에 의해 내려져야 하며 한 번 실행한 코드는 해당 스테이지/배틀이 종료될 때까지 여러분이 개입할 수 없습니다.

따라서 여러분의 코딩과 알고리즘 실력이 상당히 필요한 과정임을 체감하게 될 것입니다.

다행이도, 배틀싸피에 내릴 수 있는 커맨드의 유형은 많지 않으며 형태도 매우 간단합니다.

- 탱크를 전진시키고 싶다면, A
  - 방향을 회전해서 전진시키고 싶다면, U A, D A, L A, R A 중 하나
- 일반 포탄을 발사하고 싶다면, F
  - 방향을 회전해서 발사하고 싶다면, U F, D F, L F, R F 중 하나
  - 단, 일반 포탄을 1개 이상 보유중인지 꼭 확인 먼저 할 것
- 메가 포탄을 발사하고 싶다면, F M
  - 방향을 회전해서 발사하고 싶다면, U F M, D F M, L F M, R F M 중 하나
  - 단, 메가 포탄을 1개 이상 보유중인지 꼭 확인 먼저 할 것
- 암호를 해독하고 싶다면, G와 해독문을 같이
  - 예: G HELLO
- 제자리에서 대기하고 싶다면, S

💡 일반 포탄과 메가 포탄의 차이는?

두 종류의 포탄 모두 사거리는 3칸으로 동일합니다.

일반 포탄은 상대에게 30의 데미지를 입히고
메가 포탄은 상대에게 70의 데미지를 입히는 차이가 있습니다.

⚠️ 일반 포탄과 메가 포탄의 수량은 별개로 관리되며, 발사할 때 자동으로 전환되지 않습니다.

F M 커맨드를 사용했지만 메가 포탄을 1개도 보유하고 있지 않다면, 발사할 수 있는 메가 포탄이 없으므로 실패한 명령이 되어 턴 수만 차감되므로 실수하지 않도록 주의해야 합니다.
(일반 포탄이 1개 이상 남아있어도 대신 발사되지는 않습니다.)

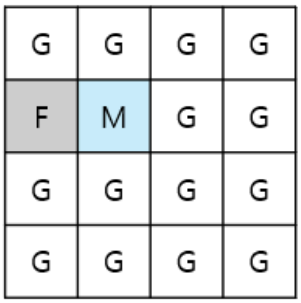
배틀싸피는 턴제로 진행되기 때문에 한 턴, 한 턴의 움직임이 매우 중요합니다. 방향 전환과 전진/발사를 결합하여 사용할 수 있는 것은 턴 소모를 줄이고 효율적인 플레이를 할 수 있도록 하기 위함입니다.

메가 포탄 얻기 (보급 시설)

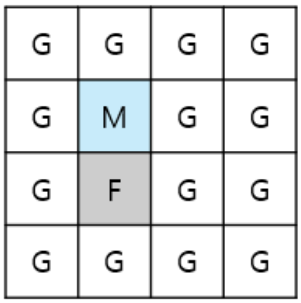
스테이지 모드 또는 팀 배틀 모드를 시작할 때 기본적으로 주어지는 포탄의 수량이나 파괴력은 충분할 수도 있지만 때로는 부족할 수도 있습니다.

특정 스테이지 및 팀 배틀 모드에 등장하는 보급 시설은 여러분이 더 계산적인 플레이를 할 수 있는 기회를 제공합니다.

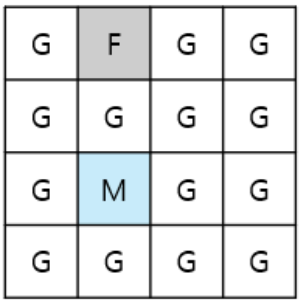
현재의 나의 탱크(M) 위치가 보급 시설(F)과 상하좌우로 인접한 상태라면, 입력 데이터에 암호문을 포함하여 수신합니다.



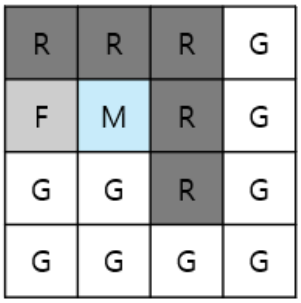
탱크(M)와 보급시설(F)이
상하좌우 인접하므로
해독 가능



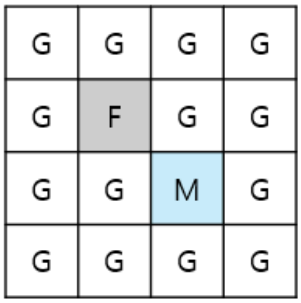
탱크(M)와 보급시설(F)이
상하좌우 인접하므로
해독 가능



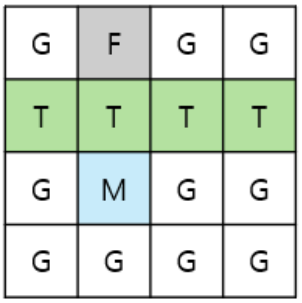
탱크(M)와 보급시설(F)이
상하좌우 인접하지 않으므로
해독 불가능
(위로 이동 후 다음 턴에 가능)



탱크(M)와 보급시설(F)이
상하좌우 인접하므로
해독 가능



탱크(M)와 보급시설(F)이
상하좌우 인접하지 않으므로
해독 불가능
(대각선 X. 이동 후 가능)



탱크(M)와 보급시설(F)이
인접하지 않으므로
해독 불가능
(나무 파괴 및 이동 후 가능)

현재 탱크가 바라보는 방향이 어느쪽이든 중요하지 않습니다. 위치가 보급 시설과 인접하기만 하면 보급 시설의 반대 방향을 바라 본 상태에서도 G 커맨드를 사용할 수 있습니다.

카이사르 암호

배틀싸피에서는 가장 고전적인 암호 방식으로 잘 알려진 카이사르 암호를 사용하여 보급 시설의 암호문을 해독할 수 있습니다.

> 💡 **카이사르 암호란?**
>
> 암호화하고자 하는 내용을 알파벳별로 일정한 거리만큼 밀어서 다른 알파벳으로 치환하는 방식
>
> 예를 들어 뒤로 세 글자씩 밀어내기로 합의된 원문과 암호문은 다음과 같습니다.
> `원문: COME TO ROME`
> `암호문: FRPH WR URPH`

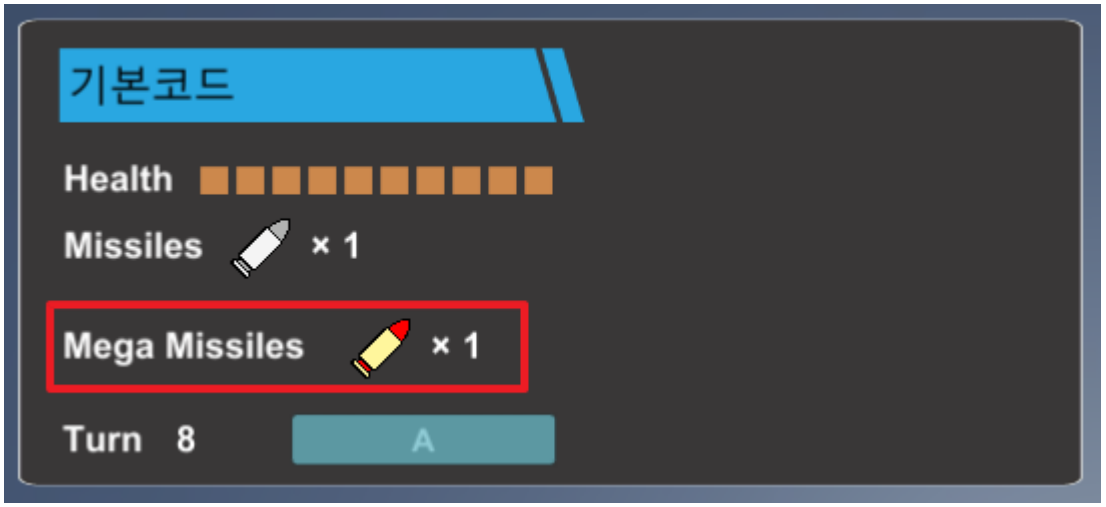
입력 데이터로 수신한 암호문이 SRKKCVJJRWP일 때, 해독한 문자열이 BATTLESSAFY가 되었다면, 커맨드를 다음과 같이 지정하면 되겠습니다.

- G BATTLESSAFY

즉, 기본 코드의 구조를 기준으로 보았을 때 , 언어마다 조금씩 다르겠지만 대개 다음과 유사한 모양이 될 것입니다.

```
output = "G BATTLESSAFY"
submit(output)
```

암호 해독에 성공했다면 메가 포탄의 수량이 증가한 것을 확인할 수 있습니다.



보급 시설에서 제시하는 암호문은 입력 데이터를 받을 때마다 다른 문장을 받게 됩니다. 한 번 정해진 카이사르 암호의 치환 간격은 동일하게 유지되므로 규칙을 한 번 찾아낸 후에는 암호를 해독하는 일이 어렵지 않을 것입니다.

보급 시설은 암호가 해독된 후에도 사라지지 않고 계속 암호문을 제공하기 때문에, 제자리에서 메가 포탄을 여러 번 획득하는 것도 가능합니다. 다만 시도한 만큼 턴이 소모되기 때문에 불필요하게 많은 메가 포탄을 획득하는 것은 오히려 손해일 수도 있습니다. 또한 암호 해독을 시도했지만 해독문이 틀린 경우에도 아무런 소득 없이 턴이 차감되므로 슬기롭게 플레이해야 합니다.

※ 별첨. 사용 가능한 커맨드 모음

	방향 + 행동 결합	행동만	행동 + 옵션 결합	방향 + 행동 + 옵션 모두 결합
전진		A		
	U A			
	D A			
	L A			
	R A			
발사		F	F M	
	U F			U F M
	D F			D F M
	L F			L F M
	R F			R F M
해독			G 해독문	
제자리 대기		S		